C++ Programming

For Embedded Systems | PROG1353 | New Brunswick Community College

David Burchill

2026

Contents

About This Book **8**
A note on AI assistance . . . 8

How to Use This Book **9**
How the book is organized . . . 9
Coming from C . . . 9
How to study so it sticks . . . 10

I The Machine and the Build **11**

1 The Toolchain and the Build **12**
1.1 What You'll Be Able to Do . . . 12
1.2 No Runtime, No Collector . . . 12
1.3 Your First Program . . . 12
1.4 Compiling and Running from the Terminal . . . 13
1.5 Reading a Compiler Error . . . 13
1.6 Compile Time vs Run Time . . . 14
1.7 One Command, Four Stages . . . 15
1.8 Doing It By Hand . . . 15
1.9 Why CMake . . . 16
1.10 Worked Example . . . 17
1.11 Predict Before You Run . . . 18
1.12 Common Errors . . . 18
1.13 Key Terms . . . 19
1.14 Exercises . . . 20
1.15 Self-Check . . . 21
1.16 Summary . . . 22

2 Types, Bits, and the Preprocessor **23**
2.1 What You'll Be Able to Do . . . 23
2.2 The Preprocessor: A Program That Edits Your Program . . . 23
2.3 Seeing It Instead of Believing It . . . 23
2.4 `#define` vs `const` vs `constexpr` . . . 24
2.5 The Macro Pitfall . . . 24
2.6 Include Guards . . . 25
2.7 `int` Is Not a Promise . . . 25

2.8 Fixed-Width Types Say What They Mean . . . 26
2.9 The Promotion Trap . . . 26
2.10 Bit Manipulation: Eight Independent Switches . . . 26
2.11 Read-Modify-Write . . . 27
2.12 The Two Errors Everyone Makes . . . 27
2.13 Worked Example . . . 28
2.14 Common Errors . . . 29
2.15 Key Terms . . . 29
2.16 Exercises . . . 30
2.17 Self-Check . . . 31
2.18 Summary . . . 32

II Memory **33**

3 The Stack and the Heap **34**
3.1 What You'll Be Able to Do . . . 34
3.2 Two Places a Value Can Live . . . 34
3.3 Deterministic Destruction . . . 35
3.4 Stack Frames . . . 36
3.5 The Debugger as a Way of Looking . . . 36
3.6 The Heap: `new` and `delete` . . . 37
3.7 A Leak, On Purpose . . . 38
3.8 Why Embedded Work Avoids the Heap . . . 38
3.9 Worked Example . . . 38
3.10 Predict Before You Run . . . 39
3.11 Common Errors . . . 40
3.12 Key Terms . . . 40
3.13 Exercises . . . 41
3.14 Self-Check . . . 43
3.15 Summary . . . 43

4 Pointers, References, and `const` **45**
4.1 What You'll Be Able to Do . . . 45
4.2 A Pointer Is a Variable That Holds an Address . . . 45
4.3 Pointer Arithmetic Is Not Integer Arithmetic . . . 46
4.4 Array Decay . . . 46
4.5 Three Ways to Pass an Argument . . . 47
4.6 `const`, and Reading It Correctly . . . 47
4.7 `const T&` — The One You Will Actually Write . . . 48
4.8 Memory-Mapped I/O . . . 49
4.9 `volatile` — What It Actually Promises . . . 49
4.10 Worked Example . . . 49
4.11 Predict Before You Run . . . 50
4.12 Common Errors . . . 51
4.13 Key Terms . . . 51
4.14 Exercises . . . 52
4.15 Self-Check . . . 54

4.16 Summary 55

III Objects and Resources **56**

5 Classes, Structs, and RAII **57**
5.1 What You'll Be Able to Do 57
5.2 `struct` and `class` Are the Same Thing 57
5.3 Constructors, Destructors, and Member Init Lists 57
5.4 Order: Members First, Object Last 58
5.5 `this` and Static Members 59
5.6 RAII 59
5.7 Alignment and Padding 60
5.8 `#pragma pack`, and When It Is Justified 61
5.9 Worked Example 61
5.10 Predict Before You Run 62
5.11 Common Errors 63
5.12 Key Terms 64
5.13 Exercises 64
5.14 Self-Check 66
5.15 Summary 67

6 Smart Pointers, Copy Semantics, and Custom Allocation **68**
6.1 What You'll Be Able to Do 68
6.2 `std::unique_ptr` — Ownership You Cannot Forget 68
6.3 Move: How Ownership Travels 69
6.4 Try to Copy One, and Read the Error 69
6.5 The Copy You Did Not Write 70
6.6 The Rule of Three, Five, and Zero 71
6.7 `std::shared_ptr` — Ownership Shared, and Counted 72
6.8 `weak_ptr`, and the Cycle 73
6.9 Custom Allocation, and Why an Embedded Course Cares 73
6.10 Worked Example 74
6.11 Predict Before You Run 75
6.12 Common Errors 76
6.13 Key Terms 77
6.14 Exercises 77
6.15 Self-Check 79
6.16 Summary 80

7 Polymorphism and Abstract Base Classes **81**
7.1 What You'll Be Able to Do 81
7.2 What You Already Know, and What Is Different 81
7.3 An Abstract Base Class 82
7.4 The vtable, and What It Costs 82
7.5 Runtime versus Compile-Time Polymorphism 83
7.6 The `virtual` Destructor Rule 83
7.7 Object Slicing 84

7.8 Two SOLID Principles, Named Where They Were Earned . . . 84
7.9 Worked Example . . . 85
7.10 Predict Before You Run . . . 86
7.11 Common Errors . . . 86
7.12 Key Terms . . . 87
7.13 Exercises . . . 87
7.14 Self-Check . . . 89
7.15 Summary . . . 90

IV Generic Programming 91

8 Templates and Generic Programming 92
8.1 What You'll Be Able to Do . . . 92
8.2 You Already Have the Idea . . . 92
8.3 A Function Template . . . 93
8.4 Instantiation Is the Whole Idea . . . 93
8.5 Code Bloat, Measured . . . 93
8.6 `std::array<T, N>` . . . 94
8.7 Reading Template Errors . . . 94
8.8 Templates Live in Headers . . . 95
8.9 The Class Template You Are About to Be Graded On . . . 96
8.10 Worked Example . . . 96
8.11 Predict Before You Run . . . 97
8.12 Common Errors . . . 98
8.13 Key Terms . . . 99
8.14 Exercises . . . 99
8.15 Self-Check . . . 100
8.16 Summary . . . 101

9 The STL: Containers, Iterators, and Algorithms 102
9.1 What You'll Be Able to Do . . . 102
9.2 Four Containers, and How to Choose . . . 102
9.3 The Tension, Named Out Loud . . . 103
9.4 Iterators . . . 103
9.5 Iterator Invalidation . . . 103
9.6 `at()` versus `[]` . . . 104
9.7 Algorithms Instead of Loops . . . 104
9.8 Lambdas, Briefly . . . 104
9.9 `auto` and Structured Bindings . . . 105
9.10 Ordered or Not: The Choice with a Visible Consequence . . . 105
9.11 Worked Example . . . 105
9.12 Predict Before You Run . . . 106
9.13 Common Errors . . . 106
9.14 Key Terms . . . 107
9.15 Exercises . . . 107
9.16 Self-Check . . . 109
9.17 Summary . . . 109

V Robustness **110**

10 Error Handling and Unit Testing **111**
10.1 What You'll Be Able to Do . . . 111
10.2 The Mechanism . . . 111
10.3 The Standard Hierarchy . . . 111
10.4 A Custom Exception . . . 112
10.5 Exception Safety, and Why There Is No `finally` . . . 112
10.6 Catch by Reference — and What Slicing Costs You . . . 113
10.7 `noexcept` . . . 114
10.8 The Embedded Caveat, Stated Honestly . . . 114
10.9 Why Test at All . . . 114
10.10 Catch2, Wired into CMake . . . 114
10.11 Writing a Test . . . 115
10.12 Read a Failure . . . 116
10.13 Worked Example . . . 117
10.14 Common Errors . . . 118
10.15 Key Terms . . . 119
10.16 Exercises . . . 119
10.17 Self-Check . . . 121
10.18 Summary . . . 122

11 File I/O and Streams **123**
11.1 What You'll Be Able to Do . . . 123
11.2 Text Streams, and the Failure Everyone Skips . . . 123
11.3 Text or Binary — the Decision Is About Who Reads It . . . 124
11.4 The Record on Disk Is Not the Struct in Memory . . . 124
11.5 Field by Field, with Explicit Byte Order . . . 124
11.6 The Contrast: What Everyone Else Does . . . 125
11.7 Buffering, and Why Output Disappears . . . 126
11.8 `std::streambuf`, in One Paragraph . . . 126
11.9 A User-Defined `operator<<` . . . 126
11.10 Worked Example . . . 127
11.11 Predict Before You Run . . . 128
11.12 Common Errors . . . 129
11.13 Key Terms . . . 129
11.14 Exercises . . . 130
11.15 Self-Check . . . 131
11.16 Summary . . . 132

VI Concurrency and Performance **133**

12 Concurrency and Multithreading **134**
12.1 What You'll Be Able to Do . . . 134
12.2 One Thread, Then Two . . . 134
12.3 The Race . . . 134
12.4 `std::mutex` and `lock_guard` . . . 135

12.5 Hold the Lock for as Little as Possible . . . 136
12.6 A Condition Variable . . . 136
12.7 Deadlock, and the Discipline That Prevents It . . . 137
12.8 `std::atomic`, Surveyed . . . 137
12.9 `volatile` Is Not a Synchronization Tool . . . 138
12.10Worked Example . . . 138
12.11Predict Before You Run . . . 139
12.12Common Errors . . . 140
12.13Key Terms . . . 140
12.14Exercises . . . 141
12.15Self-Check . . . 142
12.16Summary . . . 143

13 Architecture and Performance **144**
13.1 What You'll Be Able to Do . . . 144
13.2 SOLID, and the Two You Already Built . . . 144
13.3 MVC on a Simulated Device Panel . . . 145
13.4 Measure, Do Not Guess . . . 146
13.5 The Measurement, and Its Surprise . . . 146
13.6 What to Do with a Measurement . . . 147
13.7 Where the Debugger Fits . . . 148
13.8 Worked Example . . . 148
13.9 Common Errors . . . 149
13.10Key Terms . . . 149
13.11Exercises . . . 150
13.12Self-Check . . . 151
13.13Summary . . . 152

VII Integration **153**

14 Integration **154**
14.1 What This Chapter Is . . . 154
14.2 The Capstone Artifact . . . 154
14.3 1. Memory — Chapters 3, 5, 6 . . . 154
14.4 2. Ownership and Lifetime — Chapters 5, 6, 7 . . . 155
14.5 3. Types and Bits — Chapters 2, 4 . . . 155
14.6 4. Templates — Chapter 8 . . . 155
14.7 5. Testing — Chapter 10 . . . 156
14.8 6. Concurrency — Chapter 12 . . . 156
14.9 7. Performance — Chapter 13 . . . 156
14.10How to Revise . . . 156
14.11The Argument, Closed . . . 157
14.12Exercises . . . 157
14.13Self-Check . . . 158
14.14Summary . . . 158

Appendices **160**

A Self-Check Answers **160**
A.1 Chapter 1 — The Toolchain and the Build 160
A.2 Chapter 2 — Types, Bits, and the Preprocessor 161
A.3 Chapter 3 — The Stack and the Heap 162
A.4 Chapter 4 — Pointers, References, and `const` 162
A.5 Chapter 5 — Classes, Structs, and RAII 163
A.6 Chapter 6 — Smart Pointers, Copy Semantics, and Custom Allocation 164
A.7 Chapter 7 — Polymorphism and Abstract Base Classes 165
A.8 Chapter 8 — Templates and Generic Programming 166
A.9 Chapter 9 — The STL: Containers, Iterators, and Algorithms 167
A.10 Chapter 10 — Error Handling and Unit Testing 168
A.11 Chapter 11 — File I/O and Streams 169
A.12 Chapter 12 — Concurrency and Multithreading 170
A.13 Chapter 13 — Architecture and Performance 171
A.14 Chapter 14 — Integration 172

B Glossary **173**

About This Book

C++ Programming was written for **PROG1353** at the **New Brunswick Community College** by **David Burchill**.

A note on AI assistance

Some of the content in this book — including draft explanations, worked examples, exercises, and self-check questions — was generated with the assistance of AI language models. Every generated passage was reviewed, edited, and compiled by the author, who remains responsible for the accuracy of the final text.

If you find an error, please report it to your instructor. Reporting one is genuinely useful: it gets the book fixed, and noticing that something is wrong is itself a sign that you are reading the way this book asks you to.

How to Use This Book

You already know how to program. You have written C# for a year — classes, methods, loops, collections, exceptions. This book does not teach you to program. It teaches you what happens when you take away the runtime that has been managing memory for you, and hand you the machine instead.

That trade is the whole course. In C#, a garbage collector decides when your objects die. In C++, **you** decide, and the language gives you exact, unforgiving control over when memory is acquired and when it is released. Everything that makes C++ suitable for a microcontroller with 16 KB of RAM comes from that control, and so does everything that makes it dangerous.

Before Chapter 1, spend a few minutes on this page. How you use the book changes how much you learn from it.

How the book is organized

The book runs from the toolchain, through the memory model, to the standard library, concurrency, and performance. Each chapter introduces one set of ideas — one new thing at a time — and follows the same shape:

- **Concepts**, explained in plain language and shown immediately in real code.
- **Worked examples** you can follow line by line.
- **"Predict before you run" boxes**, where you commit to what a piece of code will do before you see the answer.
- **Exercises**, ordered from tracing code, to filling in blanks, to writing programs from scratch.
- **Self-check** questions to test yourself.
- A short **summary** that asks you to recall, not re-read.

Answers to the self-check questions are in **Appendix A**, so you can close the loop the moment you finish a chapter. Worked solutions to the *exercises* are not in this book — your instructor releases those a couple of days after each set is assigned, so you get a real run at the problems first. **Appendix B** is a glossary of every key term.

Coming from C

Your C# knowledge is an asset and a liability at the same time, and it is worth knowing which is which.

Most of the syntax will look familiar: `if`, `while`, `for`, `int`, braces, semicolons. That part is nearly free, and this book will not spend your time re-teaching it.

The trouble is the ideas that look the same and are not. A C# `int` is always 32 bits; a C++ `int` is whatever the platform says. A C# object variable is a reference; a C++ object variable *is* the object. In C#, `=` between two objects makes two names for one thing; in C++, it makes a copy. Nobody frees anything in C#; in C++ every acquisition needs a matching release, and the language's best idea — RAII — exists to make that automatic without a collector.

So when this book says "in C#, this would…", read carefully. Those passages are where your existing intuition is about to be wrong, and the point of naming them is to catch the error before the compiler — or worse, the running program — does.

How to study so it sticks

Test yourself instead of re-reading. Re-reading a page feels productive, but that feeling is recognition, not memory. Close the book and try to recall, then check. Every time you pull an idea out of your own head, you make it easier to pull out next time.

Predict before you run. When you meet a "Predict before you run" box, or you are about to run any program, write down what you think it will print *first*. Commit. A wrong prediction is the most useful thing that can happen to you — it shows exactly where your mental model and the machine disagree, and in C++ that gap is where the bugs live.

Spread your practice out. Two hours across four days beats eight hours in one. Each chapter has a flashcard deck; review a little every day rather than all at once. Returning to material after you have half-forgotten it is uncomfortable, and the discomfort is the work paying off.

Do the exercises honestly. Commit to an answer before you check it — against Appendix A for the self-checks, and against your instructor's solutions for the exercises when they are released. The couple of days before solutions appear are not a delay to wait out; they are the part where the learning happens.

Type your code; don't lean on autocomplete. Syntax you retrieve from your own memory is syntax you understand. This matters more in C++ than it did in C#, because C++ will happily compile a program that is wrong in a way no editor can see.

Read the compiler. C++ error messages are long, and students learn to skim them for the red. Don't. Read the *first* error, in full, before you change anything — the ones after it are usually consequences of it. Learning to read diagnostics is a course outcome, not a chore on the way to one.

Let's begin.

Part I

The Machine and the Build

Chapter 1

The Toolchain and the Build

1.1 What You'll Be Able to Do

- Explain what a C++ compiler produces, and how that differs from what C# produces.
- Write, compile, and run a C++ program from the terminal, with a plain text editor.
- Tell **compile time** from **run time**, and sort a question into the side that answers it.
- Name the four build stages — preprocess, compile, assemble, link — and say what each one consumes and produces.
- Produce object files with `clang++ -c` and link them by hand.
- Use the flags `-std=c++20`, `-Wall`, `-Wextra`, `-c`, `-o`, and `-E`.
- Write a minimal `CMakeLists.txt` and build a project with two commands.

1.2 No Runtime, No Collector

In C# you pressed **Run**, or typed `dotnet run`, and a program appeared. That one action hid a great deal. C++ makes the hidden steps visible — and on an embedded target you have no choice but to see them, because there is no button on a microcontroller.

The deepest difference is *what runs your program*:

- **C#** compiles to IL, an intermediate language, which runs on the **CLR** — a managed runtime that compiles IL to machine code as the program runs, and that runs a **garbage collector** to free memory on your behalf.
- **C++** compiles **directly to native machine code** for one specific kind of CPU. There is no runtime layer underneath and no garbage collector. The executable *is* the program. If you allocate memory, you free it — a story that starts in Chapter 3.

That is the trade this whole course is about. You lose the safety net; you gain a program small enough and predictable enough to run on a chip with a few kilobytes of RAM.

1.3 Your First Program

```cpp
#include <iostream>   // brings in std::cout - like C#'s using System;

int main() {          // the entry point - like C#'s Main
    std::cout << "Hello, Embedded World!\n";
    return 0;         // 0 means "exited successfully"
}
```

Three things are worth naming right now.

`#include <iostream>` makes `std::cout` available. It is the rough analogue of `using System;`, though as you'll see shortly it works by an entirely different mechanism.

`int main()` is the entry point. Unlike C#'s `Main`, which is often `void`, C++'s `main` returns an `int`, and `return 0;` means the program finished successfully. Any other value signals a problem to whatever launched the program.

`std::cout << "..."` prints. The `std::` prefix says the name lives in the standard library's namespace, `std`. And note carefully: **it does not add a newline.** `std::cout` is the equivalent of `Console.Write`, never `Console.WriteLine`. If you want a line break, you write `\n` yourself. There is no separate "print with a newline" call.

1.4 Compiling and Running from the Terminal

No Run button. Two commands:

```
clang++ -std=c++20 -Wall -Wextra hello_world.cpp -o hello_world
./hello_world        # Windows: .\hello_world.exe
```

The flags:

- `-std=c++20` — compile against the C++20 standard. This course is modern C++ throughout.
- `-Wall -Wextra` — turn on warnings. **Warnings are free bug reports.** Treat them as errors.
- `-o hello_world` — name the output file. Without `-o`, you get a file called `a.out`.

Your compiler is `clang++`, from the LLVM project, and your debugger (from Chapter 3) is `lldb`. There is no `g++` and no `gcc` on a correctly configured machine, and nothing is aliased to them. When a textbook or a web tutorial tells you to type `g++`, type `clang++` instead — every flag this course uses means the same thing under both.

There are many C and C++ compilers available. Two of the most popular are the LLVM compilers (clang, clang++), which we use in this course, and the GNU compilers (gcc, g++).

1.5 Reading a Compiler Error

Delete the semicolon after the `std::cout` line and rebuild. Clang says:

```
hello_world.cpp:4:44: error: expected ';' after expression
    4 |     std::cout << "Hello, Embedded World!\n"
      |                                            ^
      |                                            ;
1 error generated.
```

Four parts, and it is worth learning to read them in order:

1. **`hello_world.cpp`** — the file.
2. **`4:44`** — line 4, column 44.
3. **`expected ';' after expression`** — what it wanted.
4. The caret, pointing at the exact column, with the missing character printed underneath.

The compiler is not scolding you. It has located the problem more precisely than you could have, and it has told you what to type. Read the **first** error in full before changing anything — later errors are often just consequences of the first one, and they evaporate when you fix it.

1.6 Compile Time vs Run Time

You just typed two commands, and the gap between them is one of the most useful lines in this course.

```
clang++ -std=c++20 -Wall -Wextra hello_world.cpp -o hello_world    # compile time
./hello_world                                                      # run time
```

Compile time is everything that happens while your program is being *built* — while the compiler reads your source text and produces an executable. **Run time** is what happens when that finished executable *executes*.

In C++ the boundary is a **file**. To the left of it: source text, and a compiler reading it. To the right: a native executable that no longer needs the source, the compiler, or you.

> **Predict before you run.** Compile `hello_world.cpp`, then delete `hello_world.cpp` — the source file itself. Will `./hello_world` still run?
>
> It will. The source is a compile-time artifact; the executable is the run-time one, and it has no memory of where it came from. This is the boundary made physical.

Two words that look identical and are not. A *runtime* — one word, a noun — is software that manages your program while it executes; C#'s CLR is a runtime. *Run time* — two words, a moment — is when your program executes. C++ has no **runtime**. It very much has a **run time**. Both are true at once, which is why the sentence "C++ has no runtime" confuses everyone who misses the distinction.

What gets decided when:

Answered at compile time	Only knowable at run time
Is this syntax legal?	What did the user type in?
What type is this, and how big is it?	What did the sensor read?
Which function does this call resolve to?	How many times did the loop actually run?
How much stack space do these locals need?	Did that allocation succeed?

Answered at compile time	Only knowable at run time
What machine code goes in the binary?	Is this pointer still valid?

Why C++ leans so hard on compile time. A compile-time error costs you a rebuild. A run-time error costs a crashed device — and on a microcontroller there is no stack trace, no debugger attached, often no screen. So C++ pushes as much work and checking as it can to the *left* of the boundary. Watch for the pattern: `constexpr` computes values during the build (Chapter 2), `sizeof` is answered entirely by the compiler (Chapter 5), templates generate whole functions during the build (Chapter 8), and choosing between compile-time and run-time polymorphism becomes a design decision with a measurable cost (Chapter 7).

You never noticed this line in C# for two reasons. `dotnet run` did both jobs in one command, so the boundary was invisible. And the CLR's JIT compiler genuinely *does* compile while the program runs — in C#, the line actually moves. C++ compiles ahead of time, once, before the program ever starts.

1.7 One Command, Four Stages

`clang++` looks like a single step. It runs four, and naming them is what lets you read errors, because **each stage fails differently**.

1. **Preprocess** — expands `#include` directives and macros. Pure text substitution. The output is a single *translation unit*.
2. **Compile** — translates that C++ text into *assembly* for your CPU. Most syntax and type errors surface here.
3. **Assemble** — turns assembly into machine code inside an **object file** (`.o`). Not yet runnable.
4. **Link** — stitches object files and libraries into one **executable**. `undefined symbol` errors live here: the compiler was satisfied, but the linker could not find a *definition* for something you called.

```
.cpp → [preprocess] → .cpp → [compile] → .s → [assemble] → .o → [link] → executable
```

All four stages are compile time. The boundary sits at the right-hand edge of *link*, the moment the executable exists. Every error from any of the four is a compile-time error — you never got a program at all. That is a categorically different kind of trouble from a program that builds fine and then misbehaves.

> Our linker is LLVM's `lld`, and it says `undefined symbol`. GCC-based tutorials and older textbooks say `undefined reference`. Same failure, same fix — recognise both.

1.8 Doing It By Hand

Split compiling from linking, so the object file is a real thing you can see rather than an abstraction:

```
clang++ -std=c++20 -c hello_world.cpp -o hello_world.o    # stop after assemble
clang++ hello_world.o -o hello_world                      # link the .o into an executable
```

`-c` means "compile and assemble, but **do not link**." You now hold `hello_world.o` — proof that the stages are separable.

You can watch the preprocessor too:

```
clang++ -E hello_world.cpp -o hello_world.i
```

Open `hello_world.i`. It runs to tens of thousands of lines, nearly all of it `<iostream>` pasted in wholesale. Don't read it — just look at its size. That size *is* the lesson: `#include` is copy-and-paste, performed by a program that does not understand C++ at all. It is nothing like `using System;`, which is a lookup rule resolved by the C# compiler.

1.9 Why CMake

Typing `clang++` by hand is fine for one file and miserable for fifty. **CMake** is the build system the embedded electives use, so you start with it now. You describe *what* to build; CMake generates the commands.

```
cmake_minimum_required(VERSION 4.4)
project(FirstApp CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED True)

add_executable(hello hello_world.cpp)
```

Build it out-of-source, so everything generated stays in `build/`:

```
cmake -G "MinGW Makefiles" -B build    # configure: read CMakeLists.txt, generate build files
cmake --build build                    # compile + link
.\build\hello.exe                      # macOS/Linux: ./build/hello
```

Pass `-G "MinGW Makefiles"` every time on Windows. The name is a historical wart: it selects the *make program*, not the compiler. Your setup script already pointed CMake at clang.

Never edit anything inside `build/`. It is generated from `CMakeLists.txt` and thrown away on the next build. If something in there is wrong, the thing to change is `CMakeLists.txt`.

1.9.1 Why that first line says 4.4

`cmake_minimum_required(VERSION 4.4)` says your project needs CMake 4.4 or newer. Older CMake refuses to configure it at all, with a clear message rather than a strange failure later.

You will see tutorials — most of them, in fact — open with something much older, like `VERSION 3.10`. It is worth understanding what that line actually claims, because it is not just a floor.

It is a promise you cannot keep. Writing `VERSION 3.10` tells everyone that your project works on CMake 3.10. Have you run it on CMake 3.10? Almost certainly not — you have one version installed, and it is the one your setup script gave you. The number was copied from a tutorial that

copied it from an older tutorial. **Only claim to support what you have actually tested.** This is not pedantry: someone with CMake 3.10 will believe you, try it, and hit a failure you never saw and cannot help them with. In this course we run 4.4, we test on 4.4, so we require 4.4.

And it changes how CMake behaves. This is the part almost nobody knows. `cmake_minimum_required` also sets CMake's *policy defaults* to the version you name. Writing `VERSION 3.10` does not merely permit old CMake — it asks your modern CMake to behave like the old one, switching on years of backward-compatibility rules to protect projects written before those rules changed. Your project was written this morning. Naming 4.4 opts you into current behaviour, which is also the behaviour the current documentation describes.

Real libraries, which genuinely do get built on many versions by people the author will never meet, write a *range*: `cmake_minimum_required(VERSION 3.15...4.4)` means "needs at least 3.15, and I have tested up to 4.4." That is the honest form of the claim, and it is honest precisely because someone verified both ends. You do not need it here — everyone in this class runs the same toolchain — but now you know what it means when you meet it.

1.10 Worked Example

Problem. Split a one-file program into two source files and build it two ways — by hand, then with CMake.

```cpp
// greet.cpp
#include <iostream>
void greet() { std::cout << "Hello, Embedded World!\n"; }
```

```cpp
// main.cpp
void greet();                // declaration - "greet exists somewhere"
int main() { greet(); return 0; }
```

That first line of `main.cpp` is a **declaration**: it promises the compiler that a function named `greet` taking no arguments and returning nothing exists *somewhere*. The compiler believes you. Making good on the promise is the linker's problem.

By hand — compile each translation unit, then link both objects:

```
clang++ -std=c++20 -Wall -Wextra -c greet.cpp -o greet.o
clang++ -std=c++20 -Wall -Wextra -c main.cpp  -o main.o
clang++ greet.o main.o -o app
./app
```

Now break it twice, deliberately, because the two failures look completely different.

(a) Delete the declaration line in `main.cpp` and recompile:

```
main.cpp:1:14: error: use of undeclared identifier 'greet'
    1 | int main() { greet(); return 0; }
      |              ^
```

(b) Restore the declaration, then link with only `main.o` — omit `greet.o`:

```
ld.lld: error: undefined symbol: greet()
>>> referenced by main.cpp
>>>               main.o:(main)
clang++: error: linker command failed with exit code 1
```

The contrast is the whole lesson. Error (a) comes from the compiler, names a **line in your source**, and means *you never told me this thing exists.* Error (b) comes from `ld.lld`, names an **object file** and no source line at all, and means *you told me it exists and it does not.* Two stages, two shapes of message.

With CMake — list both sources:

```
cmake_minimum_required(VERSION 4.4)
project(App CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED True)
add_executable(app main.cpp greet.cpp)
```

```
cmake -G "MinGW Makefiles" -B build
cmake --build build
.\build\app.exe
```

Same result, no hand-typed `clang++`. Leave `greet.cpp` out of `add_executable` and you get failure (b) again — the same linker error, now hidden one layer down.

1.11 Predict Before You Run

Predict before you run. What does this print, and on how many lines?

```
#include <iostream>
int main() {
    std::cout << "Boot";
    std::cout << "Sequence";
    std::cout << "\n";
    std::cout << "Ready\n";
    return 0;
}
```

Two lines: `BootSequence`, then `Ready`. Four `<<` statements do not make four lines — `std::cout` breaks a line only where you write `\n`.

Predict before you run. You compile `main.cpp` (which calls `greet()`) with `-c` and it succeeds. Do you now have a program you can run?

No. `-c` stops after the assemble stage, so you have `main.o` and nothing else. There is no executable until you link, and this link will fail anyway until `greet.o` is included.

1.12 Common Errors

Symptom	Cause	Fix
`fatal error: 'iostreams' file not found`	Header name misspelled; the preprocessor could not find a file to paste in	It is `<iostream>`, singular
`error: expected ';' after expression`	Missing semicolon at the end of a statement	Add it at the column the caret points to
`ld.lld: error: undefined symbol: f()`	Declared and called, never defined — or its `.cpp` was left out of the build	Add the missing source file to the link line or to `add_executable`
Everything prints on one line	Expecting `std::cout` to behave like `Console.WriteLine`	Add `\n` where you want the break
`'clang++' is not recognized`	Toolchain not on the PATH in this terminal	Reopen the terminal; if it persists, re-run the setup script
Edits to files in `build/` keep vanishing	`build/` is generated	Edit `CMakeLists.txt` and rebuild

1.13 Key Terms

assemble — the build stage that turns assembly text into machine code inside an object file.

`clang++` — the C++ compiler used in this course, from the LLVM project.

CMake — a build system: you describe the target in `CMakeLists.txt` and CMake generates the build commands.

compile time — everything that happens while the program is being built, before an executable exists.

declaration — a promise to the compiler that a name exists somewhere, with a given type or signature. Distinct from a *definition*, which supplies the body.

executable — the native, runnable output of the link stage.

link — the build stage that stitches object files and libraries into an executable. Source of `undefined symbol` errors.

object file (`.o`) — machine code for one translation unit, not yet runnable.

preprocess — the build stage that expands `#include`s and macros by pure text substitution.

run time — the period during which the finished executable executes.

runtime — a piece of software that manages a program as it executes, such as C#'s CLR. C++ has none.

`std::cout` — the standard output stream. Prints with `<<`, and never adds a newline.

translation unit — one source file after preprocessing, with every `#include` and macro expanded. The actual input to the compiler.

`undefined symbol` — the linker found a call to something no object file defines. GCC words this as `undefined reference`.

1.14 Exercises

Work in the terminal only for this set — no Run button, no build tasks. Commit each time your code compiles cleanly.

1. **(Trace)** Without compiling it, predict exactly what this prints. How many lines, and what does each say?

```cpp
#include <iostream>

int main() {
    std::cout << "Boot";
    std::cout << "Sequence";
    std::cout << "\n";
    std::cout << "Ready\n";
    return 0;
}
```

2. **(Trace)** For each C# element on the left, write the C++ equivalent from this program.

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, Embedded World!\n";
    return 0;
}
```

C#	C++ equivalent
`using System;`	?
`static void Main()`	?
`Console.Write("hi");`	?

3. **(Trace)** Each message below comes from building a C++ program. Label each one **preprocess**, **compile**, or **link**.
    1. `fatal error: 'iostreams' file not found` (you typed `#include <iostreams>`)
    2. `error: expected ';' after return statement`
    3. `ld.lld: error: undefined symbol: compute()`

4. **(Faded)** You have `sensor.cpp` and `main.cpp`. Complete the commands to build an executable named `logger` by hand — compile each to an object file, then link.

```
clang++ -std=c++20 -Wall -Wextra -c sensor.cpp -o ________
clang++ -std=c++20 -Wall -Wextra -c main.cpp   -o ________
clang++ ________ ________ -o logger
.\logger.exe
```

5. **(Write from scratch)** Write a complete `CMakeLists.txt` that builds an executable called `blink` from two source files, `blink.cpp` and `gpio.cpp`, using C++20. Then write the two terminal commands that configure and build it. Check your work by actually building a two-file project before moving on.

6. **(Write from scratch)** From an empty folder, using the terminal only:

   1. Create `boot.cpp` with a `main` that prints your board's name followed by `booted OK` on one line, then a newline.
   2. Compile it with warnings on and the C++20 standard, output named `boot`.
   3. Run it.
   4. Write down the three commands you used and the exact output.

   Then add a second print with no `\n` and predict what changes before you run it.

7. **(Bridge from C#)** Rewrite this C# snippet in C++ so that it produces the *same* output.

```
Console.Write("A");
Console.WriteLine("B");
Console.WriteLine("C");
```

1.15 Self-Check

These questions mix this chapter's material. Try them from memory; answers are in Appendix A.

1. What are the four stages of a C++ build, in order?

   - Compile, preprocess, link, assemble
   - Preprocess, compile, assemble, link
   - Preprocess, link, compile, assemble
   - Compile, assemble, link, preprocess

2. Which stage produces an object file?

   - Preprocess
   - Compile
   - Assemble
   - Link

3. What does `clang++ -c main.cpp -o main.o` do?

   - Builds a runnable program named `main.o`
   - Compiles and assembles, but does not link
   - Runs the preprocessor only
   - Links `main.cpp` against the standard library

4. You see `ld.lld: error: undefined symbol: compute()`. Which stage failed, and what should you look for?

5. What is a translation unit?
6. True or false: `std::cout << "Hi";` prints `Hi` followed by a newline.
7. True or false: after a successful build, deleting the `.cpp` file prevents the executable from running.
8. Which flag stops the build after the preprocessor so you can inspect the expanded source?
   - `-c`
   - `-E`
   - `-o`
   - `-std=c++20`
9. In one sentence each: what does C# have at run time that C++ does not, and what does that cost you?
10. A classmate's error message begins `main.cpp:7:22: error:`. Yours begins `ld.lld: error:`. Without reading further, what do you already know about each?

1.16 Summary

Close the book and answer these from memory before moving on.

- What does the C++ compiler produce, and what does C# produce instead?
- Name the four build stages and what each one consumes and produces.
- Where exactly does compile time end and run time begin?
- What is the difference between a *runtime* and a *run time*?
- How can you tell a compiler error from a linker error at a glance?
- What do `-c`, `-E`, `-o`, `-Wall`, and `-std=c++20` each do?

The next chapter takes the types you thought you knew from C# — `int`, `char`, `bool` — and shows you why their sizes are not guaranteed, why that matters on a microcontroller, and how to manipulate individual bits inside a hardware register.

Chapter 2

Types, Bits, and the Preprocessor

2.1 What You'll Be Able to Do

- Explain what the preprocessor does, and inspect its output with `clang++ -E`.
- Choose between `#define`, `const`, and `constexpr`, and say why one of them is the default.
- Write include guards and explain the problem they solve.
- Explain why the size of `int` and `long` is implementation-defined, and why that matters in embedded work.
- Use fixed-width types from `<cstdint>` and justify each choice.
- Set, clear, toggle, and test an individual bit with `&`, `|`, `~`, `^`, `<<`, and `>>`.
- Explain why changing one bit of a hardware register requires a read-modify-write.

2.2 The Preprocessor: A Program That Edits Your Program

Before the compiler sees your code at all, a separate program edits it.

The preprocessor does not understand C++. It does not know what a type is, or a function, or a scope. It does **text substitution**, and hands the result on. Everything it does is driven by lines beginning with `#`.

```
#include <iostream>     // paste the entire contents of <iostream> here
#define MAX_RETRIES 3   // replace the text MAX_RETRIES with the text 3, everywhere
```

C# has no real equivalent. It has `#if DEBUG`, but there is no textual `#include` and no macro substitution, because C# has no header files at all. This is the first place where a C++ mechanism has no C# counterpart to map onto — you are meeting something genuinely new rather than a renamed version of something you know.

2.3 Seeing It Instead of Believing It

```
clang++ -E main.cpp -o main.i
```

Open `main.i`. This is what the compiler *actually* receives.

Search it for the name of a macro you defined. It is not there. It has been replaced by its value, everywhere, and by the time the compiler runs the name has ceased to exist. That absence is the entire lesson: **macros are gone before compilation begins.** Nothing after the preprocessor knows they were ever there — not the compiler, not the linker, and, as you will see in Chapter 3, not the debugger.

2.4 `#define` vs `const` vs `constexpr`

Three ways to name a constant. Only one is a good default.

```
#define BUFFER_SIZE 256              // preprocessor: pure text, no type, no scope
const int bufferSize = 256;          // a real variable, typed, scoped
constexpr int kBufferSize = 256;     // a real variable, typed, scoped, known at compile time
```

	`#define`	`const` / `constexpr`
Has a type?	**no** — it is text	yes
Obeys scope?	**no** — from the `#define` to the end of the file	yes
Visible to the debugger?	**no** — it is gone before compilation	yes
Errors point at?	the *expansion*, often confusingly	the line you wrote

The rule for this course: use `constexpr` for constants. Use `#define` only for include guards and conditional compilation.

`constexpr` is also the first construct that hangs directly on Chapter 1's compile-time / run-time boundary. It means *compute this during the build*. The value is finished before the program starts.

2.5 The Macro Pitfall

```
#define SQ(x) x*x

int main() {
    std::cout << SQ(2 + 3) << "\n";
}
```

> **Predict before you run.** What does this print?
>
> It prints **11**, not 25. The substitution is textual, so `SQ(2 + 3)` becomes `2 + 3*2 + 3`. Multiplication binds tighter than addition, and there are no parentheses to stop it.

The patch is defensive parentheses:

```
#define SQ(x) ((x)*(x))
```

The better answer needs no patch at all:

```
constexpr int sq(int x) { return x * x; }
```

The function has no bug to fix, because `2 + 3` is evaluated to `5` *before* the call happens. A macro pastes text; a function receives values. That difference, in one line, is the argument for `constexpr`.

2.6 Include Guards

If two files both `#include "sensor.h"`, the preprocessor pastes it twice — and a type can only be defined once per translation unit. You get redefinition errors from a file you never edited.

```
// sensor.h
#pragma once            // what we use in this course

// the portable equivalent, which you will see everywhere:
#ifndef SENSOR_H
#define SENSOR_H
// ...declarations...
#endif
```

Use `#pragma once` in your own code: it is one line and cannot be gotten wrong. Recognise the `#ifndef` form, because most headers you read use it. This is the one sanctioned use of `#define` — it names no value, it just marks "seen already".

A header must also stand on its own. If your header uses `uint8_t`, it must `#include <cstdint>` itself. A header that compiles only because some *other* file happened to include the right thing first is a bug waiting for someone to reorder two lines.

2.7 `int` Is Not a Promise

In C#, `int` is **always** exactly 32 bits. The language guarantees it. In C++, `int` is "at least 16 bits, whatever the implementation chose", and the same is true of `long`.

Type	Windows (x64 **and** ARM64)	Linux / macOS 64-bit
`int`	4 bytes	4 bytes
`long`	**4 bytes**	**8 bytes**
`long long`	8 bytes	8 bytes
pointer	8 bytes	8 bytes

Be precise about what this does and does not mean. In practice `int` is 4 bytes essentially everywhere you will meet it, and claiming otherwise would be overstating the danger. The honest statement is narrower and more useful: **the standard does not promise you a size, and `long` is where that bites first.** That `long` row is a real difference you can hit this term, the moment you read code written on Linux. And on a microcontroller, `int` really can be 2 bytes.

2.8 Fixed-Width Types Say What They Mean

```
#include <cstdint>

uint8_t   statusRegister = 0;   // exactly 8 bits, unsigned - a register
int32_t   temperatureMilliC;    // exactly 32 bits, signed
uintptr_t address;              // an unsigned integer big enough to hold a pointer
```

The naming is a pattern, not a list to memorise: `u` means unsigned, the number is the count of bits, `_t` means "type". Once you see it, `int16_t` needs no separate explanation.

When a value maps to hardware, a wire protocol, or a file format, **the width is part of the specification**. Say it in the type.

2.9 The Promotion Trap

```
uint8_t a = 200;
uint8_t b = 100;
std::cout << a + b << "\n";
std::cout << sizeof(a + b) << "\n";

uint8_t c = a + b;
std::cout << static_cast<int>(c) << "\n";
```

> **Predict before you run.** Three lines of output. What are they?
>
> `300`, then `4`, then `44`.
>
> Nearly everyone predicts `44` on the first line, reasoning that 8 bits cannot hold 300. But **C++ has no 8-bit arithmetic.** Both operands are promoted to `int` *before* the `+`, which is exactly what `sizeof(a + b) == 4` proves. The addition happens in 32 bits, and 300 fits comfortably. The wrap only happens on the third line, where the 32-bit `300` is stored back into 8 bits: $300 - 256 = 44$.

State the rule once and keep it: **C++ widens, computes, and narrows only when you assign.**

That `static_cast<int>` on the last line is not decoration. Without it, `std::cout` treats a `uint8_t` as a **character** and prints the glyph for character code 44 — a comma. You will hit this the first time you try to print a register value.

2.10 Bit Manipulation: Eight Independent Switches

A `uint8_t` register is eight independent switches. These six operators are how you work one switch without disturbing the other seven.

Operator	Name	Use
`&`	AND	**test** a bit; **clear** bits (with `~`)
`|`	OR	**set** a bit

Operator	Name	Use
`^`	XOR	**toggle** a bit
`~`	NOT	invert every bit — builds a clear-mask
`<<`	left shift	build a mask: `1 << 3` is bit 3
`>>`	right shift	move a bit down to read it

The four moves. Learn these as a set; they are the whole of the rest of this chapter.

```
#include <cstdint>

uint8_t reg = 0;

reg |=  (1 << 3);                  // SET bit 3
reg &= ~(1 << 3);                  // CLEAR bit 3   <-- note the ~
reg ^=  (1 << 3);                  // TOGGLE bit 3
bool on = (reg & (1 << 3)) != 0;   // TEST bit 3
```

Build each one by asking a single question: *what do I want to happen to the other seven bits?* The answer is always "nothing", and that requirement is what forces the mask form.

Write `1 << 3`, never `8`. The shift form says *bit three* — which is the thing you actually mean — and it keeps working when the bit number is a variable or a named constant.

2.11 Read-Modify-Write

```
reg = (1 << 3);            // WRONG: sets bit 3 and clears all seven others
reg |= (1 << 3);           // RIGHT: reads reg, ORs in bit 3, writes it back
```

`|=` is a **read-modify-write**: fetch the current value, change one bit, store it back. On real hardware that is three bus operations, which is why writing to a register is never simply a store.

The two lines above differ only in the bits nobody mentioned — and that is exactly where the bug lives. Try it: start from `0b0000'0101`, and compare what each line leaves behind.

2.12 The Two Errors Everyone Makes

1. Clearing with the wrong mask. This is the most common bit-manipulation error there is.

```
reg &= (1 << 3);           // WRONG: keeps ONLY bit 3, clears everything else
reg &= ~(1 << 3);          // RIGHT: clears bit 3, keeps everything else
```

2. Precedence — and this one the compiler catches for you.

```
if (reg & 0x02 == 0)       // WRONG
if ((reg & 0x02) == 0)     // RIGHT
```

`&` binds *looser* than `==`, so the first line means `reg & (0x02 == 0)`, which is `reg & 0`, which is always `0`. Compiled with `-Wall`:

```
warning: & has lower precedence than ==; == will be evaluated first [-Wparentheses]
    6 |     std::cout << (reg & 0x02 == 0) << "\n";
      |                         ^~~~~~~~~~
```

Stop and notice what just happened. That code has no error. It compiles, it runs, and it is silently, permanently wrong — the condition can never be true. The compiler found a real bug that would otherwise have cost you an afternoon. **This is what `-Wall -Wextra` is for**, and it is why this course treats warnings as errors.

2.13 Worked Example

Problem. Write a small interface for an 8-bit status register — set, clear, toggle, and test any bit — then drive it.

```cpp
#include <cstdint>
#include <iostream>

constexpr uint8_t kMotorEnable = 0;   // bit 0
constexpr uint8_t kFaultFlag   = 2;   // bit 2

void setBit(uint8_t& reg, uint8_t bit)    { reg |=  (1 << bit); }
void clearBit(uint8_t& reg, uint8_t bit)  { reg &= ~(1 << bit); }
void toggleBit(uint8_t& reg, uint8_t bit) { reg ^=  (1 << bit); }
bool testBit(uint8_t reg, uint8_t bit)    { return (reg & (1 << bit)) != 0; }

int main() {
    uint8_t status = 0;

    setBit(status, kMotorEnable);
    setBit(status, kFaultFlag);
    clearBit(status, kFaultFlag);

    std::cout << std::boolalpha
              << "motor: " << testBit(status, kMotorEnable) << "\n"
              << "fault: " << testBit(status, kFaultFlag)   << "\n";
}
```

```
motor: true
fault: false
```

Two details worth pausing on.

The first three functions take `uint8_t& reg` and the fourth takes plain `uint8_t reg`. The `&` means the function changes the caller's variable rather than a copy of it. `testBit` modifies nothing, so it does not need one. What that `&` actually *is* comes apart in Chapter 4; for now, that sentence is enough.

The bit numbers are `constexpr`, from the first half of this chapter, put to work in the second. Named constants are the difference between `setBit(status, 2)` and `setBit(status, kFaultFlag)` — and six months later only one of those is readable.

2.14 Common Errors

Symptom	Cause	Fix
A macro expands to something arithmetically wrong	Unparenthesised macro parameter	Parenthesise every parameter and the whole body — or use `constexpr`
`error: redefinition of 'X'`	A header included twice with no guard	Add `#pragma once` at the top
Header compiles in one file, fails in another	Header depends on an include it does not make itself	Give the header its own `#include`
`uint8_t` arithmetic gives a bigger number than expected	Integer promotion to `int`	Expected — it only narrows when you assign back
Printing a `uint8_t` shows a letter or punctuation	`std::cout` treats it as a character	`static_cast<int>(value)`
A register operation clobbers unrelated bits	`=` used where `|=` or `&= ~` was meant	Read-modify-write, always
A bit test is never true	`&` binds looser than `==`	Parenthesise: `(reg & mask) == 0`

2.15 Key Terms

`<cstdint>` — the header declaring the fixed-width integer types.

`#define` — a preprocessor directive that substitutes text. Reserved in this course for include guards and conditional compilation.

`constexpr` — a value (or function) computable at compile time. The default way to name a constant here.

fixed-width type — an integer type whose size is guaranteed by name: `uint8_t`, `int32_t`, and so on.

implementation-defined — the standard permits a choice and requires the implementation to document it. The size of `int` and `long` is implementation-defined.

include guard — `#pragma once`, or the `#ifndef`/`#define`/`#endif` idiom, preventing a header from being pasted twice into one translation unit.

integer promotion — the automatic widening of anything narrower than `int` to `int` before arithmetic.

macro — a name the preprocessor replaces with text.

mask — a value whose set bits select the bits you intend to operate on, usually built with `1 << position`.

`#pragma once` — the one-line include guard used in this course.

read-modify-write — reading a register, changing part of it, and writing it back. What `|=`, `&=`, and `^=` each perform.

`uint8_t` — an exactly-8-bit unsigned integer. The register type for this course.

2.16 Exercises

1. **(Trace)** Predict all three lines of output before compiling.

```cpp
#include <cstdint>
#include <iostream>

int main() {
    uint8_t a = 200;
    uint8_t b = 100;

    std::cout << a + b << "\n";
    std::cout << sizeof(a + b) << "\n";

    uint8_t c = a + b;
    std::cout << static_cast<int>(c) << "\n";
}
```

2. **(Trace)** Predict the output. The macro is deliberately unparenthesised.

```cpp
#include <iostream>

#define HALF(x) x/2

int main() {
    std::cout << HALF(10) << "\n";
    std::cout << HALF(4 + 6) << "\n";
}
```

3. **(Trace)** Each pair claims to do the same thing to `uint8_t reg`. Exactly one of each pair is correct. Pick it, and say in one sentence what the wrong one actually does.

```cpp
// A - turn bit 5 on, leave the others alone
reg = (1 << 5);
reg |= (1 << 5);

// B - turn bit 5 off, leave the others alone
reg &= (1 << 5);
reg &= ~(1 << 5);

// C - is bit 5 on?
if (reg & 1 << 5 == 0)
if ((reg & (1 << 5)) == 0)
```

4. **(Faded)** Complete the four register operations. Each must change **only** the named bit.

```cpp
#include <cstdint>

void setBit(uint8_t& reg, uint8_t bit)    { reg ________; }
void clearBit(uint8_t& reg, uint8_t bit)  { reg ________; }
void toggleBit(uint8_t& reg, uint8_t bit) { reg ________; }
bool testBit(uint8_t reg, uint8_t bit)    { return ________; }
```

5. **(Faded)** This header is included by two `.cpp` files in the same program, and the build fails with `error: redefinition of 'MotorState'`. Fix it two different ways, and say which one this course uses. There is a second problem in it as well — find that too.

```cpp
// motor.h
struct MotorState {
    uint8_t speed;
    bool enabled;
};
```

6. **(Spaced recall, Chapters 1 → 2)** You have `motor.h` (fixed, from Exercise 5), `motor.cpp`, and `main.cpp`.

    1. Write the commands to compile each `.cpp` to its own object file, then link them into `motorctl`.
    2. You run the link step and get:

    ```
    ld.lld: error: undefined symbol: setMotorSpeed(unsigned char)
    >>> referenced by main.cpp
    >>>               main.o:(main)
    ```

    Which stage failed, and what are the two most likely causes?

7. **(Write from scratch)** Build a status-register interface for a simulated motor controller.

    1. In a guarded header, define named bit positions as `constexpr`: bit 0 motor enable, bit 1 direction (0 = forward, 1 = reverse), bit 2 fault, bit 7 ready.
    2. Implement `setBit`, `clearBit`, `toggleBit`, and `testBit` against a `uint8_t`.
    3. Write a `main` that starts from `0`, enables the motor, sets direction to reverse, sets the ready flag, then reports each of the four flags by name.
    4. Print the register in binary at every step with `std::bitset<8>`.
    5. Then, **without using `clearBit`**, write one line that clears the fault bit and one that turns the motor off, and confirm the other bits survive.

    Predict each printed line on paper before you run it.

2.17 Self-Check

1. When does the preprocessor run, and how much C++ does it understand?
2. Which flag lets you read the preprocessor's output?

    - `-c`
    - `-E`

- `-Wall`
- `-std=c++20`

3. Give two things `constexpr` has that `#define` does not.
4. `#define TWICE(x) x*2`. What does `TWICE(3 + 1)` expand to, and what does it evaluate to?
5. True or false: in C++, `int` is guaranteed to be 32 bits.
6. Which type is exactly 8 bits, unsigned?
   - `uint8_t`
   - `int8_t`
   - `unsigned`
   - `byte`
7. `uint8_t a = 200, b = 100;` — what does `a + b` evaluate to, and what is `sizeof(a + b)`?
8. Write the line that turns bit 4 of `uint8_t reg` **off**, leaving the others alone.
9. Why is `reg &= (1 << 4);` wrong when you meant to turn bit 4 off?
10. What does `1 << 6` produce, and why write it that way rather than `64`?
11. What is a read-modify-write, and which operators perform one?
12. `if (reg & 0x08 == 0)` compiles without error but never behaves correctly. Why, and what tells you?
13. You print a `uint8_t` holding 65 and see the letter `A`. What happened, and how do you fix it?

2.18 Summary

Close the book and answer from memory.

- What does the preprocessor do, and what is left of a macro by the time the compiler runs?
- Why is `constexpr` preferred to `#define`, in two respects?
- What problem does an include guard solve?
- What does C++ actually guarantee about the size of `int`, and where does that bite first?
- State the promotion rule in one sentence.
- Write the four bit moves — set, clear, toggle, test — from memory.
- Why can you not just assign to a register?

Next chapter takes away the last thing C# was doing for you. You will meet the stack and the heap, find out exactly when a local variable dies, allocate memory yourself with `new`, and leak it — deliberately, under a debugger — so that you can see what a leak actually is.

Part II

Memory

Chapter 3

The Stack and the Heap

3.1 What You'll Be Able to Do

- Contrast stack and heap on allocation cost, lifetime, size limit, and fragmentation.
- Explain deterministic destruction, and trace when locals are created and destroyed.
- Predict construction and destruction order for nested scopes and function calls.
- Allocate on the heap with `new`/`delete` and `new[]`/`delete[]`, and free correctly.
- Write a memory leak deliberately, explain why it is one, and say what would have prevented it.
- Use LLDB to watch stack frames and inspect values while a program runs.

3.2 Two Places a Value Can Live

Every value in your program lives in one of two places, and which one decides almost everything about how you must treat it.

```
void readSensor() {
    int local = 42;                // the STACK - automatic storage
    int* heapValue = new int(42); // the HEAP - dynamic storage
    delete heapValue;              // and you must say when it ends
}
```

	Stack	Heap
Who frees it?	**the compiler**, automatically	**you**, explicitly
When?	at the closing brace of its scope	when you say `delete`
Cost to allocate	a pointer move — effectively free	a search for a free block
Size	small and fixed (~1 MB typical)	large
Fragmentation	impossible	**yes**, over time
Lifetime	tied to scope	whatever you choose

Read the first row again. In C#, the answer in *both* columns was "the garbage collector". This

table is where the absence of one stops being a sentence you were told and becomes something you have to act on.

3.3 Deterministic Destruction

This is the idea the whole memory arc of this course hangs on.

A stack variable is destroyed **at the closing brace of the scope it was declared in** — precisely, predictably, and in reverse order of construction. Not "eventually". Not "when the collector next runs". At the brace.

```
struct Tracer {
    std::string name;
    Tracer(std::string n) : name(std::move(n)) { std::cout << "  + " << name << " constructed\n
    ~Tracer() { std::cout << "  - " << name << " destroyed\n"; }
};

void inner() {
    Tracer c{"c (inner)"};
    std::cout << "inner body\n";
}

int main() {
    Tracer a{"a (main)"};
    {
        Tracer b{"b (block)"};
        std::cout << "inside block\n";
    }
    inner();
    std::cout << "end of main\n";
}
```

Predict before you run. Write all nine lines of output, in order, before reading on.

```
  + a (main) constructed
  + b (block) constructed
inside block
  - b (block) destroyed
  + c (inner) constructed
inner body
  - c (inner) destroyed
end of main
  - a (main) destroyed
```

Most people put `b`'s destruction at the end of `main`. It happens at the inner brace — a bare `{ }` block is a scope like any other.

Read the shape: **`a` is constructed first and destroyed last.** Destruction is the mirror image of construction. That is not a stylistic convention — it is what makes it safe for a later object to depend on an earlier one, because the thing you depend on is always still alive when you are torn down.

If this feels familiar from C#, it is worth being precise about the difference. `IDisposable` plus `using` gives you deterministic disposal *if you remember to write* ***`using`***. C++ gives it to you whether you remember or not, because it is the language rule rather than a pattern you opt into.

3.4 Stack Frames

Each function call gets a **frame**: a slice of the stack holding that call's parameters, locals, and the address to return to. Calls push frames; returns pop them.

```
        +--------------------------+
        | square()  n, result      |  <- top: currently executing
        +--------------------------+
        | compute() a, b, s        |
        +--------------------------+
        | main()    answer         |  <- bottom
        +--------------------------+
```

"Reverse order of construction" is just this stack coming apart from the top. A stack overflow is this picture growing past its limit.

3.5 The Debugger as a Way of Looking

From now on you compile with `-g`. That flag bakes in the line numbers and variable names LLDB needs to show you *your source* instead of machine code.

```
clang++ -std=c++20 -Wall -Wextra -g frames.cpp -o frames
lldb .\frames.exe
```

Command	Does
`break set --name square`	stop when `square` is entered (`b square` is the short form)

Command	Does
`run`	start the program
`next`	run the current line, stepping **over** any call
`step`	run the current line, stepping **into** a call
`print n`	show the value of one expression
`frame variable`	show **all** locals in the current frame
`bt`	print the call stack — the frame picture, live
`continue`	resume until the next breakpoint

The command that makes the case for the debugger is `bt`. You drew the frame diagram on paper a page ago; `bt` prints that same diagram from the running program:

```
(lldb) bt
* thread #1, stop reason = breakpoint 1.1
  * frame #0: frames.exe`square(n=5) at frames.cpp:4:16
    frame #1: frames.exe`compute(a=5, b=3) at frames.cpp:9:9
    frame #2: frames.exe`main at frames.cpp:14:16
```

`frame #0` is always where execution currently is; the numbers count *outward* toward the caller. Addresses and process IDs will differ on your machine. The **ordering** will not.

A note on what a debugger is for. It is not a tool of last resort for when something has broken. `print` is how you check whether the thing you *believe* is happening actually is — which is most useful on code that appears to work.

If you have used GDB, most names carry over: `break`, `run`, `next`, `step`, `print`, `bt`, and `continue` all work. Where they differ, LLDB's own form is longer and more explicit.

3.6 The Heap: `new` and `delete`

You reach for the heap when the stack cannot answer the question: the object must outlive its scope, its size is not known until run time, or it is simply too big for a ~1 MB stack.

```
int* one     = new int(42);        // single object
int* samples = new int[100];       // array

delete   one;                       // matching form
delete[] samples;                   // matching form - note the []
```

Three rules, stated once and enforced all term:

1. Every `new` has exactly one matching `delete`.
2. `new[]` pairs with `delete[]`. Mixing them is undefined behaviour.
3. After `delete`, the pointer is **not** null — it still holds the same address, which now points somewhere you no longer own.

Rule 3 produces the worst bugs of the three, because the pointer looks fine. Printing it before and after a `delete` shows the same value both times.

You may be tempted to make `p = nullptr;` your discipline. It helps — it converts a silent wrong answer into a reliable crash — but it is not the real fix. The real fix is not owning raw pointers at all, which arrives in Chapter 6 as `unique_ptr`. Knowing the destination is worth more than perfecting the manual hygiene you are about to stop needing.

3.7 A Leak, On Purpose

```
void readBatch() {
    int* readings = new int[4]{10, 20, 30, 40};
    std::cout << readings[2] << "\n";
    return;                    // <- readings goes out of scope. The memory does not.
}
```

When `readBatch` returns, the **pointer** is destroyed — it was a local, on the stack, so deterministic destruction takes it. The **400 bytes it pointed at** are not. Nothing on the heap is scope-bound. There is now no way to reach that memory and no way to free it.

In C#, the collector would eventually have reclaimed this. Here, nobody reclaims it. Not eventually, not at some convenient moment — never, until the process exits. Put that function in a loop and watch your machine's memory column climb. On a device that runs for months without restarting, that column is the entire problem.

3.8 Why Embedded Work Avoids the Heap

In the follow-on electives you will meet a blunt rule: **on a microcontroller, avoid dynamic allocation.** Here is why, so that you recognise the reasoning instead of merely obeying it.

- **No room.** A microcontroller may have kilobytes of RAM, not gigabytes.
- **Fragmentation.** Repeated allocate-and-free of different sizes chops the free space into pieces. A later request for a large block fails even though the total free space is ample — and it fails after weeks of uptime, not during testing.
- **Unpredictable timing.** `new` searches for a block, and how long that takes depends on the current state of the heap. Code that must respond within a fixed time cannot afford an operation with no fixed cost.
- **Nowhere to report failure.** There is no user, no dialog box, and often no screen.

The alternatives — stack objects, fixed-size buffers, and pools decided at compile time — are what the rest of this course builds toward.

3.9 Worked Example

Problem. A function returns a pointer to a buffer it allocated. Find the bug, then fix it two ways.

```
int* makeReadings(int count) {
    int* data = new int[count];
    for (int i = 0; i < count; ++i) { data[i] = i * 10; }
    return data;                  // caller now owns this. Does the caller know?
}

int main() {
    int* readings = makeReadings(4);
    std::cout << readings[2] << "\n";
    return 0;                     // BUG: leaked
}
```

Fix 1 — honour the contract:

```
int main() {
    int* readings = makeReadings(4);
    std::cout << readings[2] << "\n";
    delete[] readings;            // matching delete[], because it was new[]
}
```

Fix 2 — remove the question:

```
#include <vector>

std::vector<int> makeReadings(int count) {
    std::vector<int> data(count);
    for (int i = 0; i < count; ++i) { data[i] = i * 10; }
    return data;                  // ownership is not in doubt; nothing to leak
}
```

`std::vector` is a container that owns its memory and frees it in its own destructor. You will meet it properly in Chapter 9; today only one thing about it matters.

Both fixes are correct. They are not equally good. Fix 1 requires every future reader to notice an obligation and honour it. Fix 2 makes the bug **unwriteable** — there is no `delete` to forget, because there is no `delete`. That difference is the argument the whole course is building toward.

3.10 Predict Before You Run

Predict before you run. How many times is `T` constructed and destroyed here, and in what pattern?

```
for (int i = 0; i < 2; ++i) { T loop{"loop"}; }
```

`+loop`, `-loop`, `+loop`, `-loop`. **The loop body is a scope.** The object is constructed and destroyed once per iteration, at the braces. It is not two objects in one scope; it is one object in two scopes.

Predict before you run. Which of these leaks?

```
void f() {
    int* p = new int[50];
    if (p[0] == 0) { return; }
    delete[] p;
}
```

It leaks whenever the `if` is taken. There is nothing missing from the code as such — there is one `delete[]`, and it guards only one of the two exits. Nobody forgets the `delete` when the function is six lines long. They forget it on the third early `return`, added six months later.

3.11 Common Errors

Symptom	Cause	Fix
Returned pointer holds garbage	Returning the address of a local array	Return a `std::vector`, or have the caller supply the buffer
Memory grows and never falls	`new` with no matching `delete` on some path	Match every path — or stop owning raw pointers
Crash or corruption, sometimes	`delete` used on memory from `new[]`	`delete[]` pairs with `new[]`, always
Value prints fine after `delete`	Use-after-free; the pointer still holds the old address	Do not use it; `p = nullptr` at least makes the failure loud
"The pointer is on the heap"	Confusing the pointer with what it points at	The pointer lives where you declared it, usually the stack
Stack overflow on a big local array	The stack is ~1 MB, not unlimited	Allocate large buffers on the heap, or reduce the size

3.12 Key Terms

automatic storage — the stack. Storage the compiler allocates and frees for you, bound to scope.

`bt` — the LLDB command that prints the call stack, innermost frame first.

dangling pointer — a pointer to memory that has been freed or has gone out of scope.

`delete` / `delete[]` — return heap memory. The bracketed form pairs with `new[]`.

deterministic destruction — the guarantee that an object is destroyed at a known point, the closing brace of its scope, in reverse order of construction.

dynamic storage — the heap. Memory you request explicitly and must release explicitly.

fragmentation — free heap memory broken into pieces too small to satisfy a large request, even though the total is sufficient.

frame — one call's slice of the stack: its parameters, locals, and return address.

`-g` — the compiler flag that includes debug information so the debugger can show your source.

leak — heap memory that can no longer be reached and will never be freed.

`new` / `new[]` — allocate on the heap and return a pointer to it.

static storage duration — storage that lives for the whole run of the program, as given by `static`.

use-after-free — reading or writing memory through a pointer after it has been deleted.

3.13 Exercises

1. **(Trace)** Write the complete output, in order, before you compile. Eleven lines.

```cpp
#include <iostream>

struct T {
    const char* n;
    T(const char* s) : n(s) { std::cout << "+" << n << "\n"; }
    ~T() { std::cout << "-" << n << "\n"; }
};

void f() { T x{"x"}; }

int main() {
    T a{"a"};
    for (int i = 0; i < 2; ++i) { T loop{"loop"}; }
    f();
    T b{"b"};
    std::cout << "done\n";
}
```

2. **(Trace)** For each variable below, say **stack** or **heap**, and say **who frees it**. One of them is a trick — there are two things there, not one.

```cpp
void configure(int count) {
    int limit = count * 2;              // (a)
    int* buffer = new int[count];       // (b)
    static int callCount = 0;           // (c)
    ++callCount;
    delete[] buffer;
}
```

3. **(Trace)** Each of these compiles cleanly. Each is wrong. Say what is wrong and how you would fix it.

```cpp
// A
int* makeBuffer(int n) {
    int buffer[100];
    return buffer;
}
```

```cpp
// B
void useIt() {
    int* p = new int[50];
    if (p[0] == 0) { return; }
    delete[] p;
}

// C
void alsoUseIt() {
    int* p = new int[50];
    delete p;
}

// D
void andThis() {
    int* p = new int(7);
    delete p;
    std::cout << *p << "\n";
}
```

4. **(Trace)** You stop at a breakpoint inside `square` and type `bt`:

```
* thread #1
  * frame #0: app`square(n=5) at frames.cpp:4
    frame #1: app`compute(a=5, b=3) at frames.cpp:9
    frame #2: app`main at frames.cpp:14
```

   1. Which function is currently executing?
   2. Which line of `main` is *waiting* for this call to finish?
   3. If you type `next` twice and `square` returns, what happens to `n`?
   4. Which command shows you every local in `compute` rather than in `square`?

5. **(Spaced recall, Chapters 2 → 3)** You allocate eight status registers on the heap and want to set bit 3 of the fifth one, leaving everything else alone.

```cpp
#include <cstdint>

uint8_t* registers = new uint8_t[8]{};   // all zero

// Set bit 3 of registers[4], change nothing else:
--------------------------------

delete[] registers;
```

   Write the missing line. Then say what would be wrong with `registers[4] = 8;`.

6. **(Write from scratch)** Demonstrate a leak, prove it, then fix it.

   1. Write `allocateReadings(int count)` returning an `int*` to a heap array filled with `i * 10`.

2. Write a `main` that calls it **1,000,000 times** and never deletes, printing nothing inside the loop.
3. Run it with your task manager open. Record what the memory column does.
4. Add the matching `delete[]`. Run again. Record what changes.
5. Rewrite `allocateReadings` to return a `std::vector<int>`, with no `new` and no `delete` anywhere in the program.
6. In two or three sentences: which version would you rather hand to someone else to maintain, and why?

Before step 3, predict roughly how much memory the leaking version will consume.

3.14 Self-Check

1. Who frees a stack variable, and exactly when?
2. In what order are three objects `a`, `b`, `c`, constructed in that order in one scope, destroyed — and why that order?
3. True or false: a `{ }` block with no `if` or loop attached to it is not a real scope.
4. `int* p = new int[10];` — where does the **pointer** `p` live?
   - The heap, because it points there
   - The stack, like any other local
   - Static storage
   - Nowhere; it is not a variable
5. Which `delete` matches `new int[10]`?
   - `delete p;`
   - `delete[] p;`
   - `free(p);`
   - Either of the first two
6. After `delete p;`, what is the value of `p`?
7. Name two reasons embedded code avoids dynamic allocation.
8. What does the `-g` flag do, and which chapter's tool needs it?
9. In a `bt` listing, which frame number is the function currently executing, and which direction do the numbers count?
10. A function allocates with `new[]`, has one `delete[]` at the bottom, and three `return` statements above it. What is the defect, and what is the fix that cannot be forgotten?
11. Why does returning a pointer to a local array not work, when returning a local `int` by value does?

3.15 Summary

Close the book and answer from memory.

- State the difference between stack and heap in terms of who frees the memory and when.

- What is deterministic destruction, and how does it differ from `using` in C#?
- Draw the frame diagram for `main` calling `compute` calling `square`.
- What are the three rules for `new` and `delete`?
- Explain, in your own words, what is left behind after a leak and who can reach it.
- Give two reasons a microcontroller programmer avoids the heap.

The next chapter is about the thing you have been using all chapter without a definition: the pointer. You will take apart what a pointer actually holds, meet references as the safer alternative, learn what `const` protects and what it does not, and write to a hardware address directly.

Chapter 4

Pointers, References, and `const`

4.1 What You'll Be Able to Do

- Use `&` (address-of) and `*` (dereference), and print addresses to see memory.
- Do pointer arithmetic, and explain how a C-style array decays to a pointer.
- Distinguish pass-by-value, pass-by-pointer, and pass-by-reference, and choose correctly.
- Apply `const` to pointers and references, and read which form protects what.
- Read and act on the compiler error you get from modifying through a `const` reference.
- Reach a fixed hardware address through a pointer and drive a register with Chapter 2's masks.
- Explain what `volatile` promises, and what it does not.

4.2 A Pointer Is a Variable That Holds an Address

That is the whole definition, and almost every pointer bug is a failure to keep it in mind.

```
int value = 42;
int* p = &value;    // & means "address of"

std::cout << value << "\n";   // 42        - the object
std::cout << &value << "\n";  // 0x...     - where it lives
std::cout << p << "\n";       // 0x...     - the same address, stored in p
std::cout << *p << "\n";      // 42        - * means "go to that address"
```

Draw it. Every time.

```
  value                p
 +------+            +-----------+
 |  42  | <--------- | 0x7ffd10 |
 +------+            +-----------+
 0x7ffd10             0x7ffd18
```

Two boxes and an arrow, with the addresses written *underneath* the boxes rather than inside them. When you are stuck on a pointer question — this chapter or any later one — the question that unsticks you is always: **which box am I talking about?**

One warning before you meet the next section. The `&` symbol has two completely unrelated jobs in C++. Here it means *address of*. In a declaration like `int& r`, it means *reference*, which is something else entirely. Do not try to unify them; they are two symbols that happen to be typed the same way.

4.3 Pointer Arithmetic Is Not Integer Arithmetic

```cpp
int arr[4] = {10, 20, 30, 40};
int* p = arr;

*p            // 10
*(p + 2)      // 30   - NOT the address plus 2 bytes
p[2]          // 30   - identical to *(p + 2); [] is just notation
(p + 2) - p   // 2    - the difference is in ELEMENTS, not bytes
```

Adding 1 to an `int*` moves forward by `sizeof(int)` bytes, not by one byte. The compiler scales for you — which is exactly why `p[2]` and `*(p + 2)` are the same thing. Indexing *is* pointer arithmetic wearing a friendlier notation.

Keep the binding straight, because one pair of parentheses separates two different answers:

```cpp
*(p + 1)   // the NEXT element
*p + 1     // THIS element, plus one
```

Dereference binds tighter than `+`.

4.4 Array Decay

```cpp
int arr[4] = {10, 20, 30, 40};
int* p = arr;                       // no & needed - the array converts to a pointer

std::cout << sizeof(arr) << "\n";  // 16  - four ints
std::cout << sizeof(p) << "\n";    // 8   - one address
```

When you pass an array to a function it **decays**: the function receives a pointer, and the length is gone. Not hidden — gone. This is why every C-style array parameter comes with a separate count:

```
void process(int* data, int count);   // the count is not optional
```

A C# array knows its own `Length`. A C-style array does not, and the information is genuinely lost at the call. `std::array` and `std::vector` fix this by carrying their size with them; you will meet both properly in Chapter 9.

4.5 Three Ways to Pass an Argument

The same function, three ways. Only the first cannot change the caller's variable.

```
void byValue(int x)    { x = 99; }   // a copy. Caller unaffected.
void byPointer(int* x) { *x = 99; }  // an address. Caller's value changes.
void byRef(int& x)     { x = 99; }   // an alias. Caller's value changes.

int v = 1;
byValue(v);    // v is still 1
byPointer(&v); // v is now 99
byRef(v);      // v is now 99
```

```
after byValue:   1
after byPointer: 99
after byRef:     99
```

	Syntax at the call	Can be null?	Can be reseated?
by value	`f(v)`	—	—
by pointer	`f(&v)`	**yes**	yes
by reference	`f(v)`	no	no

A reference is an alias — another name for an existing object. Not a copy, and not a pointer you dereference. Once bound, it can never refer to anything else.

The row that decides most real choices is "can be null". Use a pointer when the argument is genuinely optional; use a reference when it is not.

And notice the trap in the first column. **At the call site, by-value and by-reference look identical** — `f(v)` either way. You must read the *signature* to know whether your variable can change. C# announces this at the call with `ref` and `out`; C++ does not, and expecting it to is a habit worth breaking now.

4.6 `const`, and Reading It Correctly

```
int a = 1, b = 2;

const int* p = &a;   // pointer to const int
*p = 5;              // ERROR - cannot change the value
p = &b;              // fine  - can point somewhere else

int* const q = &a;   // const pointer to int
*q = 5;              // fine  - can change the value
q = &b;              // ERROR - cannot point somewhere else
```

Read it right to left from the variable name. `int* const q` → "q is a const pointer to int". `const int* p` → "p is a pointer to const int". The `const` protects whatever it sits next to.

> **Predict before you run.** Four lines are marked above. Which of them fail to compile?
>
> Exactly two: `*p = 5;` and `q = &b;`.
>
> ```
> error: read-only variable is not assignable
>     4 |     *p = 5;
> error: cannot assign to variable 'q' with const-qualified type 'int *const'
>     8 |     q = &b;
> ```
>
> Most people predict all four, or the wrong pair. **The two that compile are as informative as the two that fail** — they prove each `const` guards a different operation. Predicting "all four" means reading `const` as "nothing here can change" without asking what it is attached to.

4.7 `const T&` — The One You Will Actually Write

```
void logReading(const Reading& r);   // no copy, and cannot modify
```

This is the default for passing anything bigger than a machine word. You avoid the copy a by-value parameter would make, and the `const` promises the caller you will not touch their object.

Break the promise and clang is specific about it:

```
void logIt(const Reading& r) {
    r.value = 99;      // ERROR
}
```

```
error: cannot assign to variable 'r' with const-qualified type 'const Reading &'
    3 |     r.value = 99;
      |     ~~~~~~~ ^
note: variable 'r' declared const here
    2 | void logIt(const Reading& r) {
```

Read the **second** line of that diagnostic. Clang does not just say what you did wrong; it points at where the promise was made. Getting into the habit of reading a whole diagnostic rather than its first line will save you more time this term than any other single thing.

4.8 Memory-Mapped I/O

On a microcontroller a hardware register is not reached through an API. It lives at a **fixed address**, and you talk to it by pointing at that address.

```
#include <cstdint>

// On real hardware: a number from the datasheet.
// Here: a simulated register we can actually run.
uint8_t simulatedRegister = 0;
volatile uint8_t* const GPIO_PORT = &simulatedRegister;

*GPIO_PORT |= (1 << 3);      // set bit 3 - Chapter 2's mask, through a pointer
*GPIO_PORT &= ~(1 << 3);     // clear it
```

On the real device that declaration would instead be:

```
volatile uint8_t* const GPIO_PORT = reinterpret_cast<volatile uint8_t*>(0x40021018);
```

Stop and notice what those two lines are. Chapter 2's bit masks, reached through Chapter 4's pointer indirection, at an address you chose. This is the first point in the course where the embedded material stops being an analogy.

Read the type carefully: *a const pointer to a volatile* `uint8_t`. The **address** never changes — the register is bolted down at a location the chip designer picked — but the **value** can change without warning, because hardware writes to it.

4.9 `volatile` — What It Actually Promises

`volatile` tells the compiler one thing: **this location can change without your knowledge, so do not optimise the access away.**

```
while (*STATUS_REG == 0) { }   // wait for hardware to set a bit
```

Without `volatile`, the compiler sees a loop reading a value that nothing in your program modifies, correctly concludes the value cannot change, reads it once, and rewrites the loop as `while (true) { }`. That is a valid optimisation of the code as written, and it hangs your device.

`volatile` is not a threading tool. The internet will tell you otherwise. It does not make anything atomic and it does not order operations between threads. It means exactly one thing — do not cache this access. Chapter 12 comes back to this properly.

4.10 Worked Example

Problem. Write a register interface that takes the register by pointer, and a logging function that takes a reading by `const` reference. Then break both on purpose.

```
#include <cstdint>
#include <bitset>
#include <iostream>
```

```
struct Reading { int milliVolts; };

void setBit(volatile uint8_t* reg, uint8_t bit)   { *reg |=  (1 << bit); }
void clearBit(volatile uint8_t* reg, uint8_t bit) { *reg &= ~(1 << bit); }
bool testBit(const volatile uint8_t* reg, uint8_t bit) { return (*reg & (1 << bit)) != 0; }

void logReading(const Reading& r) {
    std::cout << "reading: " << r.milliVolts << " mV\n";
}

int main() {
    uint8_t simulated = 0;
    volatile uint8_t* const PORT = &simulated;

    setBit(PORT, 3);
    setBit(PORT, 0);
    clearBit(PORT, 3);

    std::cout << std::bitset<8>(simulated) << "\n";
    std::cout << std::boolalpha << testBit(PORT, 0) << "\n";

    Reading r{3300};
    logReading(r);
}
```

```
00000001
true
reading: 3300 mV
```

Now break it twice.

Add `r.milliVolts = 0;` inside `logReading`, and you get the `const`-reference error from the previous section. Add `PORT = &other;` in `main`, and you get *cannot assign to variable 'PORT' with const-qualified type* — because the pointer itself is `const`.

Two different `const`s. Two different errors. On the same page. That is why this chapter teaches `const T*` and `T* const` together rather than separately: each one protects a different thing, and the only way to know which you need is to know which thing you are protecting.

4.11 Predict Before You Run

Predict before you run. Five lines of output.

```cpp
int a = 10;
int b = 20;
int* p = &a;

std::cout << *p << "\n";   // 1
*p = 15;
std::cout << a  << "\n";   // 2

p = &b;
std::cout << *p << "\n";   // 3
std::cout << a  << "\n";   // 4

*p = 25;
std::cout << b  << "\n";   // 5
```

10, 15, 20, 15, 25.

Line 4 is where people go wrong, usually answering 20. `p = &b` **moves the arrow**; it does not copy anything into `a`, and `a` does not follow `p` anywhere. Assignment to a *pointer* changes where it points. Assignment to a *dereferenced pointer* changes what it points at. Two different lines, two different targets.

4.12 Common Errors

Symptom	Cause	Fix
Changing the pointer when you meant the value	`p = x` instead of `*p = x`	Ask which box you are writing into
`*p + 1` does not give the next element	Dereference binds tighter than `+`	`*(p + 1)`
A function cannot find the array's length	The array decayed to a pointer at the call	Pass a count, or use `std::vector`/`std::array`
Caller's variable did not change	Passed by value	Take `T&`, or `T*` if it may be absent
`error: read-only variable is not assignable`	Writing through a `const T*` or `const T&`	Either the `const` is wrong or the write is
`cannot assign to variable ... const-qualified`	Reseating a `T* const`	The pointer is fixed by design; you want a different pointer
A hardware wait loop hangs forever	The compiler cached the read	Declare the register `volatile`

4.13 Key Terms

address-of (`&`) — the operator yielding the address of an object. Unrelated to `&` in a reference declaration.

alias — another name for an existing object. What a reference is.

array decay — the automatic conversion of an array to a pointer to its first element, losing the length.

`const T&` — a reference that cannot be written through. The default for passing anything larger than a machine word.

`const T*` — a pointer to something that cannot be modified through it. The pointer may be moved.

dereference (`*`) — the operator that reaches the object a pointer points at.

memory-mapped I/O — reaching hardware by reading and writing fixed memory addresses.

pass by reference — passing an alias, so the function operates on the caller's object.

pass by value — passing a copy, so the caller's object cannot change.

pointer arithmetic — arithmetic on pointers, scaled by the size of the pointed-to type.

`T* const` — a pointer that cannot be moved. What it points at may still be modified.

`volatile` — tells the compiler a location may change outside the program's control, so accesses must not be optimised away. Not a synchronization mechanism.

4.14 Exercises

1. **(Trace)** Draw the boxes-and-arrows diagram first. Then predict all five lines.

```
int a = 10;
int b = 20;
int* p = &a;

std::cout << *p << "\n";   // 1
*p = 15;
std::cout << a  << "\n";   // 2

p = &b;
std::cout << *p << "\n";   // 3
std::cout << a  << "\n";   // 4

*p = 25;
std::cout << b  << "\n";   // 5
```

2. **(Trace)** Predict all four lines.

```cpp
int arr[3] = {5, 10, 15};
int* p = arr;

std::cout << *p       << "\n";
std::cout << *(p + 1) << "\n";
std::cout << p[2]     << "\n";
std::cout << *p + 1   << "\n";
```

3. **(Trace)** Write down which lines fail before compiling. There are four candidates.

```cpp
int a = 1, b = 2;

const int* p = &a;
*p = 5;               // A
p = &b;               // B

int* const q = &a;
*q = 5;               // C
q = &b;               // D
```

4. **(Faded)** For each function, say whether the parameter should be by value, by pointer, by `const` reference, or by non-`const` reference — and why, in one sentence.

```cpp
// (a) returns true if the reading is above a threshold; does not modify it
bool isOverLimit(  ______ Reading r, int limit);

// (b) fills the caller's Reading with fresh sensor data
void sample(  ______ Reading r);

// (c) takes a Reading that may legitimately be absent
void logIfPresent(  ______ Reading r);

// (d) takes a loop counter
void repeat(  ______ int times);
```

5. **(Spaced recall, Chapters 2 + 3 → 4)** Complete the blanks. The function clears bit 2 of a chosen register, taking the array by pointer; the caller's memory must be freed.

```cpp
#include <cstdint>

void clearFault(________ registers, int index) {
    ________________________;
}

int main() {
    uint8_t* regs = new uint8_t[8]{0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
    clearFault(regs, 4);
    // ... print regs[4] in binary ...
    ________________;
}
```

6. **(Write from scratch)** Build the register interface that Assignment A1 will extend.
    1. A simulated 8-bit register: a `uint8_t` variable and a `volatile uint8_t* const` pointing at it.
    2. Functions `setBit`, `clearBit`, `toggleBit`, `testBit`, each taking the register **by pointer**.
    3. `testBit` must take a pointer that promises not to modify.
    4. A `main` that sets bits 0 and 3, clears bit 3, toggles bit 7, then prints the register in binary and reports bit 0 by name.
    5. Deliberately add `PORT = nullptr;` in `main`. Record the error. Remove it.
    6. Deliberately add `*reg = 0;` inside `testBit`. Record the error. Remove it.
    7. In two sentences: what did each of those errors protect you from?

    Predict the binary output before you run it.

4.15 Self-Check

1. In one sentence, what is a pointer?
2. `int* p = &a;` — what does `p` hold, and what does `*p` give you?
3. `int arr[4]; int* p = arr;` — what does `sizeof(arr)` give, and what does `sizeof(p)` give?
4. What does `*(p + 2)` mean, and how many bytes past `p` does it read for an `int*`?
5. What is the difference between `*p + 1` and `*(p + 1)`?
6. Which of these can be null?
    - A reference
    - A pointer
    - Both
    - Neither
7. True or false: you can tell from the call site `f(v)` whether `v` might be modified.
8. Read this aloud in words: `const int* p`. Then: `int* const q`.
9. Which line fails, and why? `const int* p = &a; p = &b;`

10. What should the parameter type be for a function that takes a large struct it will not modify — and why not by value?

11. What does `volatile` tell the compiler, and name one thing it does **not** do.

12. `volatile uint8_t* const PORT = ®` — which is fixed, the address or the value, and why is that the right way round for hardware?

4.16 Summary

Close the book and answer from memory.

- Draw the two-box diagram for `int* p = &value;` and label both addresses.
- What is lost when an array is passed to a function?
- Give the three ways to pass an argument, and the question that chooses between them.
- Read `const int*` and `int* const` aloud, and say what each protects.
- Why does a hardware register need `volatile`, and why does its pointer need `const`?

The next chapter puts a boundary around all of this. You will build your own types with constructors and destructors, watch them destroy in a guaranteed order, and meet the idea that makes C++ safe without a garbage collector: tying a resource's lifetime to an object's.

Part III

Objects and Resources

Chapter 5

Classes, Structs, and RAII

5.1 What You'll Be Able to Do

- State the only difference between `struct` and `class` in C++, and map both onto the OOP you already know.
- Write constructors and destructors, and use member initialisation lists.
- Predict construction and destruction order for members and for nested scopes.
- Use `this` and static members correctly.
- Explain RAII, and build a class that acquires a resource in its constructor and releases it in its destructor.
- Explain alignment and padding, predict a struct's `sizeof`, and say when packing is justified.

5.2 `struct` and `class` Are the Same Thing

In C++ there is exactly one difference between them: **default access**.

```
struct Point { int x; int y; };        // members public by default
class  Point { int x; int y; };        // members private by default
```

Everything else — constructors, destructors, inheritance, methods — works identically. The convention this course follows, and the one you will meet in embedded code:

- **`struct`** for plain data with no invariant to protect.
- **`class`** when the object owns something, or has to keep itself consistent.

This is a place your C# knowledge will actively mislead you. In C#, `struct` and `class` are genuinely different: one is a value type, the other a reference type, and that decides where instances live. That distinction does not carry over. In C++, *where* an object lives is decided by how you declare it, not by which keyword defined its type. A `class` can be a stack local; a `struct` can be `new`ed onto the heap.

5.3 Constructors, Destructors, and Member Init Lists

```cpp
class Sensor {
    int id_;
    double scale_;
public:
    Sensor(int id, double scale) : id_(id), scale_(scale) {   // member init list
        std::cout << "Sensor " << id_ << " ready\n";
    }
    ~Sensor() {
        std::cout << "Sensor " << id_ << " shut down\n";
    }
};
```

The `: id_(id), scale_(scale)` part is the **member initialisation list**. It *initialises* the members. You could assign them in the body instead:

```cpp
Sensor(int id, double scale) { id_ = id; scale_ = scale; }   // works, but
```

…that default-constructs them first and then overwrites — wasted work for a `double`, and outright impossible for a `const` member or a reference member, which must be initialised and can never be assigned.

Prefer the init list. Always.

One detail that will bite you: members are initialised in **declaration order**, not in the order you write them in the list. Write them out of order and clang tells you so with `-Wreorder-ctor`, which is one more reason the warnings stay on.

5.4 Order: Members First, Object Last

```cpp
class GpioPin {
    int pin_;
    Member m1_{"m1"};
    Member m2_{"m2"};
public:
    explicit GpioPin(int pin) : pin_(pin) { std::cout << "  GpioPin ACQUIRED\n"; }
    ~GpioPin() { std::cout << "  GpioPin released\n"; }
};
```

Construction: **members, in declaration order, then the constructor body.** Destruction: **the destructor body, then members in reverse order.**

Predict before you run. Six lines. Write them down.

```
    member m1 ctor
    member m2 ctor
  GpioPin 13 ACQUIRED
  GpioPin 13 released
    member m2 dtor
    member m1 dtor
```

> The common wrong answer puts `ACQUIRED` first. Think about what that would mean: if the body really ran first, `m1_` would not exist yet, and any use of it would be reading uninitialised memory.

That ordering is not a detail to memorise — it is what makes both bodies safe to write. By the time your constructor body runs, every member already exists, so you may use them. Your destructor body runs while the members are still alive, so you may use them there too.

5.5 `this` and Static Members

```
class Counter {
    int value_ = 0;
    static int instances_;          // declared here...
public:
    Counter() { ++instances_; }
    Counter& add(int n) { value_ += n; return *this; }   // return the object itself
    static int instances() { return instances_; }
};

int Counter::instances_ = 0;        // ...defined here, exactly once
```

- `this` is a **pointer to the current object**; `*this` is the object.
- A `static` member belongs to the **class**, not to any instance. There is exactly one, shared by every object.
- A `static` data member must be *defined* once outside the class.

That last point is worth trying deliberately. Omit the definition line and the class compiles perfectly — then the build fails with:

```
ld.lld: error: undefined symbol: Counter::instances_
```

A **linker** error, from a class the compiler was entirely happy with. That is Chapter 1's four-stage model paying off in a context that looks nothing like Chapter 1: the declaration was a promise, and nothing kept it.

5.6 RAII

Resource Acquisition Is Initialisation. The name is unhelpful; the idea is simple.

> **Acquire the resource in the constructor. Release it in the destructor. Then let scope do the work.**

Because destruction is deterministic — the guarantee from Chapter 3 — releasing is not something you remember to do. It is something the language does.

```
class GpioPin {
    int pin_;
public:
    explicit GpioPin(int pin) : pin_(pin) {
        std::cout << "pin " << pin_ << " configured as output\n";    // ACQUIRE
    }
    ~GpioPin() {
        std::cout << "pin " << pin_ << " released\n";                // RELEASE
    }
    void write(bool high) { /* ... */ }
};

void blink() {
    GpioPin led{13};
    led.write(true);
    // no cleanup code here. There is nothing to forget.
}
```

In C# you would write:

```
using (var led = new GpioPin(13)) { led.Write(true); }   // works - IF you write `using`
```

C# gives you deterministic disposal *when you remember the keyword.* C++ gives it to you because it is the language rule. Forgetting is not an available option.

The argument that should actually convince you is the early return.

```
void blink() {
    GpioPin led{13};
    if (!selfTest()) { return; }    // the pin is still released
    led.write(true);
}
```

Ask yourself where a hand-written `releasePin(13);` would have gone. At the bottom of the function — where this `return` never reaches. Every early return, and every error path someone adds later, is a place manual cleanup gets missed and RAII does not.

5.7 Alignment and Padding

```
struct Bad  { uint8_t flag; uint32_t value; uint8_t mode; };
struct Good { uint32_t value; uint8_t flag; uint8_t mode; };
```

Predict before you run. What is `sizeof` each one? Write both numbers down first.

```
sizeof(Bad)    = 12
sizeof(Good)   = 8
sum of fields  = 6
alignof(Bad)   = 4
offset flag    = 0
offset value   = 4      <-- 3 bytes of padding sit between them
offset mode    = 8
```

Nearly everyone answers 6 and 6. Padding is invisible in the source, which is exactly why it is worth predicting rather than being told.

A `uint32_t` must sit at an address divisible by 4. In `Bad`, the one-byte `flag` takes byte 0, so `value` cannot start until byte 4 — three bytes wasted — and the struct is then rounded up to a multiple of its alignment so the next element of an array is also aligned.

Same three fields, same three types, one third less memory — purely from declaration order. In an array of a few hundred, on a device with kilobytes of RAM, that is a real difference.

5.8 `#pragma pack`, and When It Is Justified

```
#pragma pack(push, 1)
struct Packed { uint8_t flag; uint32_t value; uint8_t mode; };
#pragma pack(pop)
```

`sizeof(Packed)` is **6** — exactly the sum of the fields.

This matters the moment your struct has to match something outside your program: a hardware register block, a binary file format, or bytes on a wire. The device does not know about your compiler's padding rules. If you `memcpy` a `Bad` onto a 6-byte protocol frame, you send twelve bytes and everything after it is garbage.

But packing is not free, and the honest rule is not "always pack": unaligned access is slower on x86 and on some architectures it **faults**. So — **pack only where an external format requires it, and reorder your fields everywhere else.** That judgment returns in Chapter 11, where you will write these structures to a file.

5.9 Worked Example

Problem. Build an RAII wrapper for a simulated peripheral, and prove it releases on every exit path.

```
#include <cstdint>
#include <iostream>

class GpioPin {
    uint8_t pin_;
    volatile uint8_t* port_;
public:
    GpioPin(uint8_t pin, volatile uint8_t* port) : pin_(pin), port_(port) {
```

```
        *port_ |= (1 << pin_);                      // ACQUIRE: drive the pin high
        std::cout << "pin " << int(pin_) << " acquired\n";
    }

    ~GpioPin() {
        *port_ &= ~(1 << pin_);                     // RELEASE: always
        std::cout << "pin " << int(pin_) << " released\n";
    }

    void write(bool high) {
        if (high) { *port_ |=  (1 << pin_); }
        else      { *port_ &= ~(1 << pin_); }
    }
};

bool selfTest() { return false; }                   // pretend it fails

void blink(volatile uint8_t* port) {
    GpioPin led{3, port};
    if (!selfTest()) {
        std::cout << "self-test failed, bailing out\n";
        return;                                     // early exit
    }
    led.write(true);
}

int main() {
    uint8_t simulated = 0;
    blink(&simulated);
    std::cout << "register now: " << int(simulated) << "\n";
}
```

```
pin 3 acquired
self-test failed, bailing out
pin 3 released
register now: 0
```

That final `0` is the proof. The register returned to its clean state even though the function exited from the middle, and nothing in `blink` says "clean up" — the closing brace does it.

Look at what is in this one class. Chapter 2's bit masks. Chapter 4's `volatile` pointer. Chapter 3's deterministic destruction. Chapter 5's constructor and destructor. This is the first point in the course where everything you have learned assembles into one thing.

5.10 Predict Before You Run

Predict before you run. Where does the output of the destructor appear relative to `end`?

```
int main() {
    std::cout << "start\n";
    { GpioPin led{13}; }
    std::cout << "end\n";
}
```

Before `end`. `led` is destroyed at the *inner* closing brace, not at the end of `main` — the same scope rule as Chapter 3, now with an object that does something on its way out.

Predict before you run. This class looks complete. What is not handled?

```
class Buffer {
    int* data_;
public:
    explicit Buffer(int n) : data_(new int[n]) { }
    ~Buffer() { delete[] data_; }
};
```

Copying. Nothing stops someone writing `Buffer b = a;` — after which both objects hold the *same* pointer, both are destroyed, and both destructors `delete[]` the same address.

RAII solves *lifetime*: when is this released. It does not, by itself, solve *ownership*: how many objects think they own it. That is the next chapter, and it opens by watching this exact class fail under the debugger.

5.11 Common Errors

Symptom	Cause	Fix
`-Wreorder-ctor` warning	Init list written in a different order from the declarations	Reorder the list to match; the compiler prints the correct order
Cannot initialise a `const` member	Assigning in the body instead of initialising	Use the member init list
`undefined symbol: C::member_`	A `static` data member declared but never defined	Add `int C::member_ = 0;` at file scope, once
Resource still held after an early return	Cleanup written as a statement at the bottom	Put the release in a destructor
`sizeof` is larger than the sum of the fields	Alignment padding	Expected — reorder largest-first if it matters
A struct does not match a wire format	Compiler padding	`#pragma pack` for that struct only
Assumed a `struct` is on the stack and a `class` on the heap	Carried over from C#	In C++, storage is decided by declaration, not keyword

5.12 Key Terms

alignment — the requirement that an object of a given type start at an address divisible by some number.

constructor — the special member function that runs when an object is created.

destructor — the special member function that runs when an object is destroyed. Written `~ClassName()`.

member initialisation list — the `: a_(x), b_(y)` clause that initialises members before the constructor body runs.

padding — unused bytes the compiler inserts between members to satisfy alignment.

`#pragma pack` — a directive removing padding, so a struct matches an external byte layout exactly.

RAII — Resource Acquisition Is Initialisation: acquire in the constructor, release in the destructor, and let scope guarantee the release.

`static` member — a member belonging to the class rather than to any instance, existing once and shared by all.

`this` — inside a member function, a pointer to the object the function was called on.

5.13 Exercises

1. **(Trace)** Write the complete output before compiling. Eight lines.

```
struct Member {
    std::string n;
    Member(std::string s) : n(std::move(s)) { std::cout << "    member " << n << " ctor\n"
    ~Member() { std::cout << "    member " << n << " dtor\n"; }
};

class GpioPin {
    int pin_;
    Member m1_{"m1"};
    Member m2_{"m2"};
public:
    explicit GpioPin(int pin) : pin_(pin) { std::cout << "  GpioPin " << pin_ << " ACQUIRE
    ~GpioPin() { std::cout << "  GpioPin " << pin_ << " released\n"; }
};

int main() {
    std::cout << "start\n";
    { GpioPin led{13}; }
    std::cout << "end\n";
}
```

2. **(Trace)** Write `sizeof` for each struct before compiling, then check.

```
struct Bad  { uint8_t flag; uint32_t value; uint8_t mode; };
struct Good { uint32_t value; uint8_t flag; uint8_t mode; };
```

3. **(Trace)** This class acquires something and never gives it back. Fix it — and say what would *still* have gone wrong if you had "fixed" it by adding a `release()` method for callers to call.

```
class FileLogger {
    std::FILE* file_;
public:
    explicit FileLogger(const char* path) {
        file_ = std::fopen(path, "w");
        std::cout << "opened " << path << "\n";
    }
    void write(const char* msg) {
        if (file_) { std::fputs(msg, file_); }
    }
};

void run() {
    FileLogger log{"run.txt"};
    log.write("starting\n");
    if (somethingWrong()) { return; }
    log.write("done\n");
}
```

4. **(Faded)** Complete this RAII wrapper. It must set its bit on construction and clear it on destruction.

```
class GpioPin {
    uint8_t pin_;
    volatile uint8_t* port_;
public:
    GpioPin(uint8_t pin, volatile uint8_t* port) : _________________ {
        ____________________;          // acquire
    }

    ________ {
        ____________________;          // release
    }

    void write(bool high) {
        if (high) { ____________________; }
        else      { ____________________; }
    }
};
```

5. **(Spaced recall, Chapters 3 + 4 → 5)** A colleague writes this and says "RAII means I do not have to worry about memory."

```
class Buffer {
    int* data_;
public:
    explicit Buffer(int n) : data_(new int[n]) { }
    ~Buffer() { delete[] data_; }
};

void use() {
    Buffer b{100};
}
```

1. Is the memory freed when `use()` returns? Explain using the rules from Chapter 3.
2. What would happen if the destructor said `delete data_;` instead?
3. Your colleague is right about this code and wrong in general. Name the thing that is not yet handled. *(You are not expected to know the fix — only to spot the gap.)*

6. **(Write from scratch)** Build a `SensorSession` class and prove it is early-return-safe.

   1. It takes a sensor id and a `volatile uint8_t*` register in its constructor.
   2. On construction: set the bit matching the sensor id, print `sensor N online`.
   3. On destruction: clear that bit, print `sensor N offline`.
   4. A `read()` method returning a fake value, printing `sensor N read`.
   5. A function `collect(port)` that creates a session, reads once, then returns early **before** a second read.
   6. In `main`, call `collect` and print the register in binary — it must be `00000000`.
   7. Add a second `SensorSession` in the same scope with a different id, and predict the destruction order before running.
   8. Define `struct SensorRecord { uint8_t id; uint32_t reading; uint8_t status; };`, predict its `sizeof`, check it, then reorder the fields to make it smaller. Record both numbers.

5.14 Self-Check

1. What is the only difference between `struct` and `class` in C++?
2. True or false: in C++, a `struct` is a value type on the stack and a `class` is a reference type on the heap.
3. In what order do these four things happen: constructor body, destructor body, member construction, member destruction?
4. Why can a `const` member not be initialised in the constructor body?
5. Members are initialised in which order?
   - The order written in the init list
   - Declaration order
   - Alphabetical order
   - Unspecified
6. What is `this`, and what is `*this`?

7. You declare a `static int` member and use it. The class compiles but the build fails with `undefined symbol`. What is missing, and which build stage reported it?
8. State RAII in one sentence, without using the words "resource acquisition is initialisation".
9. Why is a `release()` method that callers must remember to call not equivalent to a destructor?
10. `struct S { uint8_t a; uint32_t b; uint8_t c; };` — why is `sizeof(S)` 12 rather than 6, and how would you get it to 8?
11. When is `#pragma pack` the right choice, and what does it cost?
12. A class allocates in its constructor and frees in its destructor. What problem does that still leave unsolved?

5.15 Summary

Close the book and answer from memory.

- What decides whether a C++ object lives on the stack or the heap?
- Give the full construction and destruction order for an object with two members.
- Why must a `static` data member be defined outside the class?
- State RAII in your own words, and give the argument from early returns.
- Why does field order change `sizeof`, and when is packing justified?
- What does RAII *not* solve?

The next chapter answers that last question. You will watch a resource-owning class get copied, see the same memory freed twice under the debugger, and meet the tools that make ownership explicit: the rule of three, move semantics, and `unique_ptr`.

Chapter 6

Smart Pointers, Copy Semantics, and Custom Allocation

6.1 What You'll Be Able to Do

- Use `std::unique_ptr` to express exclusive ownership, and rewrite raw `new`/`delete` code with it.
- Explain move semantics as far as ownership transfer needs, and use `std::move`.
- Say what the compiler writes for you when you write a class, and predict the double-free when a resource-owning class is copied.
- State the rule of three, the rule of five, and the rule of zero, and apply the minimum fix.
- Use `std::shared_ptr` and `std::weak_ptr`, and explain reference counting and what it costs.
- Explain overloading `operator new`/`delete`, placement `new`, and why fixed-size pools matter where the heap is forbidden.

The previous chapter ended on a question: RAII settles *when* a resource is released, but not how many objects believe they own it. This chapter answers it.

6.2 `std::unique_ptr` — Ownership You Cannot Forget

Chapter 3 left you with a rule you had to remember: every `new` needs a `delete`. `unique_ptr` makes that rule the compiler's job.

```cpp
#include <memory>

void oldWay() {
    Sensor* s = new Sensor(1);
    // ... every early return from here leaks ...
    delete s;
}

void newWay() {
    auto s = std::make_unique<Sensor>(1);
    // ... no delete. No path that leaks. Nothing to forget.
}
```

`unique_ptr` is an RAII class — exactly Chapter 5's idea — whose resource happens to be heap memory. It acquires in its constructor and `delete`s in its destructor.

And it costs nothing:

```
sizeof(int*)                  = 8
sizeof(unique_ptr<Sensor>)    = 8
```

One pointer. The safety is enforced entirely at compile time, so there is no run-time object to pay for. If you have been told smart pointers are too expensive for embedded work, that claim is about `shared_ptr`, which you will meet later in this chapter — and it is only half true even there.

6.3 Move: How Ownership Travels

There is exactly one `unique_ptr` for a resource. So it cannot be copied — but it can be handed over.

```cpp
auto a = std::make_unique<Sensor>(1);
auto b = std::move(a);        // ownership transfers
```

```
Sensor 1 ctor
a holds: yes
after move, a holds: no
after move, b holds: yes
```

`std::move` does not move anything. It is a cast that says *"I am done with this; you may take its guts."* The type on the receiving end decides what that means. After the transfer `a` is empty — still a valid object you may assign to or destroy, just not one that owns anything.

6.4 Try to Copy One, and Read the Error

```cpp
auto a = std::make_unique<Sensor>();
auto b = a;          // ERROR
```

```
error: call to implicitly-deleted copy constructor of 'unique_ptr<Sensor>'
    5 |     auto b = a;
      |          ^   ~
note: copy constructor is implicitly deleted because 'unique_ptr<Sensor>' has a
      user-declared move constructor
```

> **Stop and answer before reading on.** Why would the authors of the standard library go to the trouble of forbidding this? What would the copy have done?

Two `unique_ptr`s. One address. Two destructors, each calling `delete` on it.

Now hold that thought, because your `GpioPin` from the last chapter has exactly the same problem — except that nobody forbade anything.

6.5 The Copy You Did Not Write

Write a class and the compiler quietly writes four more functions for you: a copy constructor, a copy assignment operator, a destructor, and — conditions permitting — the two move operations.

The compiler-generated copy constructor copies each member. For a raw pointer member, **copying the pointer is exactly the wrong thing.**

```
class Buffer {
    int* data_;
public:
    explicit Buffer(int n) : data_(new int[n]) { }
    ~Buffer() { delete[] data_; }
};

int main() {
    Buffer a{4};
    Buffer b = a;      // the copy constructor you never wrote
}
```

With the addresses printed:

```
alloc  0x103351640
end of main
free   0x103351640
free   0x103351640
```

Then the program aborts.

The two identical addresses are the whole lesson. One allocation, freed twice, because two objects each believed they owned it.

Note the ordering too: `end of main` prints before either `free`, because both objects are destroyed at `main`'s closing brace. That is Chapter 3's scope rule, still holding.

6.6 The Rule of Three, Five, and Zero

Rule of three: if you write any of destructor, copy constructor, or copy assignment, you almost certainly need all three. They are three halves of one decision — *how is this resource owned?* — and answering it once is not enough.

Rule of five adds the two move operations, which the compiler stops generating for you once you declare any of the three.

You have three ways to fix `Buffer`, and they are not equally good.

```cpp
// 1. Forbid copying. Honest, and one line each.
Buffer(const Buffer&) = delete;
Buffer& operator=(const Buffer&) = delete;

// 2. Implement copying. Correct, and now you own five functions forever.
Buffer(const Buffer& other) : data_(new int[other.n_]), n_(other.n_) {
    std::copy(other.data_, other.data_ + n_, data_);
}

// 3. Stop owning a raw pointer. Nothing to write at all.
class Buffer {
    std::unique_ptr<int[]> data_;
public:
    explicit Buffer(int n) : data_(std::make_unique<int[]>(n)) { }
};
```

Rule of zero: design your classes so you write none of the five. Hold resources in types that already manage themselves — `unique_ptr`, `vector`, `string` — and the compiler-generated operations are correct for free.

Count the difference. Option 2 is roughly twenty lines that must stay correct under every future edit. Option 3 is zero lines. And notice what happened to the copy in option 3: it will not compile either, because `unique_ptr` is not copyable and that propagates to the containing class. The class inherited the right behaviour without anyone writing it.

6.7 `std::shared_ptr` — Ownership Shared, and Counted

Sometimes exclusive ownership is genuinely the wrong model: several parts of a system need the same object, and none of them can say which will finish last.

```cpp
auto s1 = std::make_shared<Sensor>(2);   // use_count = 1
{
    auto s2 = s1;                        // use_count = 2  (copying is allowed)
    ...
}                                        // s2 dies         use_count = 1
```

```
Sensor 2 ctor
use_count = 1
use_count = 2
use_count = 1
-- end of main --
Sensor 2 dtor
```

The object is destroyed when the **last** `shared_ptr` goes away. That is the feature. Here is the cost:

```
sizeof(unique_ptr<Sensor>)    = 8
sizeof(shared_ptr<Sensor>)    = 16
```

Twice the size, because a `shared_ptr` carries a pointer to the object *and* a pointer to a separately allocated control block holding the counts. Every copy and every destruction adjusts that count, and in threaded code those adjustments must be atomic — which is not free.

Reach for `unique_ptr` first. Use `shared_ptr` when you can name the second owner. If you cannot name it, you do not have shared ownership; you have a design you have not finished.

6.8 `weak_ptr`, and the Cycle

Two `shared_ptr`s that point at each other never reach zero.

```
struct Node {
    std::shared_ptr<Node> peer;    // if two Nodes point at each other...
};
```

Neither count ever drops to zero, so neither destructor ever runs, and the memory leaks by a mechanism whose entire purpose was to prevent leaks.

`std::weak_ptr` is a non-owning observer: it can see the object, does not keep it alive, and must be `lock()`ed to produce a usable `shared_ptr`. Recognise the cycle when you meet it; you are not expected to build one.

6.9 Custom Allocation, and Why an Embedded Course Cares

Chapter 3 gave you the rule: on a microcontroller, avoid the heap. This section is what you do instead.

Overloading `operator new/delete` lets a class decide where its instances come from:

```
class Packet {
public:
    static void* operator new(std::size_t);
    static void operator delete(void*);
};
```

Placement `new` constructs an object *in memory you already have*, allocating nothing:

```
unsigned char storage[sizeof(Packet)];
Packet* p = new (&storage) Packet{};   // construct here
p->~Packet();                          // and destroy explicitly - no delete
```

That pairing — placement `new` plus an explicit destructor call — is the one place in this course where you call a destructor by hand. It is the machinery underneath every container you have used.

A fixed pool puts the two together: reserve storage for N objects at compile time, hand out slots, and never touch the heap.

```cpp
template <typename T, std::size_t N>
class Pool {
    alignas(T) unsigned char storage_[sizeof(T) * N];
    bool used_[N]{};
public:
    T* acquire() {
        for (std::size_t i = 0; i < N; ++i) {
            if (!used_[i]) {
                used_[i] = true;
                return new (&storage_[i * sizeof(T)]) T{};   // placement new
            }
        }
        return nullptr;                                       // exhausted, and it says so
    }
    void release(T* p) {
        if (!p) { return; }
        p->~T();
        auto idx = (reinterpret_cast<unsigned char*>(p) - storage_) / sizeof(T);
        used_[idx] = false;
    }
};
```

```
a: ok
b: ok
c: null   (pool exhausted, no heap touched)
d: ok   reused a's slot: yes
sizeof(Pool<Packet,2>) = 20
```

Read that against Chapter 3's four objections to the heap. There is no search, so allocation is **constant time**. There is no fragmentation, because every slot is the same size. The memory cost is **known at compile time** — 20 bytes, visible in `sizeof`. And exhaustion returns `nullptr` at a point you control, rather than failing somewhere with no way to report it.

The `alignas` is there because an object of type `T` must start at a properly aligned address — Chapter 5's rule. Raw `unsigned char` storage carries no such guarantee unless you ask for it.

6.10 Worked Example

Problem: take Chapter 3's leaking function and Chapter 5's unsafe class, and fix both by changing what owns the memory.

Before — two problems, one cause:

```
class Buffer {
    int* data_;
    int  n_;
public:
    explicit Buffer(int n) : data_(new int[n]), n_(n) { }
    ~Buffer() { delete[] data_; }
    int& operator[](int i) { return data_[i]; }
};

Buffer makeBuffer(int n) {
    Buffer b{n};
    for (int i = 0; i < n; ++i) { b[i] = i * 10; }
    return b;              // is this a copy? what happens to the pointer?
}
```

After — hold the resource in something that owns it:

```
#include <memory>

class Buffer {
    std::unique_ptr<int[]> data_;
    int n_;
public:
    explicit Buffer(int n) : data_(std::make_unique<int[]>(n)), n_(n) { }
    int& operator[](int i) { return data_[i]; }
    // no destructor. no copy constructor. no copy assignment. nothing.
};
```

What changed, precisely:

- The destructor is **gone**, because `unique_ptr` has one.
- Copying is **forbidden**, because `unique_ptr` cannot be copied and that propagates to the containing class — the double-free is now uncompilable rather than merely unlikely.
- Moving still **works**, because `unique_ptr` is movable, so `return b;` transfers ownership out of the function.
- The rule of five is satisfied by writing **none** of it.

Try to copy the new `Buffer` and you get the same error you met at the start of this chapter — this time in your own class, because you caused it.

6.11 Predict Before You Run

Predict before you run. How many lines, and in what order?

```cpp
struct Noisy {
    int id;
    explicit Noisy(int i) : id(i) { std::cout << "ctor " << id << "\n"; }
    ~Noisy() { std::cout << "dtor " << id << "\n"; }
};

int main() {
    auto a = std::make_unique<Noisy>(1);
    auto b = std::move(a);
    std::cout << "a holds: " << (a ? "yes" : "no") << "\n";
    std::cout << "end\n";
}
```

```
ctor 1
a holds: no
end
dtor 1
```

One destructor, not two — there was only ever one object. The move transferred ownership; it did not duplicate anything. And the destruction happens at the end of `main`, when `b` dies.

Predict before you run. Does this leak? If so, how much?

```cpp
struct Node {
    std::shared_ptr<Node> peer;
    ~Node() { std::cout << "Node dtor\n"; }
};

int main() {
    auto a = std::make_shared<Node>();
    auto b = std::make_shared<Node>();
    a->peer = b;
    b->peer = a;
    std::cout << "end of main\n";
}
```

```
end of main
```

Nothing else. Neither destructor runs: `a`'s count never falls below 1 because `b->peer` holds it, and vice versa. Both `Node`s leak, along with their control blocks — from the mechanism whose purpose was to prevent exactly this.

6.12 Common Errors

Symptom	Cause	Fix
`call to implicitly-deleted copy constructor`	Trying to copy a `unique_ptr`, or a class holding one	Move it (`std::move`), or take a reference

Symptom	Cause	Fix
Same address freed twice, then an abort	The compiler-generated copy constructor copied a raw pointer	Hold the resource in a `unique_ptr`, or `= delete` the copies
Object destroyed while still in use	A raw pointer outlived the `unique_ptr` that owned it	Do not keep raw copies of an owned pointer
A `shared_ptr` graph never frees	A reference cycle	Make one direction a `weak_ptr`
Program is slower than expected in a hot loop	`shared_ptr` passed by value; atomic count churn	Take `const T&` when the function only reads
`use of deleted function` on your own class	It holds a `unique_ptr`, so it inherited non-copyable	Intended — move it instead
Crash using pool memory	Storage not aligned for `T`	`alignas(T)` on the raw byte array

6.13 Key Terms

`std::unique_ptr` — a smart pointer expressing exclusive ownership. Non-copyable, movable, and the same size as a raw pointer.

`std::make_unique<T>(...)` — allocates a `T` on the heap and returns a `unique_ptr<T>` owning it. Preferred over `unique_ptr<T>(new T(...))`.

`std::move` — a cast meaning *"I am finished with this, you may take its guts."* It performs nothing at run time.

rule of three — if you write any of destructor, copy constructor, or copy assignment, you almost certainly need all three.

rule of five — the rule of three plus the move constructor and move assignment operator.

rule of zero — design so you write none of the five, by holding resources in types that manage themselves.

`std::shared_ptr` — a smart pointer allowing multiple owners; the object is destroyed when the last one goes away.

reference count — the number of `shared_ptr`s currently owning an object. Copying increments it; destruction decrements it.

`std::weak_ptr` — a non-owning observer of a `shared_ptr`'s object; must be `lock()`ed to be used.

placement `new` — constructs an object in memory you already have, allocating nothing.

pool allocator — a fixed block of storage divided into equal slots, handed out in constant time without touching the heap.

6.14 Exercises

1. **(Trace)** Write the complete output before compiling.

```cpp
struct Res {
    int id;
    explicit Res(int i) : id(i) { std::cout << "acquire " << id << "\n"; }
    ~Res() { std::cout << "release " << id << "\n"; }
};

void hand(std::unique_ptr<Res> r) {
    std::cout << "  received " << r->id << "\n";
}

int main() {
    auto a = std::make_unique<Res>(1);
    std::cout << "before\n";
    hand(std::move(a));
    std::cout << "after, a holds: " << (a ? "yes" : "no") << "\n";
}
```

2. **(Trace)** Predict `use_count` at each marked point, then check.

```cpp
auto p = std::make_shared<int>(42);
std::cout << p.use_count() << "\n";        // (A)
{
    auto q = p;
    std::cout << p.use_count() << "\n";    // (B)
    auto r = std::move(q);
    std::cout << p.use_count() << "\n";    // (C)
}
std::cout << p.use_count() << "\n";        // (D)
```

3. **(Trace)** Which of these compile? For each that does not, say what the error will be before you try it.

```cpp
std::unique_ptr<int> a = std::make_unique<int>(1);
std::unique_ptr<int> b = a;                    // (i)
std::unique_ptr<int> c = std::move(a);         // (ii)
std::shared_ptr<int> d = std::make_shared<int>(2);
std::shared_ptr<int> e = d;                    // (iii)
std::unique_ptr<int> f = d;                    // (iv)
```

4. **(Faded)** Complete the class so it follows the rule of zero. It must own a heap array of `n` readings and be movable but not copyable — and you should not write a destructor.

```
class Readings {
    ____________________ data_;      // (A) the member type
    std::size_t n_;
public:
    explicit Readings(std::size_t n)
        : data_(____________________), n_(n) { }   // (B) allocate

    double& operator[](std::size_t i) { return data_[i]; }
    std::size_t size() const { return n_; }
};
```

5. **(Spaced recall — Chapters 3 and 5)** A `unique_ptr<Sensor>` owns a `Logger`, which itself holds a `unique_ptr<int[]>`. Nothing in either class has a hand-written destructor. When the outer `unique_ptr` goes out of scope, what frees the array, and how many `delete`s run in total? Explain in terms of the destruction order from Chapter 5.

6. **(Find the bug)** This compiles and runs, and is wrong. Say what happens, and fix it with the smallest change.

```
class Device {
    std::unique_ptr<Buffer> buf_;
public:
    Device() : buf_(std::make_unique<Buffer>(64)) { }
    Buffer* buffer() { return buf_.get(); }
};

Buffer* getBuffer() {
    Device d;
    return d.buffer();
}
```

7. **(Write from scratch)** Write a `PacketStore` that holds up to four `Packet` objects in a fixed pool with no heap allocation. It must expose `acquire()` returning `Packet*` or `nullptr` when full, and `release(Packet*)`. Prove with output that the fifth `acquire()` returns `nullptr`, that a released slot is reused, and that `sizeof(PacketStore)` is known at compile time. State what your design costs compared with `new`/`delete`.

6.15 Self-Check

1. How big is a `unique_ptr` compared with a raw pointer?
   - Twice the size — it stores a reference count
   - The same size: 8 bytes on a 64-bit machine
   - Larger by one byte, for an ownership flag
   - It depends on the type it points to

2. What does `std::move` actually do?
   - Copies the object to a new location and frees the old one

- Nothing at run time — it is a cast meaning "I am finished with this, you may take its guts"
- Swaps two objects
- Marks the object for the garbage collector

3. Why will `auto b = a;` not compile when `a` is a `unique_ptr`?
4. You write a class with a destructor and nothing else. Which functions may the compiler write for you? Select every one.
   - A copy constructor
   - A copy assignment operator
   - A move constructor
   - A default constructor taking your destructor's arguments
5. What is the rule of zero, and why is it preferred?
6. A class holds a `std::unique_ptr` member. Can the class be copied? Can it be moved?
7. Select every true statement about `shared_ptr`.
   - The object is destroyed when the last `shared_ptr` to it goes away
   - It is twice the size of a `unique_ptr`
   - Two `shared_ptr`s pointing at each other will still be cleaned up correctly
   - Every copy and destruction adjusts a reference count
8. Should a function that only *reads* an object take a `std::shared_ptr` parameter by value?
9. What does placement `new` allocate?
10. Name the four objections to the heap from Chapter 3, and say how a fixed pool answers each.

6.16 Summary

Close the book and answer from memory.

- What does `unique_ptr` cost, in bytes and at run time?
- Why is `unique_ptr` non-copyable, and what would a copy have done?
- What five functions might the compiler write for you, and what stops it?
- State the rule of three and the rule of zero, and say which you should reach for.
- When is `shared_ptr` the right answer, and what does it cost?
- How does a fixed pool answer each of Chapter 3's objections to the heap?

The next chapter turns from ownership to behaviour: how one pointer can call different code depending on what it actually points at, what that dispatch costs in bytes and cycles, and the one-word omission that makes a base-class pointer leak everything the derived class owned.

Chapter 7

Polymorphism and Abstract Base Classes

7.1 What You'll Be Able to Do

- Map C# interfaces and abstract classes onto C++ abstract base classes and pure virtual functions.
- Explain runtime versus compile-time polymorphism, and say which a problem needs.
- Explain the vtable — the hidden pointer added to polymorphic classes — and what it costs in RAM and indirection.
- Explain why a base class with virtual functions needs a `virtual` destructor.
- Recognise object slicing and say what causes it.
- Name Interface Segregation and Dependency Inversion in code you have written.

7.2 What You Already Know, and What Is Different

You have written interfaces and abstract classes in C#. Almost all of that transfers.

C#	C++
`interface ISensor`	abstract base class with pure virtual functions
`abstract class Sensor`	class with at least one `= 0` member function
`abstract int Read();`	`virtual int read() const = 0;`
`override`	`override` (optional, but always write it)
every method virtual by default	**nothing is virtual unless you say so**
`IDisposable`	destructor — and it had better be `virtual`

Three differences will actually hurt you:

1. In C#, dispatch is the norm. In C++, **a function is non-virtual unless you write `virtual`**, and non-virtual is the default *because dispatch costs something.*
2. C++ objects can be held **by value**, which C# reference types cannot. That makes slicing possible.

3. C++ will let you destroy a derived object through a base pointer with a non-virtual destructor, and simply not run the derived destructor.

7.3 An Abstract Base Class

```
class Sensor {                                      // abstract base
public:
    virtual int readMilliVolts() const = 0;         // pure virtual - no body
    virtual const char* name() const = 0;
    virtual ~Sensor() = default;                    // REQUIRED - see below
};

class Thermocouple : public Sensor {
public:
    int readMilliVolts() const override { return 1250; }
    const char* name() const override { return "thermocouple"; }
};
```

`= 0` makes the function pure virtual and the class abstract. You cannot create one:

```
error: variable type 'S' is an abstract class
    3 | int main(){ S s; }
      |               ^
note: unimplemented pure virtual method 'r' in 'S'
```

Read the **second** line. Clang names exactly which pure virtual you failed to implement, and that is the message you will see most often when adding a new derived class.

Using it looks like C#, with one addition:

```
std::vector<std::unique_ptr<Sensor>> sensors;
sensors.push_back(std::make_unique<Thermocouple>());
sensors.push_back(std::make_unique<Potentiometer>());
for (const auto& s : sensors) {
    std::cout << s->name() << ": " << s->readMilliVolts() << " mV\n";
}
```

```
thermocouple: 1250 mV
potentiometer: 3300 mV
```

The addition is `unique_ptr`. In C# the container would hold references and the garbage collector would deal with lifetime. Here the container owns the objects, and Chapter 6's rule of zero means you write nothing to make that correct.

7.4 The vtable, and What It Costs

Add one virtual function to a class and the compiler adds a hidden pointer to every instance: the **vtable pointer**, pointing at a per-class table of function addresses. A virtual call reads that pointer, indexes the table, and jumps.

```
sizeof(Plain)       = 4      // struct { int a; void ping(); }
sizeof(Polymorphic) = 16     // struct { int a; virtual void ping(); virtual ~...; }
```

Four bytes of data became sixteen. Eight bytes of vtable pointer, plus four of padding to keep that pointer aligned — Chapter 5's rule, still applying. The class did not gain a member; it gained the *ability to dispatch*.

There is a second cost, smaller and harder to see: a virtual call usually cannot be inlined, because the target is not known until run time. In a tight loop that matters. In a sensor read that happens ten times a second, it does not.

Notice that the cost is per *object*, not per class. Ten thousand `Polymorphic` objects cost eighty thousand bytes of vtable pointers. On a microcontroller with 64 KB of RAM, that is the whole calculation.

7.5 Runtime versus Compile-Time Polymorphism

Runtime polymorphism is what you just built: one pointer type, several possible objects, the decision made while the program runs. It costs a vtable pointer per object and an indirect call.

Compile-time polymorphism is templates, in the next chapter: the type is chosen while the program is *built*, a separate function is generated for each, and there is no vtable and no indirection.

The question that chooses between them: **do you know the set of types at compile time?** A container of mixed sensor types decided by a config file needs runtime dispatch. A `RingBuffer` holding `int` in one place and `double` in another does not.

7.6 The `virtual` Destructor Rule

```
struct Sensor {
    virtual void read() const { std::cout << "generic read\n"; }
    ~Sensor() { std::cout << "Sensor dtor\n"; }        // NOT virtual - the bug
};
struct Thermo : Sensor {
    void read() const override { std::cout << "thermo read\n"; }
    ~Thermo() { std::cout << "Thermo dtor\n"; }
};

Sensor* p = new Thermo();
delete p;
```

Predict before you run. Which destructors run, and in what order?

```
Sensor dtor
```

That is all. **`Thermo`'s destructor never runs.** Anything it owned is leaked, and if it held a `unique_ptr` member, that member is never destroyed either.

What makes this genuinely surprising is that the *function call* dispatches correctly. `read()` finds the override. Only destruction fails, because destruction is polymorphic only if you say so.

Clang warns:

```
warning: delete called on non-final 'Sensor' that has virtual functions but
         non-virtual destructor [-Wdelete-non-abstract-non-virtual-dtor]
```

Make the destructor `virtual` and both run, derived first:

```
VDerived dtor
VBase dtor
```

The rule: if a class has any virtual function, give it a virtual destructor. If it is never deleted through a base pointer you may not need one — but you cannot know what someone will do with your class in six months, and the cost is one keyword.

7.7 Object Slicing

C++ lets you hold objects by value, and that is where this bites.

```
void byValue(Sensor s) { s.read(); }          // slicing
void byRef(const Sensor& s) { s.read(); }

Thermo t;
byValue(t);
byRef(t);
```

```
-- by value --
generic read
Sensor dtor
-- by reference --
thermo read
```

`byValue` copied the `Sensor` **part** of the `Thermo` into a new object and threw the rest away — including the vtable pointer, which is why the override is gone. The object was not converted; it was *sliced*.

Polymorphism requires indirection. Pass by reference, by pointer, or by `unique_ptr`. Never by value.

The dangerous part is that the two call sites look identical. `byValue(t)` and `byRef(t)` are the same characters. You have to read the signature to know which one you got.

7.8 Two SOLID Principles, Named Where They Were Earned

You just built an abstract base with two small pure virtual functions, and code that uses `Sensor` without knowing which sensor it has. That is two principles:

- **Interface Segregation** — `Sensor` declares `readMilliVolts` and `name`, and nothing else. A derived class is not forced to implement functions it has no use for. Fat interfaces push work onto every implementer.
- **Dependency Inversion** — the loop depends on the *abstraction* `Sensor`, not on `Thermocouple`. Adding a new sensor type changes no existing code.

You will meet the other three SOLID principles in Chapter 13, once there is more code to apply them to.

7.9 Worked Example

Problem: build a sensor hierarchy that works polymorphically.

```
#include <iostream>
#include <memory>
#include <vector>

class Sensor {
public:
    virtual int readMilliVolts() const = 0;
    virtual const char* name() const = 0;
    virtual ~Sensor() = default;
};

class Thermocouple : public Sensor {
public:
    int readMilliVolts() const override { return 1250; }
    const char* name() const override { return "thermocouple"; }
};

class Potentiometer : public Sensor {
public:
    int readMilliVolts() const override { return 3300; }
    const char* name() const override { return "potentiometer"; }
};

int main() {
    std::vector<std::unique_ptr<Sensor>> sensors;
    sensors.push_back(std::make_unique<Thermocouple>());
    sensors.push_back(std::make_unique<Potentiometer>());
    for (const auto& s : sensors) {
        std::cout << s->name() << ": " << s->readMilliVolts() << " mV\n";
    }
    std::cout << "sizeof(Thermocouple) = " << sizeof(Thermocouple) << "\n";
}
```

```
thermocouple: 1250 mV
potentiometer: 3300 mV
sizeof(Thermocouple) = 8
```

`sizeof(Thermocouple)` is 8 — it has no data members at all, so the entire object *is* the vtable pointer.

Now break it three ways and read each failure. This is the part worth doing at the keyboard:

1. **Make the destructor non-virtual**, give `Thermocouple` a `unique_ptr` member with a

printing destructor, and delete through the base pointer. The member's destructor never runs. The leak is now visible.

2. **Add `void logAll(std::vector<Sensor> v)`** taking by value. It will not compile — you cannot instantiate an abstract class. That is a *useful* failure: the abstractness caught the slicing attempt at compile time.
3. **Misspell an override** (`readMillivolts`) and omit `override`. It silently becomes a new function, the base's pure virtual stays unimplemented, and the error appears at the *construction* site rather than the definition. That is the argument for always writing `override`.

7.10 Predict Before You Run

Predict before you run. Two questions: what prints, and what is `sizeof(Derived)`?

```
struct Base {
    int a = 1;
    virtual void who() const { std::cout << "Base\n"; }
    virtual ~Base() = default;
};
struct Derived : Base {
    int b = 2;
    void who() const override { std::cout << "Derived\n"; }
};

int main() {
    Derived d;
    Base& r = d;
    r.who();
    std::cout << sizeof(Base) << " " << sizeof(Derived) << "\n";
}
```

```
Derived
16 16
```

`r` is a reference, so no slicing and the override runs. `Base` is 16: 8 bytes of vtable pointer, 4 of `int`, 4 of padding. `Derived` is **also** 16 — its `int b` lands in the padding `Base` already had, so the derived class costs nothing extra. Chapter 5's padding rule, paying off in a place you would not think to look.

Change `Base& r = d;` to `Base r = d;` and the output becomes `Base` — the same two characters that separate correct dispatch from a sliced copy.

7.11 Common Errors

Symptom	Cause	Fix
`variable type 'S' is an abstract class`	Instantiating a class with an unimplemented pure virtual	Read the `note:` line — it names the function
Base version runs instead of the override	The object was passed or stored **by value** — sliced	Pass by reference, pointer, or `unique_ptr`

Symptom	Cause	Fix
Derived destructor never runs; memory leaks	Base destructor not `virtual`	Add `virtual ~Base() = default;`
`-Wdelete-non-abstract-non-virtual-dtor`	Deleting through a base pointer with a non-virtual destructor	Same fix; the warning is telling you exactly this
Override silently ignored	Misspelled name or wrong `const`; no `override` keyword to catch it	Always write `override`
`sizeof` jumped unexpectedly	The first `virtual` added a vtable pointer	Expected — budget one pointer per object
Cannot put objects in a `vector<Base>`	Value semantics slice; abstract classes cannot be instantiated	`vector<unique_ptr<Base>>`

7.12 Key Terms

abstract base class — a class with at least one pure virtual function; cannot be instantiated.

pure virtual function — a virtual function declared `= 0`, with no body, that derived classes must implement.

`override` — an optional keyword asserting that a function overrides a base virtual. Always write it; it turns a silent mistake into a compile error.

vtable — a per-class table of function addresses used to dispatch virtual calls.

vtable pointer — the hidden pointer added to every instance of a polymorphic class, costing 8 bytes on a 64-bit machine.

virtual destructor — a destructor declared `virtual`, so deleting through a base pointer runs the derived destructor too.

object slicing — copying a derived object into a base-typed value, discarding the derived part including the vtable pointer.

runtime polymorphism — dispatch decided while the program runs, via virtual functions.

compile-time polymorphism — dispatch decided while the program is built, via templates.

Interface Segregation — keep interfaces small so implementers are not forced to write functions they do not need.

Dependency Inversion — depend on abstractions rather than concrete types.

7.13 Exercises

1. **(Trace)** Write the complete output before compiling.

```cpp
struct A {
    A() { std::cout << "A ctor\n"; }
    virtual void speak() const { std::cout << "A\n"; }
    virtual ~A() { std::cout << "A dtor\n"; }
};
struct B : A {
    B() { std::cout << "B ctor\n"; }
    void speak() const override { std::cout << "B\n"; }
    ~B() override { std::cout << "B dtor\n"; }
};

int main() {
    std::unique_ptr<A> p = std::make_unique<B>();
    p->speak();
    std::cout << "-- end --\n";
}
```

2. **(Trace)** Predict `sizeof` for each, then check.

```cpp
struct S1 { int a; };
struct S2 { int a; void f(); };
struct S3 { int a; virtual void f(); virtual ~S3() = default; };
struct S4 : S3 { int b; };
```

3. **(Trace)** Three of these are wrong. Find them, and say what each does at run time.

```cpp
struct Shape {
    virtual double area() const = 0;
    ~Shape() = default;                              // (i)
};
struct Circle : Shape {
    double r;
    double area() const override { return 3.14159 * r * r; }
};

void print(Shape s);                                 // (ii)
void report(const Shape& s) { std::cout << s.area(); }

int main() {
    Shape* p = new Circle{};
    delete p;                                        // (iii)
}
```

4. **(Faded)** Complete the abstract base and the derived class so the loop below dispatches correctly and leaks nothing.

```
class Actuator {
public:
    ____ void engage() ____ = 0;       // (A) pure virtual, const
    ____ ~Actuator() = ____;           // (B) the keyword and the body
};

class Relay : ____ Actuator {          // (C) inheritance
public:
    void engage() const ____ {         // (D) assert the override
        std::cout << "relay engaged\n";
    }
};

int main() {
    std::vector<std::unique_ptr<Actuator>> parts;
    parts.push_back(std::make_unique<Relay>());
    for (const auto& p : parts) { p->engage(); }
}
```

5. **(Spaced recall — Chapters 5 and 6)** A `std::unique_ptr<Sensor>` owns a `Logger` derived from `Sensor`, and `Logger` holds a `std::unique_ptr<int[]>`. Trace the full chain that frees the array when the outer pointer goes out of scope. Then say precisely which link breaks if `~Sensor()` is not `virtual`, and why the compiler does not warn you about it.

6. **(Write from scratch)** Build a polymorphic sensor rig: an abstract `Sensor` with `readMilliVolts()` and `name()`, at least three derived types, and a function that takes a `const std::vector<std::unique_ptr<Sensor>>&` and prints a reading table. Then demonstrate, with output, that (a) removing `virtual` from the destructor leaks a derived member, and (b) a function taking `Sensor` by value would slice. Report `sizeof` for one derived type and account for every byte.

7.14 Self-Check

1. How much does adding one virtual function to a `struct { int a; }` change its size?
   - Not at all — dispatch is resolved at compile time
   - From 4 bytes to 8
   - From 4 bytes to 16 — an 8-byte vtable pointer plus alignment padding
   - It doubles for each virtual function added

2. Given a base with a virtual function but a **non-virtual** destructor, what runs on `delete p;` where `p` is a `Base*` pointing at a `Derived`?
   - `~Derived()` then `~Base()`
   - `~Base()` only — `~Derived()` never runs, and anything it owned leaks
   - `~Derived()` only
   - Neither; it fails to compile

3. True or false: passing a `Thermo` to a function taking `Sensor` **by value** still calls `Thermo`'s override, because the object is really a `Thermo`.

4. What makes a class abstract, and what does the compiler tell you when you try to instantiate one?
5. Why should you always write `override`, given that it is optional?
6. What question decides between runtime and compile-time polymorphism?
7. Name the two SOLID principles the sensor hierarchy demonstrates, and point at the line of code that shows each.
8. Why is the vtable cost per object rather than per class, and when does that matter?

7.15 Summary

Close the book and answer from memory.

- What is virtual by default in C++, and how does that differ from C#?
- What does a polymorphic class carry that a plain one does not, and how big is it?
- State the virtual destructor rule and what goes wrong without it.
- What is object slicing, and what does the call site look like when it happens?
- Which two SOLID principles did you apply, and where?

The next chapter takes the other road. Instead of deciding at run time which code to call, you will have the compiler generate a separate version for every type you use — no vtable, no indirection, and error messages of a size you have not yet met.

Part IV

Generic Programming

Chapter 8

Templates and Generic Programming

8.1 What You'll Be Able to Do

- Write function templates and class templates, and map them onto C# generics.
- Explain template instantiation — that a separate concrete function or class is generated per type used — and what that costs in code size.
- Use `std::array<T, N>` in place of a C-style array, and justify it.
- Triage a long template error message rather than fearing it.
- Build a fixed-capacity `RingBuffer<T, N>` and explain why the capacity is a template parameter rather than a constructor argument.

8.2 You Already Have the Idea

```
// C#
T Largest<T>(T a, T b) where T : IComparable<T> { ... }
List<int> numbers = new List<int>();
```

```
// C++
template <typename T>
T largest(T a, T b) { return (a > b) ? a : b; }

std::vector<int> numbers;
```

The syntax differs; the *purpose* is identical. Write the algorithm once, use it with many types. What differs — and it is the whole chapter — is **when**.

	C# generics	C++ templates
Resolved	at **run time**, by the CLR	at **compile time**, by the compiler
In the binary	**one** generic method	**one function per type used**
Constraints	`where T : IComparable<T>`, checked up front	duck-typed: it compiles if the operations exist

	C# generics	C++ templates
Error timing	at the declaration	at **instantiation**, often far from your code
Cost	one implementation, boxed for value types	zero indirection, larger binary

8.3 A Function Template

```
template <typename T>
T clampValue(T v, T lo, T hi) { return (v < lo) ? lo : (v > hi) ? hi : v; }

clampValue(15, 0, 10);                  // T deduced as int
clampValue(2.5, 0.0, 1.0);              // T deduced as double
clampValue<uint8_t>(200, 0, 100);       // T given explicitly
```

```
10
1
d
```

That third line is not a typo. `T` is `uint8_t`, which is `unsigned char`, so `std::cout` prints it as a **character** — and 100 is the character `d`. The clamp worked perfectly; the *printing* is what surprised you. This is Chapter 2's lesson about `uint8_t` arriving in a new place.

Note there is no `where` clause. C++ templates are **duck-typed**: `clampValue` compiles for any `T` supporting `<` and `>`. Give it a type that does not, and the failure appears when the template is *instantiated*, not where it is declared.

8.4 Instantiation Is the Whole Idea

The compiler does not compile a template. It compiles the **functions the template generates**, one per type you actually use.

Here is the proof — the same program, its symbol table read with `nm -C`:

```
T double clampValue<double>(double, double, double)
T unsigned char clampValue<unsigned char>(unsigned char, unsigned char, unsigned char)
T int clampValue<int>(int, int, int)
```

Three real functions, three separate blocks of machine code, from one template you wrote once.

Everything else in this chapter follows from that sentence. If you understand that a template is a *recipe for generating code* rather than code, you can predict the code size, the error messages, and the header rule without being told any of them.

8.5 Code Bloat, Measured

Two programs, identical except that one calls `twice()` with one type and the other with five:

```
template <typename T> T twice(T v) { return v + v; }
```

Measured on the object files:

	instantiations	`__text` size
one type	1	**64 bytes**
five types	5	**304 bytes**

Nearly five times the machine code, from the same one template. On a microcontroller with 64 KB of flash, that ratio is a budget question rather than a curiosity.

This is the trade against the previous chapter. **Virtual dispatch costs RAM per object** and an indirect call, with one copy of the code. **Templates cost flash per type**, with no indirection at all. Neither is free, and which you can afford depends on the device.

8.6 `std::array<T, N>`

A C-style array decays to a pointer and loses its length (Chapter 4). `std::array` is the same storage with the length kept.

```
std::array<int, 4> a{10, 20, 30, 40};
a.size();     // 4      - and it still knows this inside a function
sizeof(a);    // 16     - no overhead: exactly four ints
a[2];         // 30
```

It costs nothing and it does not decay. Prefer it to `int a[4]` everywhere.

Note that `N` is a template parameter, so the size is part of the *type*: an `array<int,4>` and an `array<int,8>` are different types, and neither can silently become the other. That is occasionally inconvenient and always safer.

8.7 Reading Template Errors

Templates produce the worst error messages in the language. The fix is a triage strategy, not courage.

A small one first — `largest` called on a type with no `operator>`:

```
terr.cpp:3:55: error: invalid operands to binary expression ('Reading' and 'Reading')
    3 | template <typename T> T largest(T a, T b) { return (a > b) ? a : b; }
      |                                                     ~ ^ ~
terr.cpp:6:18: note: in instantiation of function template specialization
                     'largest<Reading>' requested here
    6 |     std::cout << largest(r1, r2).mv << "\n";
      |                  ^
1 error generated.
```

Seven lines, and the two you need are marked. The first says *what* is wrong. The `in instantiation of ... requested here` note says **which line of your code asked for it** — and that second line is the one you can actually change.

Now the same mistake inside the standard library — `std::sort` on a type with no `operator<`:

```
308 lines of output
 44 "in instantiation of" notes
  1 of them naming a file you wrote
```

Three hundred and eight lines, and exactly **one** of them is about your code. The volume is not a measure of how badly you have failed; it is a measure of how deep the library call stack is.

The triage strategy, in order:

1. **Read the first error only.** Everything after it is usually fallout.
2. **Search the output for your own filename.** In the 308-line case exactly one line qualifies, and it is the line to look at.
3. **Find the last `in instantiation of ... requested here`** that names your code. That is where you asked for the impossible thing.
4. **Ask what operation the template needed** that your type does not provide. It is nearly always a missing operator, constructor, or `const`.

8.8 Templates Live in Headers

```cpp
// util.h
template <typename T> T largest(T a, T b);   // declaration only

// util.cpp
template <typename T> T largest(T a, T b) { return (a > b) ? a : b; }
```

This compiles. It does not link:

```
ld.lld: error: undefined symbol: int largest<int>(int, int)
>>> referenced by main.cpp
>>>               main.o:(main)
```

The reason follows directly from instantiation: when the compiler builds `main.cpp` it needs to *generate* `largest<int>`, and to generate it, it needs the body. The body is in another translation unit, which the compiler cannot see. Nothing ever generated `largest<int>`, so the linker finds nothing.

Put template definitions in the header. That is not a style rule — it is a consequence of when instantiation happens.

Notice which tool reported it, too. This is a **linker** error, not a compiler error, exactly as in Chapter 1: the message names no line of your source, because there is no line to name. The function was never generated.

8.9 The Class Template You Are About to Be Graded On

```
template <typename T, std::size_t N>
class RingBuffer {
    std::array<T, N> data_{};
    std::size_t head_ = 0, count_ = 0;
public:
    bool push(const T& v);
    bool pop(T& out);
    std::size_t size() const { return count_; }
    static constexpr std::size_t capacity() { return N; }
};
```

Two template parameters, and they are different kinds. `T` is a **type** parameter. `N` is a **non-type** parameter — an actual value, known at compile time, that becomes part of the type.

That second one is the design decision worth understanding. Capacity is a template parameter rather than a constructor argument because:

- The storage is a `std::array<T, N>` **member**, so the whole buffer lives wherever the object lives — on the stack, or in a static, and **never on the heap**. Chapter 3's rule, honoured by construction.
- `sizeof` is known at compile time: `sizeof(RingBuffer<int,3>)` is **32**; `sizeof(RingBuffer<double,64>)` is **528**. You can put those numbers in a memory budget before the code runs.
- `capacity()` is `static constexpr` — a compile-time constant, usable as an array bound or in a `static_assert`.

The cost is the other side of instantiation: `RingBuffer<int,8>` and `RingBuffer<int,16>` are **different types** and generate **separate code**. Being able to state that trade is the point.

8.10 Worked Example

Problem: write `clampValue` as a function template, prove three functions were generated, then convert a C-array API to `std::array`.

```
#include <array>
#include <cstdint>
#include <iostream>

template <typename T>
T clampValue(T v, T lo, T hi) { return (v < lo) ? lo : (v > hi) ? hi : v; }
```

```cpp
// Before: the length has to be passed alongside, and nothing checks it.
int sumRaw(const int* data, std::size_t n) {
    int total = 0;
    for (std::size_t i = 0; i < n; ++i) { total += data[i]; }
    return total;
}

// After: the length is part of the type, so it cannot be wrong.
template <std::size_t N>
int sumArray(const std::array<int, N>& data) {
    int total = 0;
    for (std::size_t i = 0; i < N; ++i) { total += data[i]; }
    return total;
}

int main() {
    std::cout << clampValue(15, 0, 10) << "\n";
    std::cout << clampValue(2.5, 0.0, 1.0) << "\n";
    std::cout << int(clampValue<std::uint8_t>(200, 0, 120)) << "\n";

    std::array<int, 4> readings{10, 20, 30, 40};
    std::cout << sumArray(readings) << "\n";
}
```

```
10
1
120
100
```

Two things to notice. The `int(...)` cast on the third line is what makes `120` print as a number rather than the character `x`. And `sumArray` never receives a length — `N` is deduced from the argument's type, so passing the wrong count is not a mistake you can make.

8.11 Predict Before You Run

Predict before you run. How many `twice` functions exist in the final binary?

```cpp
template <typename T> T twice(T v) { return v + v; }

int main() {
    std::cout << twice(3) << "\n";
    std::cout << twice(4) << "\n";
    std::cout << twice(2.5) << "\n";
    std::cout << twice<int>(7) << "\n";
}
```

Two. `twice<int>` and `twice<double>`.

Four call sites, but instantiation is per *type*, not per call. The first, second, and fourth all ask for `twice<int>` and share one generated function; only `twice(2.5)` adds a second.

Predict before you run. Does this compile? If not, what is the error?

```cpp
template <typename T> T largest(T a, T b) { return (a > b) ? a : b; }

int main() { std::cout << largest(3, 9.5) << "\n"; }
```

It does not compile:

```
error: no matching function for call to 'largest'
note: candidate template ignored: deduced conflicting types for parameter 'T'
      ('int' vs. 'double')
```

There is **one** `T` and two arguments of different types, so deduction cannot pick a winner. It is not that `int` and `double` are incomparable — they compare fine. The template as written demands they be the *same type*. Fix it by saying which you want, `largest<double>(3, 9.5)`, or by giving the template two parameters.

8.12 Common Errors

Symptom	Cause	Fix
`undefined symbol: int f<int>(int)` at link time	Template definition in a `.cpp`	Move the definition into the header
`deduced conflicting types for parameter 'T'`	One `T`, arguments of different types	Specify `f<double>(...)`, or use two parameters
Hundreds of lines of error from one mistake	A library template instantiated with an unsuitable type	Triage: first error, your filename, last `in instantiation of`
`invalid operands to binary expression` inside a template	Your type lacks an operator the template needs	Provide the operator, or use a different algorithm
Binary much larger than expected	Many instantiations	Count them with `nm -C`; reduce the type set
`array<int,4>` will not convert to `array<int,8>`	`N` is part of the type	Intended — take a template parameter `N` instead
A `uint8_t` prints as a letter	It is `unsigned char`	Cast to `int` for printing

8.13 Key Terms

template — a pattern from which the compiler generates concrete functions or classes.

instantiation — the generation of a concrete function or class from a template for a particular set of arguments.

function template — a template that generates functions.

class template — a template that generates classes.

type parameter — a template parameter standing for a type, written `typename T`.

non-type parameter — a template parameter that is a compile-time *value*, such as `std::size_t N`, which becomes part of the type.

duck typing — the C++ template rule that any type works if it supports the operations used; there is no declared constraint.

code bloat — the growth in binary size caused by generating a separate copy per instantiated type.

`std::array<T, N>` — a fixed-size array that keeps its length, does not decay to a pointer, and costs no more than a C array.

8.14 Exercises

1. **(Trace)** Write the output before compiling.

```
template <typename T> void show(T v) { std::cout << "generic: " << v << "\n"; }

int main() {
    show(42);
    show(3.5);
    show('x');
    show("hello");
}
```

2. **(Trace)** How many distinct functions does the compiler generate here, and what are they?

```
template <typename T> T addUp(T a, T b) { return a + b; }

int main() {
    addUp(1, 2);
    addUp(3, 4);
    addUp(1.5, 2.5);
    addUp<double>(1, 2);
    addUp<long>(1, 2);
}
```

3. **(Trace)** Which of these compile? For each failure, name the error before you try it.

```cpp
template <typename T> T largest(T a, T b) { return (a > b) ? a : b; }
struct Reading { int mv; };

largest(1, 2);                  // (i)
largest(1, 2.0);                // (ii)
largest<double>(1, 2.0);        // (iii)
largest(Reading{1}, Reading{2}); // (iv)
```

4. **(Faded)** Complete the fixed-capacity stack. Storage must be a member, capacity must be a compile-time constant, and nothing may touch the heap.

```cpp
template <typename T, ____________>          // (A) the non-type parameter
class FixedStack {
    ____________ data_{};                    // (B) the storage member
    std::size_t count_ = 0;
public:
    bool push(const T& v) {
        if (count_ == ____) { return false; }  // (C) the full test
        data_[count_++] = v;
        return true;
    }
    bool pop(T& out) {
        if (count_ == 0) { return false; }
        out = data_[--count_];
        return true;
    }
    std::size_t size() const { return count_; }
    ____________________ capacity() { return N; }   // (D) compile-time constant
};
```

5. **(Spaced recall — Chapters 4 and 7)** A function takes `const std::array<int, 8>&`; another takes `const int*` plus a length. Say what the first one knows that the second cannot, and connect it to Chapter 4's rule about array decay. Then answer the Chapter 7 half: you need one function that works for several sensor types. When should it be a template, and when should it take a `const Sensor&`?

6. **(Write from scratch)** Build `RingBuffer<T, N>` — a fixed-capacity circular buffer with `push`, `pop`, `size`, and a `static constexpr capacity()`. Storage must be a `std::array<T, N>` member; no heap allocation anywhere. `push` returns `false` when full without overwriting; `pop` returns `false` when empty without touching its argument. Demonstrate it with two different `T` and two different `N`, print `sizeof` for each, and explain why the four combinations are four distinct types.

8.15 Self-Check

1. C# generics and C++ templates do the same job. What is the one real difference?
2. What is template instantiation?

3. How would you check how many instantiations your program generated?
4. Select every true statement about the cost of templates.
    - A separate copy of the code is generated per type used
    - There is no run-time indirection, unlike a virtual call
    - The binary can grow substantially with many instantiated types
    - Each instantiation adds a pointer to every object
5. Templates and virtual dispatch both let one piece of code serve many types. Which question decides between them?
6. Why does `largest(3, 9.5)` fail to compile, given `template <typename T> T largest(T a, T b)`?
7. A `std::sort` on a type with no `operator<` produced **308 lines** of error output. What does that volume tell you?
8. Which line of a long template error actually points at *your* mistake?
9. Why must template definitions live in the header, and which build stage reports it when they do not?
10. Select every true statement about `std::array<int, 4>`.
    - It knows its own length inside a function
    - It is larger than `int[4]` because it stores the size
    - Its size is part of its type
    - It decays to a pointer when passed to a function

8.16 Summary

Close the book and answer from memory.

- What does the compiler actually compile when it meets a template?
- State the cost of templates and the cost of virtual dispatch, and say which resource each spends.
- Give the four steps for triaging a long template error.
- Why must template definitions be in the header, and which tool complains?
- Why is `RingBuffer`'s capacity a template parameter rather than a constructor argument? Give the memory reason and the budgeting reason.

The next chapter puts templates to work without writing any: the standard library's containers, iterators, and algorithms are all templates, and knowing which container to reach for is mostly a question about how you will use it.

Chapter 9

The STL: Containers, Iterators, and Algorithms

9.1 What You'll Be Able to Do

- Use `std::vector`, `std::array`, `std::map`, and `std::unordered_map`, and choose between them.
- Explain iterators and iterator categories, and traverse with them and with range-based `for`.
- Apply `<algorithm>` — `sort`, `find_if`, `accumulate`, `transform`, `for_each` — instead of hand-written loops.
- Use `auto`, lambdas, and structured bindings.
- Say what choosing an algorithm over a manual loop costs and buys.

Everything in this chapter is a template. You do not have to write one to use them, but the error messages from the previous chapter are the error messages you will get here.

9.2 Four Containers, and How to Choose

Container	Storage	Lookup	Use it when
`std::array<T,N>`	**stack**, fixed at compile time	by index, O(1)	the size is known when you build
`std::vector<T>`	**heap**, grows	by index, O(1)	the size is not known until run time
`std::map<K,V>`	heap, ordered tree	by key, O(log n)	you need keys **in sorted order**
`std::unordered_map<K,V>`	heap, hash table	by key, O(1) average	you need keys and do not care about order

```
sizeof(vector<int>)   = 24     // three pointers: begin, end, capacity-end
sizeof(array<int,4>)  = 16     // exactly four ints. nothing else.
```

That contrast is the chapter in one line. **A `vector` is a small object on the stack pointing at heap storage; an `array` *is* the storage.** A `vector` holding a million elements still has `sizeof` 24.

9.3 The Tension, Named Out Loud

This course has spent six chapters telling you to avoid the heap. `std::vector` is a heap container. Both of those remain true, and the resolution is a rule rather than a contradiction:

> **On the workstation, reach for `vector` first. On the device, reach for `array` first.** If the size is known at compile time, `array` is strictly better — no allocation, no indirection, and the size is part of the type.

`std::vector` is not banned; it is *charged for*. It allocates, and it can reallocate at a moment you did not choose. On a microcontroller those are Chapter 3's objections, unchanged.

9.4 Iterators

An iterator is a generalised pointer: something you can dereference with `*`, advance with `++`, and compare with `!=`. That is the whole interface, and it is why one `sort` works on a `vector` and an `array` alike.

```
for (auto it = v.begin(); it != v.end(); ++it) { std::cout << *it; }
for (const auto& x : v) { std::cout << x; }        // the same loop, written for humans
```

`v.end()` points **one past** the last element. It is not an element and must never be dereferenced — which is why the loop condition is `!= end()` and never `<= end()`.

The categories appear in every error message you will read, so learn the names: **forward** (go forward one at a time), **bidirectional** (also backwards — `map`), **random access** (jump by n — `vector`, `array`). `std::sort` needs random access, which is exactly why you cannot sort a `std::map`.

9.5 Iterator Invalidation

```
std::vector<int> v{1,2,3};
// ... hold an iterator or a pointer to v[0] ...
v.push_back(4);   // may reallocate - and if it does, your iterator is dangling
```

```
before: data=0x100bb94e0 cap=3
after : data=0x100bb95c0 cap=24
```

The buffer **moved**. Every iterator, pointer, and reference into the old buffer now points at freed memory — Chapter 3's dangling pointer, arriving through a container that was supposed to be safe.

Note the capacity too: it went from 3 to 24, not to 4. A `vector` grows in jumps to keep the amortised cost low, which means most `push_back`s do not reallocate and the one that does catches you by surprise.

9.6 `at()` versus `[]`

`v[i]` does not check. `v.at(i)` throws `std::out_of_range`.

Use `at()` while you are learning, and in any code where the index came from outside your program. Use `[]` in a loop you have already bounded. The difference is one bounds check, and "which one is faster" is the wrong question until you have measured something.

9.7 Algorithms Instead of Loops

```
#include <algorithm>
#include <numeric>

std::sort(v.begin(), v.end());
auto it = std::find_if(v.begin(), v.end(), [](int x) { return x > 1000; });
int sum  = std::accumulate(v.begin(), v.end(), 0);
std::transform(v.begin(), v.end(), out.begin(), [](int mv) { return mv / 1000; });
```

On `{1250, 3300, 800, 3300}`:

```
sorted: 800 1250 3300 3300
first>1000 = 1250 at index 1
accumulate = 8650
transform: 0 1 3 3
```

The argument for these is not speed — often they compile to the same code as your loop. It is that the name says what is happening. A reader of `std::accumulate` knows the result at a glance; a reader of a hand-written `for` loop has to check whether you got the bounds right, because sometimes people do not.

Note `accumulate` lives in `<numeric>`, not `<algorithm>`. That catches everyone once.

9.8 Lambdas, Briefly

```
[](int x) { return x > 1000; }                  // takes an int, returns bool
[threshold](int x) { return x > threshold; }    // captures a local by value
[&count](int x) { if (x > 0) ++count; }         // captures by reference
```

Read `[ ]` as *"what from the surrounding scope does this need?"*

Capture by value copies; capture by reference does not. A by-value capture is `const` inside the lambda by default, so `[count](int x){ ++count; }` will not compile — you captured a copy and then tried to modify it. And a reference capture that outlives the thing it captured is Chapter 4's dangling reference in new clothing.

9.9 auto and Structured Bindings

```
for (const auto& [name, reading] : sensorMap) {
    std::cout << name << ": " << reading << "\n";
}
```

Structured bindings unpack a pair or struct into named variables. That is the difference between `it->second` and `reading`, and it is worth the keystrokes every time.

9.10 Ordered or Not: The Choice with a Visible Consequence

```
std::map<std::string,int> ordered{{"thermo",1250},{"pot",3300},{"hall",800}};
for (const auto& [name, val] : ordered) { std::cout << " " << name; }
```

```
map order: hall pot thermo
```

Alphabetical — not insertion order. `std::unordered_map` gives you no order at all, and O(1) average lookup instead of O(log n).

Choose on **whether you need the order**, not on which sounds faster.

9.11 Worked Example

Problem: take a list of sensor readings, find the first over threshold, total them, convert to volts, and report per-band counts.

```
#include <algorithm>
#include <iostream>
#include <map>
#include <numeric>
#include <string>
#include <vector>

int main() {
    std::vector<int> mv{1250, 3300, 800, 3300};

    std::sort(mv.begin(), mv.end());
    auto over = std::find_if(mv.begin(), mv.end(), [](int v) { return v > 1000; });
    int total = std::accumulate(mv.begin(), mv.end(), 0);

    std::vector<int> volts(mv.size());
    std::transform(mv.begin(), mv.end(), volts.begin(), [](int m) { return m / 1000; });

    std::map<std::string,int> counts;
    for (int v : mv) { counts[v > 1000 ? "high" : "low"]++; }

    std::cout << "first>1000 = " << *over << " at index " << (over - mv.begin()) << "\n";
    std::cout << "total = " << total << "\n";
```

```
    for (const auto& [band, n] : counts) { std::cout << band << ": " << n << "\n"; }
}
```

```
first>1000 = 1250 at index 1
total = 8650
high: 3
low: 1
```

Note `counts[...]++` on a `std::map`. Subscripting a missing key **default-constructs** it — to 0 for an `int` — and returns a reference to it. That is convenient here and a trap elsewhere: *reading* `m[k]` for a key that is absent silently inserts it. If you only want to look, use `m.find(k)` or `m.contains(k)`.

9.12 Predict Before You Run

Predict before you run. What prints, and is the pointer still good?

```
std::vector<int> v{1, 2, 3};
int* p = &v[0];
std::cout << *p << "\n";
v.push_back(4);
std::cout << *p << "\n";      // ?
```

The first line prints `1`. The second is **undefined behaviour**. On this machine it prints:

```
1
2
```

Not a crash, not obvious garbage — `2`, a number that looks entirely plausible coming from this container. `push_back` exceeded the capacity, so the vector allocated a new buffer, copied the elements, and freed the old one; the allocator then reused that freed block, and `p` now reads whatever landed there.

That is the worst possible outcome, and it is the reason this bug survives testing. Your program does not stop, and the number it prints is not obviously wrong. Run it on a different machine or a different day and you may get `1`, or garbage, or a crash — the standard promises you nothing.

Predict before you run. How many entries does the map hold at the end?

```
std::map<std::string,int> m{{"a", 1}};
if (m["b"] == 0) { std::cout << "b is missing\n"; }
std::cout << m.size() << "\n";
```

```
b is missing
2
```

Two. The `if` looked like a read, but `m["b"]` default-constructed an entry for `"b"` and returned a reference to it. The message is right and the map is now wrong.

9.13 Common Errors

Symptom	Cause	Fix
Value read after `push_back` is garbage	Iterator/pointer invalidated by reallocation	Re-acquire it after any modification; do not cache `&v[0]`
Crash at the last element	Dereferencing `v.end()`	`end()` is one past the last; loop with `!= end()`
`std::sort` on a `map` will not compile	`map` has bidirectional, not random-access, iterators	Copy to a `vector`, or use the map's own order
Map grew unexpectedly	`m[key]` on a missing key inserts it	Use `find` or `contains` to look without inserting
`++count` in a lambda will not compile	Captured by value, which is `const` inside	Capture by reference `[&count]`, if the lifetime is safe
`accumulate` not declared	It is in `<numeric>`, not `<algorithm>`	Add `#include <numeric>`
No bounds error, just wrong data	`[]` does not check	Use `at()` where the index came from outside
Insertion order not preserved	`std::map` is sorted by key	Use a `vector` of pairs if order of arrival matters

9.14 Key Terms

container — a class template holding a collection of objects.

iterator — a generalised pointer supporting `*`, `++`, and `!=`.

`end()` — an iterator one past the last element; never dereference it.

iterator invalidation — the loss of validity of iterators, pointers, and references when a container reallocates or erases.

random-access iterator — an iterator that can jump by n; required by `std::sort`.

lambda — an unnamed function object written inline, with a capture list saying what it takes from the enclosing scope.

capture by value / by reference — `[x]` copies; `[&x]` refers, and dangles if the referent dies first.

structured binding — `auto [a, b] = pair;`, unpacking into named variables.

`std::accumulate` — totals a range; lives in `<numeric>`.

9.15 Exercises

1. **(Trace)** Write the output before compiling.

```cpp
std::vector<int> v{5, 3, 9, 1};
std::sort(v.begin(), v.end());
for (int x : v) { std::cout << x << " "; }
std::cout << "\n" << std::accumulate(v.begin(), v.end(), 0) << "\n";
std::cout << *std::find_if(v.begin(), v.end(), [](int x){ return x > 4; }) << "\n";
```

2. **(Trace)** Predict the iteration order and the final size.

```cpp
std::map<std::string,int> m;
m["zulu"] = 1;
m["alpha"] = 2;
m["mike"] = 3;
for (const auto& [k, val] : m) { std::cout << k << " "; }
std::cout << "\n";
std::cout << m["november"] << "\n";
std::cout << m.size() << "\n";
```

3. **(Trace)** What happened to the pointer? Say what prints and why it is not trustworthy.

```cpp
std::vector<int> v;
v.reserve(2);
v.push_back(10);
int* p = &v[0];
v.push_back(20);
std::cout << *p << "\n";      // (A)
v.push_back(30);
std::cout << *p << "\n";      // (B)
```

4. **(Faded)** Complete the summary. It must total the readings, count those above 1000, and print the highest — using algorithms, not hand-written loops.

```cpp
std::vector<int> mv{1250, 3300, 800, 3300, 450};

int total = std::__________(mv.begin(), mv.end(), 0);          // (A)

auto high = std::__________(mv.begin(), mv.end(),               // (B) count matching
                            [](int x){ return x > 1000; });

auto biggest = std::__________(mv.begin(), mv.end());           // (C) largest element

std::cout << total << " " << high << " " << ____ << "\n";       // (D) print the value
```

5. **(Spaced recall — Chapters 6 and 8)** `std::vector<std::unique_ptr<Sensor>> sensors;` is populated and then you write `auto copy = sensors;`. It does not compile. Explain why, in terms of Chapter 6, and say what `auto copy = std::move(sensors);` would do instead. Then the Chapter 8 half: `sizeof(std::vector<int>)` is 24 and `sizeof(std::array<int,4>)` is 16 — account for both numbers.

6. **(Write from scratch)** Given a `std::vector<int>` of millivolt readings, produce a summary report using **no hand-written loops over the data**: the sorted readings, the total, the

mean, the first reading above a threshold (and its index), and a per-band count in a `std::map` printed in band order. Then answer in a comment: which container would you use for the readings on a microcontroller, and why?

9.16 Self-Check

1. `sizeof(std::vector<int>)` is 24 and does not change when you push more elements. Why?
2. `sizeof(std::array<int,4>)` is 16. What does that tell you?
3. This course spent six weeks arguing against the heap, and `std::vector` is a heap container. Which rule resolves that?
4. What is `v.end()`?
5. Select every true statement about iterator invalidation.
   - `push_back` may reallocate and invalidate every iterator into the vector
   - A pointer to an element is invalidated by the same reallocation
   - `std::array` iterators can be invalidated by adding an element
   - After invalidation, dereferencing usually still appears to work
6. True or false: `v[i]` and `v.at(i)` behave identically when `i` is out of range.
7. Which algorithm totals a range, and which header is it in?
8. What is the real argument for `<algorithm>` over a hand-written loop?
9. Why does `[count](int x){ ++count; }` fail to compile?
10. `std::map<std::string,int> m;` is empty. You write `std::cout << m["missing"];`. Select every true statement.
    - It prints `0`
    - It inserts an entry for `"missing"`
    - `m.size()` is afterwards 1
    - It throws `std::out_of_range`

9.17 Summary

Close the book and answer from memory.

- Give the choosing rule for `array`, `vector`, `map`, and `unordered_map`.
- Why is `sizeof(vector<int>)` the same for one element and a million?
- What is iterator invalidation, and which earlier chapter's bug is it?
- Why can you not `std::sort` a `std::map`?
- What is the argument for algorithms over loops, and what is it *not*?
- What does `m[key]` do when `key` is absent?

The next chapter is about what happens when things go wrong on purpose: the standard exception hierarchy, writing your own exception types, and the tests that prove your code fails the way you intended.

Part V

Robustness

Chapter 10

Error Handling and Unit Testing

10.1 What You'll Be Able to Do

- Use `try`/`catch`/`throw` and the standard exception hierarchy.
- Write custom exception classes deriving from `std::exception`.
- Explain exception safety, how RAII guarantees cleanup on the exceptional path, and where `noexcept` belongs.
- Write unit tests with Catch2 wired into CMake, including tests that assert the correct exception is thrown.
- Distinguish a unit test from an integration test.

10.2 The Mechanism

```
try {
    read(7);
} catch (const SensorFault& e) {
    std::cout << "caught: " << e.what() << "\n";
}
```

`throw` abandons the current function immediately and unwinds the stack until a matching `catch` is found. Nothing after the `throw` in that function runs.

You know this shape from C#. Three things differ enough to matter:

1. **You can throw anything** — an `int`, a `std::string`, a class. You should throw a type deriving from `std::exception`, and nothing else.
2. **Catch by `const` reference**, always. Catching by value slices.
3. **There is no `finally`.** C++ does not need one, and the reason is the whole of Chapter 5.

10.3 The Standard Hierarchy

```
std::exception
 |
 +-- std::logic_error        a bug: the caller broke a precondition
 |    |
 |    +-- std::invalid_argument
 |    +-- std::out_of_range        <- v.at(999)
 |
 +-- std::runtime_error      the world misbehaving: a file, a device, a network
      |
      +-- (yours goes here)
```

`what()` returns a `const char*` describing the problem. That is the only member `std::exception` guarantees.

The `logic_error` / `runtime_error` split is worth taking seriously: **a logic error is something you should fix in code; a runtime error is something you should handle.** An out-of-range index is a bug. A missing sensor is Tuesday.

10.4 A Custom Exception

```cpp
class SensorFault : public std::runtime_error {
    int id_;
public:
    SensorFault(int id, const std::string& what)
        : std::runtime_error("sensor " + std::to_string(id) + ": " + what), id_(id) {}
    int id() const noexcept { return id_; }
};
```

Derive from `std::runtime_error`, pass the message up to it, and add whatever the handler needs in order to *act* — here, which sensor failed. A handler can then catch `SensorFault` specifically and read `id()`, or catch `std::exception` and just log `what()`.

That is the test of a good custom exception: does it carry something the handler can make a decision with? A type that adds nothing but a name could have been a `runtime_error` with a better message.

10.5 Exception Safety, and Why There Is No `finally`

```cpp
struct Guard {
    const char* n;
    explicit Guard(const char* s) : n(s) { std::cout << "  acquire " << n << "\n"; }
    ~Guard() { std::cout << "  release " << n << "\n"; }
};

void read(int id) {
    Guard g{"bus"};
    if (id == 7) { throw SensorFault(id, "no response"); }
    std::cout << "  read ok\n";
}
```

> **Predict before you run.** On the throwing path, does `release bus` print at all — and if so, before or after the catch handler?

```
-- normal path --
  acquire bus
  read ok
  release bus
-- throwing path --
  acquire bus
  release bus
caught SensorFault, id=7: sensor 7: no response
```

Read the second block carefully. **`release bus` prints before the catch handler runs.** Unwinding the stack destroys every fully-constructed local on the way out, in reverse order, exactly as a normal return would.

That is why C++ has no `finally`: **the destructor already is one.** RAII was never only about the scope exits you can see — it covers the exit you did not write, which is the one that actually loses resources in production.

Chapter 5 argued for RAII using an early `return`. This is the same argument with a much larger example: there is no `throw`-shaped hole in your cleanup, because you never wrote the cleanup as a statement in the first place.

10.6 Catch by Reference — and What Slicing Costs You

```cpp
try { throw Derived{}; } catch (Base e)        { e.kind(); }   // "Base"
try { throw Derived{}; } catch (const Base& e) { e.kind(); }   // "Derived"
```

```
catch by value:     Base
catch by reference: Derived
```

Catching by value copies the base part and discards the rest — Chapter 7's slicing, in a new place where it silently destroys the very information you threw the exception to convey. You catch a `SensorFault` by value and the `id()` goes with it.

Always `catch (const T&)`.

Order matters too: handlers are tried in order, so the most derived type must come first. Put `catch (const std::exception&)` above `catch (const SensorFault&)` and the specific handler is unreachable — clang will warn you.

10.7 `noexcept`

`noexcept` promises a function will not throw. Break the promise and the program calls `std::terminate` — it does not propagate.

Use it where it is genuinely true and useful: destructors, swap functions, and simple accessors.

Destructors are `noexcept` by default and **must not throw**. A destructor that throws while the stack is already unwinding from another exception terminates the program, because there is no sensible answer to "which exception wins".

10.8 The Embedded Caveat, Stated Honestly

Many embedded toolchains build with `-fno-exceptions`. Exceptions cost binary size — the unwinding tables — and their timing is not bounded, which hard-real-time code cannot accept.

Where exceptions are off, the answer is a **returned error value**: an enum or a status code the caller must check. That is not a lesser technique; it is the right technique in that context.

Being able to say **which mechanism applies where** is the actual competency. Exceptions for a file that might be missing; a returned error code for a byte-level parser in a build with no exception support.

10.9 Why Test at All

You have been testing all term by running `main` and reading output. That works until you have twenty behaviours, at which point checking them by eye after every change stops happening — not because you are lazy, but because it costs more than the change did.

A unit test is that check, written once and run in a second.

Unit versus integration: a *unit* test exercises one class or function in isolation — `RingBuffer::push` refuses when full. An *integration* test exercises pieces together — the parser feeds the queue and the consumer sees every record. Both are useful; unit tests are where you start, because when one fails you already know where the bug is.

10.10 Catch2, Wired into CMake

```
cmake_minimum_required(VERSION 4.4)
project(RingTests CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED True)

include(FetchContent)
FetchContent_Declare(Catch2
  GIT_REPOSITORY https://github.com/catchorg/Catch2.git
  GIT_TAG        v3.5.2)
FetchContent_MakeAvailable(Catch2)

add_executable(tests test_ring.cpp)
target_link_libraries(tests PRIVATE Catch2::Catch2WithMain)
```

The first build downloads and compiles Catch2, which takes a few minutes. That is normal, not a hang. Note the pinned tag: `v3.5.2`, not “latest”. A test suite that changes behaviour because a dependency moved underneath it is worse than no test suite.

10.11 Writing a Test

```
#include <catch2/catch_test_macros.hpp>
#include "ring.h"

TEST_CASE("push fills to capacity and then refuses", "[ring]") {
    RingBuffer<int, 3> rb;
    REQUIRE(rb.push(1));
    REQUIRE(rb.push(2));
    REQUIRE(rb.push(3));
    REQUIRE_FALSE(rb.push(4));
    REQUIRE(rb.size() == 3);
}

TEST_CASE("at() throws out_of_range past the end", "[ring][exceptions]") {
    RingBuffer<int, 3> rb;
    rb.push(10);
    REQUIRE(rb.at(0) == 10);
    REQUIRE_THROWS_AS(rb.at(1), std::out_of_range);
}
```

Four macros cover this course: `TEST_CASE`, `REQUIRE`, `REQUIRE_FALSE`, and `REQUIRE_THROWS_AS`.

```
All tests passed (21 assertions in 5 test cases)
```

Notice what the first test asserts beyond the return value: `rb.size() == 3` **after** the refused push. A `push` that correctly returns `false` and also corrupts the count would pass the weaker test. Assert the return value *and* the state.

10.12 Read a Failure

Change one expected value and rerun:

```
-------------------------------------------------------------------------------
the buffer wraps invisibly
-------------------------------------------------------------------------------
test_ring.cpp:27
...............................................................................

test_ring.cpp:33: FAILED:
  REQUIRE( v == 99 )
with expansion:
  10 == 99
```

Read the `with expansion` line. Catch2 re-runs the expression and prints the **actual values**, so you learn that `v` was 10 without adding a print statement. The test case name and both line numbers — where the case starts and where it failed — are given for free.

A failing test is information, in exactly the sense a compiler error is information.

10.13 Worked Example

Problem: give a lookup function a checked interface, prove the resource is released on the throwing path, and test both.

```
#include <iostream>
#include <stdexcept>
#include <string>

class SensorFault : public std::runtime_error {
    int id_;
public:
    SensorFault(int id, const std::string& what)
        : std::runtime_error("sensor " + std::to_string(id) + ": " + what), id_(id) {}
    int id() const noexcept { return id_; }
};

struct BusGuard {
    explicit BusGuard() { std::cout << "  bus acquired\n"; }
    ~BusGuard() { std::cout << "  bus released\n"; }
};

int readSensor(int id) {
    BusGuard g;
    if (id < 0 || id > 3) { throw std::out_of_range("sensor id " + std::to_string(id)); }
    if (id == 2)          { throw SensorFault(id, "no response"); }
    return 1000 + id * 100;
}

int main() {
    for (int id : {1, 2, 9}) {
```

```
        std::cout << "reading " << id << ":\n";
        try {
            int v = readSensor(id);            // call first, then print
            std::cout << "  value " << v << "\n";
        } catch (const SensorFault& e) {
            std::cout << "  fault on " << e.id() << ": " << e.what() << "\n";
        } catch (const std::exception& e) {
            std::cout << "  error: " << e.what() << "\n";
        }
    }
}
```

```
reading 1:
  bus acquired
  bus released
  value 1100
reading 2:
  bus acquired
  bus released
  fault on 2: sensor 2: no response
reading 9:
  bus acquired
  bus released
  error: sensor id 9
```

The bus is released on all three paths, and nobody wrote a release statement. Note also that `bus released` appears before `value 1100` on the normal path — the guard is destroyed when `readSensor` returns, which is before `main` prints the result it returned.

The `int v = readSensor(id);` line is deliberate. Writing `std::cout << "  value " << readSensor(id)` instead would insert `"  value "` into the stream *before* calling the function, so the guard's own output lands in the middle of your sentence. Worth knowing the first time a stream expression prints in an order you did not expect.

The two handlers are ordered most-derived first. Swap them and the `SensorFault` handler becomes unreachable, losing `id()`.

10.14 Common Errors

Symptom	Cause	Fix
Handler runs but the derived data is gone	Caught by value — sliced	`catch (const T&)`
Specific handler never runs	A base-class handler is listed first	Order most-derived first
Program terminates instead of unwinding	A `noexcept` function threw, or a destructor threw	Remove the false promise; never throw from a destructor
Resource leaked when an exception passed through	Cleanup written as a statement, not a destructor	RAII — there is no `finally` to add

Symptom	Cause	Fix
`terminate called after throwing` with no handler	No matching `catch` anywhere up the stack	Add one, or fix the throw
Catch2 build appears to hang	`FetchContent` is downloading and building it	Expected on first build; needs network
Test passes but the bug is real	Only the return value was asserted, not the state	Assert both
`REQUIRE_THROWS_AS` fails on the right exception	The type does not match exactly	Check the hierarchy; `out_of_range` is not `runtime_error`

10.15 Key Terms

`throw` — abandons the current function and begins unwinding the stack.

stack unwinding — the destruction of every fully-constructed local, in reverse order, as an exception propagates.

`std::exception` — the base of the standard hierarchy; guarantees only `what()`.

`std::logic_error` — an error representing a bug: a precondition the caller broke.

`std::runtime_error` — an error representing the world misbehaving.

exception safety — the property that a function leaves the program in a valid state even when an exception passes through it.

`noexcept` — a promise that a function will not throw; breaking it calls `std::terminate`.

unit test — a test exercising one class or function in isolation.

integration test — a test exercising several pieces working together.

`REQUIRE_THROWS_AS` — the Catch2 assertion that a given expression throws a given exception type.

10.16 Exercises

1. **(Trace)** Write the complete output before compiling.

```
struct Guard {
    const char* n;
    explicit Guard(const char* s) : n(s) { std::cout << "acquire " << n << "\n"; }
    ~Guard() { std::cout << "release " << n << "\n"; }
};

void inner() {
    Guard b{"inner"};
    throw std::runtime_error("boom");
}

int main() {
```

```
    Guard a{"outer"};
    try {
        inner();
        std::cout << "not reached\n";
    } catch (const std::exception& e) {
        std::cout << "caught " << e.what() << "\n";
    }
    std::cout << "done\n";
}
```

2. **(Trace)** Which handler runs in each case, and what prints?

```
struct AppError : std::runtime_error { using std::runtime_error::runtime_error; };

void demo(int which) {
    try {
        if (which == 0) { throw AppError("app"); }
        if (which == 1) { throw std::out_of_range("range"); }
        throw 42;
    } catch (const AppError& e)       { std::cout << "A: " << e.what() << "\n"; }
      catch (const std::exception& e) { std::cout << "B: " << e.what() << "\n"; }
      catch (...)                     { std::cout << "C: something else\n"; }
}
```

3. **(Trace)** Which exception should each of these throw, and why? Give the standard type, or say that an exception is the wrong mechanism.

   - A caller passes index 9 to a four-element buffer.
   - A configuration file named in a command-line argument does not exist.
   - A byte-level packet parser, in a build compiled with `-fno-exceptions`, meets a bad checksum.
   - A function is asked to take the square root of a negative number.

4. **(Faded)** Complete the custom exception and its handler.

```cpp
class CalibrationError : public ____________________ {   // (A) which base
    int channel_;
public:
    CalibrationError(int ch, const std::string& msg)
        : ____________________, channel_(ch) {}          // (B) pass the message up
    int channel() const ________ { return channel_; }    // (C) the promise
};

// ...
try {
    calibrate(3);
} catch (____________________ e) {                        // (D) how to catch
    std::cout << "channel " << e.channel() << ": " << e.what() << "\n";
}
```

5. **(Write the tests)** Given a `RingBuffer<T, N>` with `push`, `pop`, `at`, `size`, and `capacity`, write Catch2 test cases covering: a new buffer is empty; `push` fills then refuses without changing `size()`; `pop` returns elements oldest-first; `pop` on empty returns `false` and leaves its argument untouched; the buffer wraps correctly; and `at()` throws `std::out_of_range` past the end. For each, write one line saying **which bug it would catch**.

6. **(Write from scratch)** Take your `RingBuffer` from Chapter 8, add a checked `at()` that throws `std::out_of_range`, and write a Catch2 suite that proves it. Then **deliberately break** `push` so it writes at `head_` instead of `(head_ + count_) % N`, and report which of your tests fail. If none do, your suite is missing the wrap test — add it and say what it asserts.

10.17 Self-Check

1. What does `throw` do to the current function?
2. Which branch of the standard hierarchy does `std::out_of_range` belong to, and what does that classification mean?
3. Why does C++ have no `finally`?
4. Why must you catch by `const` reference rather than by value?
5. You write two handlers, one for `SensorFault` and one for `std::exception`. Which must come first?
6. Select every true statement about `noexcept` and destructors.
   - A `noexcept` function that throws calls `std::terminate`
   - Destructors are `noexcept` by default
   - A destructor that throws during unwinding terminates the program
   - `noexcept` makes a function faster in all cases
7. Why do many embedded builds use `-fno-exceptions`, and what replaces exceptions there?
8. What is the difference between a unit test and an integration test?

9. How do you assert that a call throws a particular exception type?
10. A Catch2 failure prints a `with expansion:` line. What is it for?

10.18 Summary

Close the book and answer from memory.

- What happens to locals when an exception passes through their scope?
- State the `logic_error` / `runtime_error` distinction and give an example of each.
- Why is there no `finally`, and which chapter's idea replaces it?
- What does catching by value cost you?
- When is a returned error code the right mechanism rather than an exception?
- What should a test assert besides the return value?

The next chapter takes error handling somewhere it is unavoidable: files. Streams fail, formats are strict, and binary data does not forgive a misread byte.

Chapter 11

File I/O and Streams

11.1 What You'll Be Able to Do

- Read and write text with `ifstream`/`ofstream`, and handle open failures.
- Read and write binary data field by field, with explicit byte order, and explain why the on-disk layout must not depend on compiler padding.
- Convert between host order and a fixed on-disk order by shifting and masking, and use `std::bit_cast` for a floating-point field.
- Contrast that discipline with the `reinterpret_cast`-onto-a-packed-struct idiom, and say exactly what breaks it.
- Explain stream buffering and the role of `std::streambuf`.
- Implement a user-defined `operator<<`.

11.2 Text Streams, and the Failure Everyone Skips

```
#include <fstream>

std::ofstream out("readings.txt");
out << "thermo " << 1250 << "\n";

std::ifstream in("readings.txt");
std::string name; int mv;
while (in >> name >> mv) { std::cout << name << " = " << mv << "\n"; }
```

`ifstream` and `ofstream` are RAII classes: the constructor opens, the destructor closes. You will not see a `close()` call in this course, and you should not write one.

Opening can fail, and it fails silently:

```
missing file is_open=0 good=0
```

No exception, no crash — the stream simply enters a failed state and every subsequent read does nothing. Your loop runs zero times and your program reports success on an empty result. Check it:

```
std::ifstream in(path);
if (!in) { throw LogError("cannot open " + path); }
```

Note the shape of the read loop above: `while (in >> name >> mv)`. **Loop on the operation, never on `eof()`.** `while (!in.eof())` is wrong because `eof()` is only set *after* a read has already failed, so the body runs one extra time on stale data. The same rule applies to `getline` and to `read`.

11.3 Text or Binary — the Decision Is About Who Reads It

Text is for humans and for other programs' convenience. Binary is for size, speed, and exactness — and for talking to hardware, where the format is not yours to choose.

On Windows, `std::ios::binary` is not optional. Without it, a byte that happens to equal `0x0A` is translated on the way out and your file is quietly corrupt. This is a real difference you will hit on the lab machines and not on a Mac.

```
std::ofstream f("log.bin", std::ios::binary);
```

11.4 The Record on Disk Is Not the Struct in Memory

```
struct SensorSample { uint8_t id; uint16_t flags; float temp; uint32_t ms; };
```

```
sizeof(SensorSample) = 12      wire record = 11
```

Twelve bytes in memory, eleven on the wire. The difference is padding — Chapter 5's lesson, now with a consequence: **write the struct's bytes directly and you write twelve bytes, one of which is garbage the receiver will misread.**

So the record format is specified independently of any struct:

Offset	Size	Field	Encoding
0	1	`id`	`uint8_t`
1	2	`flags`	`uint16_t`, big-endian
3	4	`temp`	IEEE-754 `binary32`, big-endian
7	4	`ms`	`uint32_t`, big-endian

Total 11 bytes. Note `temp` at **offset 3** — not a multiple of 4. No C++ struct can place a `float` there, and on some processors a 4-byte load from an odd address faults outright. That misalignment is why the next section exists.

11.5 Field by Field, with Explicit Byte Order

```cpp
void encode(const SensorSample& s, uint8_t* out) {
    out[0] = s.id;
    out[1] = static_cast<uint8_t>(s.flags >> 8);
    out[2] = static_cast<uint8_t>(s.flags);
    uint32_t t = std::bit_cast<uint32_t>(s.temp);
    out[3] = static_cast<uint8_t>(t >> 24);
    out[4] = static_cast<uint8_t>(t >> 16);
    out[5] = static_cast<uint8_t>(t >> 8);
    out[6] = static_cast<uint8_t>(t);
    // ... ms the same way
}
```

Encoding `{7, 0x00A5, -12.5f, 0xDEADBEEF}`:

```
wire bytes: 07 00 A5 C1 48 00 00 DE AD BE EF
```

Read those bytes against the table. `07` is the id. `00 A5` is the flags, most significant byte first. `C1 48 00 00` is −12.5 as an IEEE-754 float, big-endian. `DE AD BE EF` is the timestamp, and you can read it straight off the page — which is the point of choosing a byte order and sticking to it.

Three rules:

1. **Assemble multi-byte values by shifting and masking**, not with a cast. This is Chapter 2's bit manipulation, returning nine weeks later.
2. **Cast to `uint32_t` before shifting.** `b[0] << 24` on a `uint8_t` promotes to `int` and shifts into the sign bit — Chapter 2's promotion trap, now with a real consequence.
3. **The float goes through `std::bit_cast`.** Assemble the four bytes into a `uint32_t` in host order first, then reinterpret. Never byte-swap a `float` directly.

Decoding runs the same way in reverse:

```cpp
uint32_t t = (uint32_t(b[3]) << 24) | (uint32_t(b[4]) << 16)
           | (uint32_t(b[5]) <<  8) |  uint32_t(b[6]);
float temp = std::bit_cast<float>(t);
```

11.6 The Contrast: What Everyone Else Does

```cpp
#pragma pack(push, 1)
struct Packed { uint8_t id; uint16_t flags; float temp; uint32_t ms; };
#pragma pack(pop)

std::ofstream f("log.bin", std::ios::binary);
f.write(reinterpret_cast<const char*>(&sample), sizeof(Packed));   // 11 bytes. done?
```

This is shorter, it is what most tutorials show, and **on your machine it works**. Run it and it produces an eleven-byte record.

Here is precisely what is wrong with it:

- **It is not portable.** `#pragma pack` is a compiler extension, not standard C++.

- **It writes host byte order.** Move the file to a big-endian machine, or send it to a device expecting network order, and every multi-byte field is reversed.
- **It is undefined behaviour** to read those bytes back through a pointer of a different type — the strict-aliasing rule.
- **Taking the address of a packed member** and dereferencing it faults on strict-alignment targets.

And here is what is **not** wrong with it, because being imprecise costs you the argument: on Clang and GCC a packed struct is accessed byte-wise, so it does **not** fault on x86-64. If you try it and it works, that is expected. It works on *this* machine, writing a file *this* machine will read. The field-by-field version works everywhere, forever, and that is the whole difference.

11.7 Buffering, and Why Output Disappears

A stream does not write to disk on every `<<`. It fills a buffer and flushes when the buffer is full, when you ask, or when the stream is destroyed.

That is normally invisible — until a program crashes and the last thing you printed never appears. You met this in Chapter 6's double-free demo, where `std::endl` was needed to see the output before the abort.

- `"\n"` — a newline. No flush.
- `std::endl` — a newline **and a flush**. Slower.
- `std::flush` — a flush, no newline.

Prefer `"\n"`. Use `std::endl` when you genuinely need the bytes out now: logging around a crash, or progress on a line that must appear before the next slow operation.

11.8 `std::streambuf`, in One Paragraph

Every stream is two pieces: the **stream** (`ostream` — formatting and `<<`) and the **stream buffer** (`streambuf` — the bytes, and where they go). `std::cout` is an `ostream` wrapped around a `streambuf` that writes to standard output; an `ofstream` is the same `ostream` interface over a `streambuf` that writes to a file.

That separation is why you can redirect `std::cout` into a file by swapping its buffer, and why writing a custom `streambuf` lets `<<` write anywhere — a serial port, a network socket, a ring buffer.

11.9 A User-Defined `operator<<`

```
std::ostream& operator<<(std::ostream& os, const SensorSample& s) {
    return os << "sensor " << int(s.id)
              << " temp " << s.temp
              << " at " << s.ms << "ms";
}

std::cout << sample << "\n";      // now works
```

Take the stream by non-`const` reference, take your type by `const` reference, and **return the stream** so calls chain. That is the whole pattern, and it works for `std::cout`, an `ofstream`, or anything else that is an `ostream`.

Note `int(s.id)`. Without the cast, a `uint8_t` prints as a character — Chapter 2 again.

11.10 Worked Example

Problem: write sensor records to a binary log, read them back, and prove the round trip.

```
#include <bit>
#include <cstdint>
#include <fstream>
#include <iostream>
#include <vector>

struct SensorSample { uint8_t id; uint16_t flags; float temp; uint32_t ms; };
constexpr std::size_t kRecord = 11;

void put32(uint8_t* p, uint32_t v) {
    p[0] = uint8_t(v >> 24); p[1] = uint8_t(v >> 16);
    p[2] = uint8_t(v >>  8); p[3] = uint8_t(v);
}
uint32_t get32(const uint8_t* p) {
    return (uint32_t(p[0]) << 24) | (uint32_t(p[1]) << 16)
         | (uint32_t(p[2]) <<  8) |  uint32_t(p[3]);
}

void encode(const SensorSample& s, uint8_t* out) {
    out[0] = s.id;
    out[1] = uint8_t(s.flags >> 8);
    out[2] = uint8_t(s.flags);
    put32(out + 3, std::bit_cast<uint32_t>(s.temp));
    put32(out + 7, s.ms);
}

SensorSample decode(const uint8_t* in) {
    SensorSample s{};
    s.id    = in[0];
    s.flags = uint16_t((uint16_t(in[1]) << 8) | in[2]);
```

```
    s.temp  = std::bit_cast<float>(get32(in + 3));
    s.ms    = get32(in + 7);
    return s;
}

int main() {
    std::vector<SensorSample> samples{
        {7, 0x00A5, -12.5f, 0xDEADBEEF}, {2, 0x0001, 21.25f, 1000}};

    {
        std::ofstream f("log.bin", std::ios::binary);
        if (!f) { std::cerr << "cannot open for writing\n"; return 1; }
        for (const auto& s : samples) {
            uint8_t buf[kRecord];
            encode(s, buf);
            f.write(reinterpret_cast<const char*>(buf), kRecord);
        }
    }   // destructor closes and flushes

    std::ifstream f("log.bin", std::ios::binary);
    if (!f) { std::cerr << "cannot open for reading\n"; return 1; }

    uint8_t buf[kRecord];
    while (f.read(reinterpret_cast<char*>(buf), kRecord)) {
        SensorSample s = decode(buf);
        std::cout << "id " << int(s.id) << " flags " << s.flags
                  << " temp " << s.temp << " ms " << s.ms << "\n";
    }
    std::cout << "trailing bytes: " << f.gcount() << "\n";
}
```

```
id 7 flags 165 temp -12.5 ms 3735928559
id 2 flags 1 temp 21.25 ms 1000
trailing bytes: 0
```

Three things worth noticing. The writing block is in **braces** so the `ofstream` is destroyed — and therefore flushed and closed — before the file is reopened for reading. The loop condition is `while (f.read(...))`, the operation and not the flag. And `gcount()` after the loop reports **0**, meaning the file ended exactly on a record boundary; any other number means the file is truncated and you should say so rather than ignore it.

11.11 Predict Before You Run

Predict before you run. What does this print for a file that does not exist?

```
std::ifstream in("no-such-file.txt");
std::string line;
int n = 0;
while (std::getline(in, line)) { ++n; }
std::cout << "read " << n << " lines\n";
```

```
read 0 lines
```

No error, no exception, no warning. The open failed, the stream is in a failed state, the loop body never runs, and the program cheerfully reports a result. This is the most common file bug there is, and it does not look like a bug — it looks like an empty file.

Add `if (!in) { throw ... }` after the open. Always.

11.12 Common Errors

Symptom	Cause	Fix
Program reports zero records from a real file	The open failed silently	Check `if (!in)` immediately after opening
Last record processed twice	Looping on `!in.eof()`	Loop on the operation: `while (in.read(...))`
File corrupt on Windows, fine on Mac	Missing `std::ios::binary`	Add the flag for every binary stream
Multi-byte field reversed on another machine	Host byte order written to disk	Encode field by field, big-endian
A shifted byte comes out negative or wrong	`uint8_t << 24` promoted to `int`	Cast to `uint32_t` before shifting
Record is one byte longer than the spec	The struct's padding was written	Encode field by field; never write the struct
Output missing after a crash	Buffered, never flushed	`std::endl` or `std::flush` where it matters
`uint8_t` field prints as a letter	It is `unsigned char`	Cast to `int` for printing

11.13 Key Terms

`ifstream` / `ofstream` — RAII input and output file streams; the constructor opens, the destructor closes.

`std::ios::binary` — the open flag that disables newline translation; mandatory on Windows for binary files.

wire format — the byte layout of a record on disk or on a link, specified independently of any struct.

big-endian — most significant byte first; the conventional on-disk and network order.

`std::bit_cast` — reinterprets an object's bits as another type of the same size, without undefined behaviour.

strict aliasing — the rule that reading an object through a pointer of an unrelated type is undefined behaviour.

buffering — a stream accumulates output and writes it in blocks rather than per operation.

`std::streambuf` — the byte-moving half of a stream, separate from the formatting half.

`gcount()` — the number of characters the last unformatted read actually extracted.

11.14 Exercises

1. **(Trace)** Predict each number before compiling.

```
struct A { uint8_t id; uint32_t ms; };
struct B { uint32_t ms; uint8_t id; };
struct C { uint8_t id; uint16_t flags; float temp; uint32_t ms; };
// sizeof(A)? sizeof(B)? sizeof(C)? And the wire size of C's 4 fields?
```

2. **(Trace)** Predict all four lines. The two assemblies differ only in the casts.

```
uint8_t b[4] = {0xDE, 0xAD, 0xBE, 0xEF};

uint64_t bad  = (b[0] << 24) | (b[1] << 16) | (b[2] << 8) | b[3];
uint64_t good = (uint32_t(b[0]) << 24) | (uint32_t(b[1]) << 16)
              | (uint32_t(b[2]) << 8)  |  uint32_t(b[3]);

std::printf("bad  %016llX\n", (unsigned long long)bad);
std::printf("good %016llX\n", (unsigned long long)good);
std::printf("positive? %s\n", ((b[0] << 24) > 0) ? "yes" : "no");
std::printf("scaled %.1f\n", (b[0] << 24) / 1000.0);
```

3. **(Find the bug)** Three defects. Name each and say what it costs.

```cpp
void save(const std::vector<SensorSample>& v, const std::string& path) {
    std::ofstream f(path);
    for (const auto& s : v) {
        f.write(reinterpret_cast<const char*>(&s), sizeof(SensorSample));
    }
}

std::vector<SensorSample> load(const std::string& path) {
    std::ifstream f(path, std::ios::binary);
    std::vector<SensorSample> v;
    SensorSample s;
    while (!f.eof()) {
        f.read(reinterpret_cast<char*>(&s), sizeof(SensorSample));
        v.push_back(s);
    }
    return v;
}
```

4. **(Faded)** Complete the decoder for the 11-byte record.

```cpp
SensorSample decode(const uint8_t* in) {
    SensorSample s{};
    s.id    = in[0];
    s.flags = uint16_t((uint16_t(in[1]) << 8) | in[2]);

    uint32_t raw = (____________ << 24) | (____________ << 16)     // (A) note the casts
                 | (____________ <<  8) |  ____________;
    s.temp = _____________________(raw);                           // (B) bits, not a va

    s.ms = /* the same four-byte assembly from offset 7 */;         // (C)
    return s;
}
```

5. **(Spaced recall — Chapters 2, 5, and 10)** A record is 11 bytes on the wire and its struct is 12 in memory. Name the Chapter 5 mechanism responsible. Then say why `uint32_t(b[0]) << 24` needs its cast, in Chapter 2's terms. Finally, a `SensorLog` class opens its file in the constructor and the open fails: what should the constructor do, and what does Chapter 10 say about the state of a half-constructed object?

6. **(Write from scratch)** Build a binary sensor log. Specify an on-disk record format in a table first, then write `encode` and `decode` field by field with explicit big-endian order and `std::bit_cast` for the float. Write at least three records, read them back, and assert the round trip is exact. Then **truncate the file by three bytes** and show that your reader detects it rather than returning a bad final record.

11.15 Self-Check

1. True or false: opening a file that does not exist throws an exception in C++.

2. Why must a binary file on Windows be opened with `std::ios::binary`?
3. The wire format puts `temp` at offset 3. Why does that matter?
4. How do you assemble a big-endian `uint32_t` from four bytes?
5. Why must you cast to `uint32_t` **before** shifting a `uint8_t` left by 24?
6. You need to byte-swap a `float`. What do you do?
7. Select every true charge against `reinterpret_cast` onto a `#pragma pack`ed struct.
   - `#pragma pack` is a compiler extension, not standard C++
   - It writes host byte order
   - Reading the bytes back through a different pointer type is undefined behaviour
   - It always faults on x86-64
8. Why is `while (!in.eof())` the wrong way to read a file?
9. After the read loop ends, `in.gcount()` returns 3 on a file of 11-byte records. What does that mean, and what should the program do?
10. Why should a decode function fill a temporary and assign to its output parameter only on success?

11.16 Summary

Close the book and answer from memory.

- What happens when an `ifstream` fails to open, and what must you write?
- Why is the record on disk a different size from the struct in memory?
- Give the three rules for encoding a multi-byte field.
- State the four charges against the packed-struct idiom — and the one charge that is *not* true.
- Why loop on the read rather than on `eof()`?
- What are the two halves of a stream?

The next chapter runs two things at once, where a shared variable read by one thread and written by another is no longer a question about correctness of logic but about time.

Part VI

Concurrency and Performance

Chapter 12

Concurrency and Multithreading

12.1 What You'll Be Able to Do

- Launch and join a `std::thread`, and pass data to it safely.
- Explain a data race, demonstrate one, and protect shared state with `std::mutex` and `lock_guard`/`unique_lock`.
- Use a `std::condition_variable` to coordinate a producer/consumer hand-off.
- Explain deadlock and the lock-ordering discipline that avoids it.
- Describe `std::atomic`, and say why `volatile` is **not** a synchronization tool.

12.2 One Thread, Then Two

```
#include <thread>

void work() { /* ... */ }

int main() {
    std::thread t(work);
    t.join();            // wait for it to finish
}
```

A `std::thread` starts running the moment it is constructed. Before its destructor runs you must **`join()`** it (wait) or **`detach()`** it (abandon it). Do neither and the destructor calls `std::terminate` — the program dies deliberately, rather than leaving a thread running over freed memory.

Arguments go after the function and are **copied** by default. To pass a reference, wrap it in `std::ref` — and take on the responsibility of ensuring the referent outlives the thread. A reference to an expiring local is Chapter 4's dangling reference with a race attached.

12.3 The Race

Read this before the word "mutex" appears.

```
int counter = 0;
void bump() { for (int i = 0; i < 100000; ++i) { ++counter; } }

std::thread a(bump), b(bump);
a.join(); b.join();
// counter should be 200000
```

Three consecutive runs:

```
run 1 unsafe: 101969  (expected 200000)
run 2 unsafe: 107174  (expected 200000)
run 3 unsafe: 101388  (expected 200000)
```

Wrong every time, and **wrong differently every time.**

`++counter` is not one operation. It is read, add, write. Two threads interleave those three steps and updates are lost — roughly half of them here, though the exact number is unrepeatable.

That unrepeatability is the whole difficulty of this chapter. Every bug you have met so far has been deterministic: the same input produced the same wrong answer, and you could re-run it under a debugger. This one produces a different wrong answer every time, and running it under a debugger changes the timing enough that it may not happen at all.

And with a smaller loop count it often prints 200000 and looks correct. A threaded program that passes a hundred runs is not a correct threaded program; it is a program whose race has not yet been observed.

12.4 `std::mutex` and `lock_guard`

```
std::mutex m;

void bumpSafe() {
    for (int i = 0; i < 100000; ++i) {
        std::lock_guard<std::mutex> lk(m);
        ++counter;
    }
}
```

```
safe: 200000  (expected 200000)
```

Every time.

A mutex is a token: only the holder may proceed, and everyone else waits. `lock_guard` is **RAII for locks** — it locks in its constructor and unlocks in its destructor, so the lock is released on every exit path, including one taken by an exception.

Never call `m.lock()` and `m.unlock()` by hand. Chapter 5's argument, unchanged: the early return you add in six months will not reach the unlock. And the failure is worse than a leak — a leaked allocation costs memory, while a leaked lock stops the program forever, with no error message.

`std::unique_lock` is `lock_guard`'s flexible sibling: it can be unlocked early and re-locked, which is what a condition variable requires. Use `lock_guard` unless you need that.

12.5 Hold the Lock for as Little as Possible

Everything inside the lock is serialised — no two threads run it at once. So do the work outside and the update inside:

```
int computed = expensiveCalculation();                          // outside
{ std::lock_guard<std::mutex> lk(m); shared = computed; }        // inside, briefly
```

Note the braces. They exist to end the guard's scope early, which is the only way to release a `lock_guard` before the end of the function.

12.6 A Condition Variable

A mutex protects data. It does not let one thread *wait for* another to do something. That is what a condition variable is for.

```
std::queue<int> q;
std::mutex m;
std::condition_variable notEmpty, notFull;
bool done = false;
constexpr std::size_t kCap = 4;

void producer(int n) {
    for (int i = 1; i <= n; ++i) {
        std::unique_lock<std::mutex> lk(m);
        notFull.wait(lk, [] { return q.size() < kCap; });
        q.push(i);
        lk.unlock();
        notEmpty.notify_one();
    }
    { std::lock_guard<std::mutex> lk(m); done = true; }
    notEmpty.notify_all();
}

void consumer() {
    for (;;) {
        std::unique_lock<std::mutex> lk(m);
        notEmpty.wait(lk, [] { return !q.empty() || done; });
        if (q.empty() && done) { break; }
        int v = q.front(); q.pop();
        lk.unlock();
        notFull.notify_one();
        // ... use v ...
    }
}
```

```
produced 9 (queue 1)
produced 10 (queue 2)
consumed 10 items, total 55
both threads joined; queue size 0
```

Four things in that code are not obvious, and each is a bug if you get it wrong.

1. **`wait` takes a predicate.** `notEmpty.wait(lk, pred)` re-checks `pred` every time it wakes, and goes back to sleep if it is false. Without the predicate you must handle *spurious wakeups* yourself — a `wait` can return with nothing having happened.
2. **`wait` unlocks while it sleeps** and re-locks before returning. That is why it needs a `unique_lock` rather than a `lock_guard`: it has to be able to unlock.
3. **The `done` flag is protected by the same mutex** as the queue. A flag shared between threads is shared state like any other.
4. **The consumer's exit condition is `q.empty() && done`**, not `done`. Finishing production does not mean the queue is drained, and the difference is the last few records silently vanishing.

Note also the final `notify_all` rather than `notify_one`. When you set `done`, every waiting consumer needs to learn it — waking one and leaving the rest asleep means they wait forever.

12.7 Deadlock, and the Discipline That Prevents It

Two locks, two threads, opposite orders:

```
Thread 1: lock A ... then lock B
Thread 2: lock B ... then lock A
```

Each holds what the other needs. Neither proceeds. Nothing crashes and nothing reports an error — **the program simply stops**, which is why deadlock is diagnosed by attaching a debugger and reading the stacks of both threads.

The discipline: always acquire multiple locks in the same global order. Write the order down. If every thread takes A before B, the cycle cannot form.

12.8 `std::atomic`, Surveyed

```
std::atomic<int> counter{0};
++counter;      // indivisible: no lock needed
```

An atomic operation cannot be interleaved. For a single counter or flag, `std::atomic` is simpler and faster than a mutex.

Its limit is that it protects **one variable**. The moment you need two things to change together — pop from the queue *and* decrement the count — you need a mutex, because atomicity per variable is not atomicity across an operation.

12.9 `volatile` Is Not a Synchronization Tool

In Chapter 4 you learned that `volatile` tells the compiler a location may change outside the program's control, so accesses must not be optimised away. That is true, and it is **the whole of what `volatile` does.**

It does **not**:

- make an operation atomic — `++v` on a `volatile int` is still read-add-write, and still races;
- order operations between threads;
- publish one thread's writes to another.

The confusion is inherited from Java and C#, where `volatile` genuinely does have threading semantics. **In C++ it does not.**

Here is the proof. Make the racing counter `volatile` and run it again:

```
run 1 volatile counter: 110272  (expected 200000)
run 2 volatile counter: 113520  (expected 200000)
run 3 volatile counter: 113808  (expected 200000)
```

Still wrong, still differently wrong each time. `volatile` changed nothing about the race, because it was never about races.

Use `volatile` for memory-mapped hardware, and `std::atomic` or a mutex for threads. They are unrelated tools that share a name.

12.10 Worked Example

Problem: show the race, fix it, and prove both.

```
#include <iostream>
#include <mutex>
#include <thread>

constexpr int kIters = 100000;

int  unsafeCounter = 0;
int  safeCounter   = 0;
std::mutex m;
```

```cpp
void bumpUnsafe() { for (int i = 0; i < kIters; ++i) { ++unsafeCounter; } }

void bumpSafe() {
    for (int i = 0; i < kIters; ++i) {
        std::lock_guard<std::mutex> lk(m);
        ++safeCounter;
    }
}

int main() {
    {
        std::thread a(bumpUnsafe), b(bumpUnsafe);
        a.join(); b.join();
    }
    {
        std::thread a(bumpSafe), b(bumpSafe);
        a.join(); b.join();
    }
    std::cout << "unsafe: " << unsafeCounter << "  (expected " << 2 * kIters << ")\n";
    std::cout << "safe:   " << safeCounter   << "  (expected " << 2 * kIters << ")\n";
}
```

```
unsafe: 100000  (expected 200000)     <- run 1
unsafe: 106552  (expected 200000)     <- run 2
unsafe: 100342  (expected 200000)     <- run 3
safe:   200000  (expected 200000)     <- every run
```

Run the unsafe half several times. The number changes every run, which is the evidence that matters — a single wrong number could be a logic error, while a *different* wrong number each time can only be a race.

12.11 Predict Before You Run

Predict before you run. What happens?

```cpp
int main() {
    std::thread t([]{ std::cout << "working\n"; });
    std::cout << "main done\n";
}                                   // t is destroyed here - never joined
```

The program **calls `std::terminate` and aborts.** Observed:

```
libc++abi: terminating
main done
working
```

with exit code 134. The three lines can arrive in any order, and which of them appear at all depends on timing — but the program will not exit normally.

This is a deliberate design choice in the standard library. The alternative — silently detaching — would leave a thread running over `main`'s dead locals. Crashing loudly at the point of the mistake is better than corrupting memory somewhere else.

12.12 Common Errors

Symptom	Cause	Fix
`terminate called` at the end of a scope	A `std::thread` destroyed without `join()` or `detach()`	Join it before it goes out of scope
Total is wrong, differently each run	A data race on shared state	Guard it with a mutex, or use `std::atomic`
Correct for a hundred runs, then wrong	The same race, not yet observed	Testing cannot prove a race absent; reason about the code
Program stops, no error, no CPU use	Deadlock	Attach a debugger, read both stacks; impose a lock order
Lock never released after an early return	`lock()`/`unlock()` called by hand	`std::lock_guard`
Consumer misses the last few items	Exit condition is `done` rather than `q.empty() && done`	Drain before exiting
Waiting thread never wakes	`notify_one` where every waiter needed to learn	`notify_all` when setting a terminal flag
A thread reads a stale value despite `volatile`	`volatile` does not synchronise	`std::atomic` or a mutex

12.13 Key Terms

thread — an independent flow of execution within a process.

`join()` — wait for a thread to finish. **`detach()`** — abandon it. One is mandatory before destruction.

data race — two threads accessing the same location concurrently with at least one write, and no synchronization. Undefined behaviour.

`std::mutex` — a lock permitting one holder at a time.

`std::lock_guard` — RAII for a mutex: locks on construction, unlocks on destruction.

`std::unique_lock` — a lock guard that can be released and re-acquired; required by `condition_variable::wait`.

`std::condition_variable` — lets a thread sleep until another signals that a condition may have become true.

spurious wakeup — a `wait` returning without a notification, which is why `wait` takes a predicate.

deadlock — two or more threads each holding a lock the other needs; the program stops without error.

`std::atomic` — a type whose operations cannot be interleaved; protects one variable, not an operation across several.

12.14 Exercises

1. **(Trace)** What are the possible outputs, and what is guaranteed?

```
int main() {
    std::thread a([]{ std::cout << "A"; });
    std::thread b([]{ std::cout << "B"; });
    a.join();
    b.join();
    std::cout << "\ndone\n";
}
```

2. **(Trace)** Run this ten times. What do you expect, and what is the *significant* observation?

```
int counter = 0;
void bump() { for (int i = 0; i < 1000; ++i) { ++counter; } }

int main() {
    std::thread a(bump), b(bump);
    a.join(); b.join();
    std::cout << counter << "\n";
}
```

3. **(Find the bug)** Three defects. Name each, and say which produces a wrong answer, which stops the program, and which crashes it.

```
std::queue<int> q;
std::mutex m;
std::condition_variable cv;
bool done = false;

void producer() {
    for (int i = 0; i < 10; ++i) {
        m.lock();
        q.push(i);
        if (i == 5) { return; }            // early out on purpose
        m.unlock();
        cv.notify_one();
    }
    done = true;
}

void consumer() {
```

```
    while (!done) {
        std::unique_lock<std::mutex> lk(m);
        cv.wait(lk);
        int v = q.front(); q.pop();
        std::cout << v << " ";
    }
}
```

4. **(Faded)** Complete the bounded queue hand-off.

```
void producer(int n) {
    for (int i = 1; i <= n; ++i) {
        std::_________________<std::mutex> lk(m);            // (A) which lock type
        notFull.wait(lk, [] { return ____________; });       // (B) the predicate
        q.push(i);
        lk.unlock();
        notEmpty.____________();                             // (C) wake one waiter
    }
    { std::lock_guard<std::mutex> lk(m); done = true; }
    notEmpty.____________();                                 // (D) wake every waiter
}
```

5. **(Spaced recall — Chapters 5 and 10)** `std::lock_guard` and the RAII wrapper you wrote in Chapter 5 are the same idea. State the idea once, then say what each acquires and releases. Then the Chapter 10 half: a function holds a lock and throws. Is the lock released? Explain what mechanism guarantees your answer, and what would happen had the code called `m.lock()` and `m.unlock()` directly.

6. **(Write from scratch)** Build a sensor replay queue: a producer thread reads records and pushes them into a bounded queue of capacity 4; a consumer thread pops and totals them. Use a mutex and two condition variables. Prove with output that **every** record produced was consumed, that the queue never exceeded its capacity, and that both threads terminated. Then remove the `notify_all` on `done` and describe what changes.

12.15 Self-Check

1. What must you do with a `std::thread` before its destructor runs, and what happens if you do neither?

2. Two threads each run `for (int i = 0; i < 100000; ++i) { ++counter; }`. What is the most likely outcome?

3. Your threaded program printed the correct answer on a hundred consecutive runs. Is that good evidence it is correct?

4. What is `std::lock_guard`, and why prefer it to calling `lock()` and `unlock()` yourself?

5. Why is forgetting to unlock worse than forgetting to `delete`?

6. Why must `wait` be given a predicate?

7. In a bounded producer/consumer queue, select every true statement.

    - The `done` flag must be protected by the same mutex as the queue
    - The consumer should exit when `done` becomes true
    - `wait` releases the mutex while it sleeps
    - A `lock_guard` cannot be used with `wait`

8. When the producer sets `done`, should it call `notify_one` or `notify_all`?

9. What is deadlock, and what discipline prevents it?

10. Select every true statement about `volatile` in C++.

    - It prevents the compiler optimising away accesses
    - It makes `++v` atomic
    - It orders operations between threads
    - It is the right tool for memory-mapped hardware

12.16 Summary

Close the book and answer from memory.

- What are the two things you may do with a thread, and what happens if you do neither?
- Why is a wrong answer that differs every run stronger evidence than a single wrong answer?
- State the argument for `lock_guard` over manual locking, and name the chapter it comes from.
- Give the four non-obvious details of a condition-variable hand-off.
- What does `std::atomic` protect, and what does it not?
- What does `volatile` do, and what does it emphatically not do?

The last full chapter steps back from the language to ask how the pieces should be arranged: the remaining SOLID principles, measuring before optimising, and what the compiler does to your code when you finally let it.

Chapter 13

Architecture and Performance

13.1 What You'll Be Able to Do

- State all five SOLID principles, identify a violation, and give its fix.
- Describe MVC and use it to separate model, control logic, and presentation.
- Locate a bottleneck by **measuring** rather than guessing, using `std::chrono` and an optimisation-level comparison.
- Apply an algorithmic improvement and a data-structure change, and measure both.
- Recommend, with evidence, which optimisation to keep — including recommending none.

13.2 SOLID, and the Two You Already Built

In Chapter 7 you wrote an abstract `Sensor` with two pure virtual functions, and a loop that used it without knowing which sensor it had. That was two principles:

- **Interface Segregation** — `Sensor` declared what every sensor needs, and nothing else.
- **Dependency Inversion** — the loop depended on the abstraction, not on `Thermocouple`.

Three remain, and each is a **question you can ask about a class**.

13.2.1 Single Responsibility — *what would make this change?*

A class should have one reason to change.

```
// violates SRP: three reasons to change
class SensorManager {
    void readHardware();
    void writeToDatabase();
    void renderDashboard();
};
```

A new sensor changes this class. A new storage format changes this class. A UI redesign changes this class. Three unrelated teams editing one file.

The test is **not** "does it do one thing" — that is unfalsifiable, because anything can be described as one thing. The test is **what would make this change?** If the answers come from different directions, split it.

13.2.2 Open/Closed — *can I add without editing?*

> Open for extension, closed for modification.

```
// violates OCP
double area(const Shape& s) {
    if (s.kind == Circle) { return ...; }
    else if (s.kind == Square) { return ...; }
    // every new shape edits this function
}
```

Every new shape means editing existing, working, tested code. The virtual-function version from Chapter 7 adds a shape by *adding a class* — nothing already working is touched.

13.2.3 Liskov Substitution — *does the derived class keep the promises?*

> A derived object must be usable anywhere its base is expected, without the caller knowing.

The classic violation is `Square` deriving from `Rectangle`: `setWidth` and `setHeight` are independent on a rectangle and cannot be on a square, so code that works on `Rectangle` breaks on `Square` — not by failing to compile, but by producing wrong answers.

Nearer to home: a `ReadOnlyLog` deriving from `Log` and throwing on `write()`. It compiles, it satisfies the interface, and it breaks every caller that reasonably expected `write` to work.

13.3 MVC on a Simulated Device Panel

Three responsibilities, kept apart:

- **Model** — the data and the rules. Sensor readings, thresholds, whether a reading is a fault. Knows nothing about screens.
- **View** — presentation. Turns the model into text on a console or pixels on a panel. Decides nothing.
- **Controller** — input and coordination. Takes a command, tells the model, asks the view to refresh.

```cpp
class SensorModel {          // no printing anywhere
    std::vector<SensorSample> samples_;
public:
    void add(const SensorSample& s);
    float peakTemp() const;
    bool inFault() const;
};

class ConsoleView {          // no decisions anywhere
public:
    void render(const SensorModel& m) const;
};

class PanelController {      // wires the two together
    SensorModel& model_;
    ConsoleView& view_;
public:
    void onSample(const SensorSample& s) { model_.add(s); view_.render(model_); }
};
```

The value is testability. `SensorModel` can be unit-tested without a screen, because it never prints — which is Chapter 10's argument arriving as an architectural constraint rather than a testing tip.

Note that this is Single Responsibility applied at a larger scale. MVC is not a separate idea; it is SRP with the three responsibilities named in advance.

13.4 Measure, Do Not Guess

The rule for this chapter: **you may not optimise anything you have not measured.**

Instrument with `std::chrono`:

```cpp
using clk = std::chrono::steady_clock;

auto t0 = clk::now();
doTheWork();
auto t1 = clk::now();
std::cout << std::chrono::duration_cast<std::chrono::microseconds>(t1 - t0).count() << " us\n"
```

Use `steady_clock`, not `system_clock` — the latter can jump when the machine's clock is adjusted.

Measure in microseconds, not milliseconds. A fast variant that reports `0 ms` tells you nothing, and comparing variants is the entire point.

13.5 The Measurement, and Its Surprise

Searching 20,000 shuffled integers for 2,000 targets, three ways:

	`-O0`	`-O2`
linear scan (`std::find`)	51,658 µs	5,549 µs
sort once, then `binary_search`	**1,297 µs**	**374 µs**
build a `std::set`, then `count`	8,595 µs	2,192 µs

> **Predict before you read on.** Rank those three from fastest to slowest. Write your ranking down first.

Three things in that table, and the third is the one worth the chapter.

1. The algorithm beats the flag. Sorting and binary-searching at `-O0` — the *unoptimised* build — is four times faster than a linear scan at `-O2`. **You cannot compile your way out of a bad algorithm.**

2. `-O2` is worth several times over, across the board — between three and nine times in the run above. Free, and it changes no ranking. Always measure both, and never report a timing from a debug build without saying so.

A note on these figures. They come from one machine on one afternoon. Run the same program yourself and the numbers will differ — a repeat run while writing this chapter gave 60,664 / 1,150 / 6,313 at `-O0`. What did **not** change is the ranking, or any conclusion drawn from it. That is the property to rely on: absolute timings are a fact about your machine, and comparisons between variants measured the same way are a fact about your code.

3. `std::set` loses to a sorted vector, by 3–6× — despite both being logarithmic. This is the one nobody predicts. `std::set` is a node-based tree: every element is a separate allocation somewhere else in memory, and every lookup chases pointers around the heap. A sorted `std::vector` is one contiguous block, and the CPU's cache loves it.

Big-O tells you how a cost *grows*. It says nothing about the constant, and on modern hardware the constant is largely a question of memory layout.

The ranking depends on the workload. Sorting costs something up front and pays it back over many lookups. With 2,000 lookups the sort is repaid many times over; with three lookups, the linear scan wins outright, because you never recover the sorting cost. Measure the workload you actually have.

13.6 What to Do with a Measurement

1. **Measure first.** Find the actual bottleneck, not the one that looks slow.
2. **Change one thing.** Two changes and one number tells you nothing.
3. **Measure again**, the same way, on the same data.
4. **Decide with evidence** — and be willing to revert. An optimisation that costs readability and buys 3% is a bad trade.

The 20,000-element table is a case where the answer is clear. **Most are not**, and being able to say *"the measurement does not justify this change"* is as much a competency as making the change.

13.7 Where the Debugger Fits

LLDB is not only for crashes. Break inside a hot loop and inspect what is actually in the container; `bt` tells you which call path is reaching this code, which is often the real question when something runs far more often than expected.

A profiler is the right tool, and you do not have one here. `perf` is Linux-only and absent from this toolchain, so `std::chrono` around a suspect region is the honest substitute. The difference matters: a real profiler finds the hot spot *for* you, while manual timing only confirms a place you already suspected. If your suspicion is wrong, timing will not tell you.

13.8 Worked Example

Problem: a lookup is slow. Find out why, fix it, and justify the fix with numbers.

```
#include <algorithm>
#include <chrono>
#include <iostream>
#include <random>
#include <set>
#include <vector>

using clk = std::chrono::steady_clock;

template <typename F>
long long timeIt(F f) {
    auto t0 = clk::now();
    f();
    auto t1 = clk::now();
    return std::chrono::duration_cast<std::chrono::microseconds>(t1 - t0).count();
}

int main() {
    constexpr int kN = 20000, kQ = 2000;
    std::vector<int> data(kN);
    for (int i = 0; i < kN; ++i) { data[i] = i; }
    std::mt19937 rng(42);                          // fixed seed: repeatable
    std::shuffle(data.begin(), data.end(), rng);

    std::vector<int> targets(kQ);
    for (int i = 0; i < kQ; ++i) { targets[i] = int(rng() % kN); }

    volatile int sink = 0;                         // stop the optimiser deleting the work

    std::cout << "linear   " << timeIt([&]{
        for (int t : targets) { sink += (std::find(data.begin(), data.end(), t) != data.end())
    }) << " us\n";
```

```
    std::cout << "sorted   " << timeIt([&]{
        std::vector<int> s = data;
        std::sort(s.begin(), s.end());
        for (int t : targets) { sink += std::binary_search(s.begin(), s.end(), t); }
    }) << " us\n";

    std::cout << "set      " << timeIt([&]{
        std::set<int> s(data.begin(), data.end());
        for (int t : targets) { sink += int(s.count(t)); }
    }) << " us\n";
}
```

Two details that are easy to get wrong and that invalidate a measurement if you do.

The fixed seed. `std::mt19937 rng(42)` means every run measures the same data. Without it you are comparing variants on different inputs.

The `volatile int sink`. At `-O2` the compiler can see that the search results are never used and delete the entire loop, producing a spectacular and meaningless `0 µs`. Accumulating into a `volatile` forces the work to happen. If your optimised timings come out near zero, this is almost always why.

13.9 Common Errors

Symptom	Cause	Fix
`-O2` timings all report ~0 µs	The optimiser deleted work whose result is unused	Accumulate into a `volatile` sink
Timings vary wildly between runs	Different data each run, or machine load	Fix the seed; run several times; compare medians
A "fast" variant reports 0 ms	Millisecond resolution is too coarse	Measure microseconds
Timings drift over the day	`system_clock` was adjusted	Use `steady_clock`
Optimisation made no difference	The measured region was not the bottleneck	Measure first; you cannot fix what you have not located
`std::set` slower than expected	Node-based layout; pointer chasing defeats the cache	Sorted `vector` + `binary_search` when the data is stable
Adding a type means editing a `switch`	Open/Closed violation	Virtual dispatch, as in Chapter 7
A derived class throws "not supported"	Liskov violation	Split the interface — Chapter 7's ISP

13.10 Key Terms

Single Responsibility — a class should have one reason to change; the test is *what would make this change?*

Open/Closed — open for extension, closed for modification: add a class rather than editing a working function.

Liskov Substitution — a derived object must be usable wherever its base is expected, without the caller knowing.

MVC — model (data and rules), view (presentation), controller (input and coordination), kept separate.

`steady_clock` — a monotonic clock suitable for measuring elapsed time.

bottleneck — the part of a program that dominates its running time; found by measurement, not inspection.

optimisation level — a compiler flag such as `-O0` or `-O2` controlling how aggressively code is transformed.

cache locality — the performance benefit of data laid out contiguously in memory.

13.11 Exercises

1. **(Rank, then measure)** Before writing any code, rank these four for searching 50,000 integers with 5,000 lookups: linear `std::find`; sort-then-`binary_search`; `std::set`; `std::unordered_set`. Write your ranking down. Then measure all four at `-O0` and `-O2`, and report where your prediction was wrong and why.

2. **(What would make this change?)** For each class, list every distinct reason it would need to be edited, and say whether it violates SRP.

```
class Logger      { void write(const std::string&); void rotateFiles(); };
class SensorPanel { void poll(); void computeAverages(); void draw(); void saveCsv(); };
class RingBuffer  { bool push(int); bool pop(int&); std::size_t size() const; };
```

3. **(Which principle is violated?)** Name the principle, explain the failure, and give the fix.

```cpp
// (a)
struct Printer { virtual void print() = 0; virtual void scan() = 0; virtual void fax() = 0
struct BasicPrinter : Printer {
    void print() override { /* ok */ }
    void scan() override  { throw std::logic_error("not supported"); }
    void fax() override   { throw std::logic_error("not supported"); }
};

// (b)
void report(const std::vector<Sample>& v, int format) {
    if (format == 0)      { /* csv  */ }
    else if (format == 1) { /* json */ }
    else if (format == 2) { /* xml  */ }
}

// (c)
class Config { void load(); void validate(); void applyToUi(); void emailAdmins(); };
```

4. **(Faded)** Complete the timing harness so it measures honestly.

```cpp
using clk = std::chrono::____________;              // (A) which clock, and why

____________ int sink = 0;                           // (B) stop the optimiser

auto t0 = clk::now();
for (int t : targets) { sink += search(data, t); }
auto t1 = clk::now();

std::cout << std::chrono::duration_cast<std::chrono::____________>(t1 - t0).count()
          << " us\n";                                // (C) the right resolution
```

5. **(Spaced recall — Chapters 7, 9, and 10)** A `std::set` and a sorted `std::vector` are both O(log n) for lookup, and the vector is several times faster. Explain, using Chapter 9's knowledge of how each is laid out. Then: your fix is to replace the `set` with a `vector`, and a suite of Chapter 10 unit tests covers the class that uses it. What should those tests tell you after the change, and what would it mean if one of them now failed?

6. **(Write from scratch)** Take a program with an obvious inefficiency — repeated linear searches over a container that does not change. **Measure it** with `std::chrono` at `-O0` and `-O2`. Apply one algorithmic improvement and one data-structure change, measuring after each, separately. Write a short recommendation naming which change to keep, supported by your numbers, and stating explicitly what you gave up. A recommendation of "keep neither" is acceptable and must be justified the same way.

13.12 Self-Check

1. Select every SOLID principle that was introduced in Chapter 7 rather than this chapter.

   - Single Responsibility

- Interface Segregation
- Dependency Inversion
- Open/Closed

2. What is the test for Single Responsibility?
3. A class implements a five-method interface, and three of its methods throw "not supported". Which principle is violated?
4. Select every true statement about MVC as taught in this chapter.
   - The model knows nothing about how it is displayed
   - The view makes no decisions
   - The controller coordinates input, model, and view
   - The model formats its own output for the console
5. Which is faster: sort-then-`binary_search` compiled at `-O0`, or a linear scan compiled at `-O2`?
6. `std::set::count` and `binary_search` on a sorted vector are both O(log n). Why is the set 3–6× slower?
7. Same 20,000 integers, but only **three** lookups instead of 2,000. Which approach wins?
8. Your `-O2` timings all report near 0 µs. What is the most likely cause?
9. An optimisation is 8% faster and considerably harder to read. What is the professionally correct response?
10. What must a performance recommendation contain to be acceptable?

13.13 Summary

Close the book and answer from memory.

- Give all five SOLID principles, and the question each one asks.
- Which two did you apply in Chapter 7, and where?
- State the four steps for acting on a measurement.
- Why does a sorted `vector` beat a `std::set` at the same complexity?
- What does `-O2` buy, and what can it never buy?
- When is the right answer to a performance investigation "change nothing"?

One chapter remains, and it is not new material. It is the whole course used at once: a system that acquires data, owns its resources, parses bytes, reports failure, and hands work between threads — with every rule from the last thirteen chapters applying at the same time.

Part VII

Integration

Chapter 14

Integration

14.1 What This Chapter Is

Not new material. One program with every chapter in it, and a set of questions about that program which you should be able to answer without looking anything up.

Use it the hard way. **Cover the answer, write yours down, then check.** Reading the questions and nodding at the answers will make everything look familiar and teach you nothing — familiarity is recognition, not memory, and the first place you will discover the difference is the exam.

You should feel *less* prepared at the end of this chapter than at the start. That is the chapter working. The alternative is feeling prepared and being wrong.

14.2 The Capstone Artifact

One class. Nine chapters visible in eleven lines.

```
// A sensor node: reads a register, buffers samples, runs a control loop.
class SensorNode {
    volatile uint8_t* const port_;              // Ch 4 - memory-mapped, const pointer
    RingBuffer<SensorSample, 16> buffer_;       // Ch 8 - fixed capacity, no heap
    std::unique_ptr<Controller> control_;       // Ch 6 - exclusive ownership
public:
    SensorNode(volatile uint8_t* port, std::unique_ptr<Controller> c);
    ~SensorNode();                              // Ch 5 - releases the pin
    void poll();                                // Ch 2 - bit masks
    Summary summarise() const;                  // Ch 9 - algorithms
};
```

Everything below is a question about this class.

14.3 1. Memory — Chapters 3, 5, 6

`buffer_` holds 16 samples. Where do those 16 samples live, and who frees them?

On the **stack** — or wherever the `SensorNode` itself lives — because `RingBuffer` holds a `std::array` **member**. Nobody frees them; they die with the node.

This is why capacity is a template parameter rather than a constructor argument. Chapter 3's rule about avoiding the heap is satisfied **by construction** rather than by discipline: there is no allocation to forget to free, because there is no allocation.

> **`control_` is a `unique_ptr`. What happens if someone copies a `SensorNode`?**

It does not compile. `unique_ptr` is not copyable, and that propagates to the containing class. The rule of zero: the double-free is *uncompilable* rather than merely unlikely, and nobody had to write a copy constructor to make it so.

14.4 2. Ownership and Lifetime — Chapters 5, 6, 7

> **The destructor releases the pin. Name three exit paths it covers.**

A normal return; an early return; and an exception unwinding through. All three, because it is a destructor rather than a cleanup statement — Chapters 5 and 10 together.

> **If `Controller` is a base class with virtual functions, what must it have?**

A **virtual destructor**. Without it, `unique_ptr`'s `delete` runs only the base destructor and the derived part leaks — **with no warning at all**, because the offending `delete` is inside a library header rather than in code anyone wrote.

14.5 3. Types and Bits — Chapters 2, 4

> **Write the line that sets bit 3 of the port without disturbing the others. Then say why `*port_ = (1 << 3);` is wrong.**

```
*port_ |= (1 << 3);      // read-modify-write
```

Assignment sets bit 3 **and clears the other seven**. It happens to look correct whenever the register was already zero, which is exactly why this bug reaches production.

> **Why is `port_` declared `volatile uint8_t* const`?**

The **address** is fixed — a register does not move — so `const` makes repointing it uncompilable. The **value** changes outside the program's control, which is what `volatile` accounts for. Two qualifiers, two different jobs, read right to left.

14.6 4. Templates — Chapter 8

> **`RingBuffer<SensorSample, 16>` and `RingBuffer<int, 8>` — same type or different, and what does that cost?**

Different types, generating separate machine code. The cost is flash per instantiated type. The benefit is a `sizeof` known at compile time and storage that never touches the heap.

That is the trade against Chapter 7: templates spend flash, virtual dispatch spends RAM per object. Which you can afford depends on the device.

14.7 5. Testing — Chapter 10

Name three tests you would write for `poll()`, and one for the exception path.

The buffer fills and then refuses; a wrap drains in the right order; the register is left in the expected state. For the exception path, `REQUIRE_THROWS_AS` on whatever a full or failed read is specified to do.

Why is `summarise() const` easier to test than a version that printed?

Because you can assert the **return value**. A function that prints has to have its output captured and parsed. That is Chapter 13's MVC argument arriving as a practical consequence: a model that formats its own output cannot be tested without a screen.

14.8 6. Concurrency — Chapter 12

Two threads call `poll()`. What breaks?

`buffer_` is shared mutable state with no synchronization — a **data race**. It will often appear to work, which is the property that makes it expensive to find.

Would making `buffer_ volatile` fix it?

No. A volatile counter races exactly as badly as a plain one; Chapter 12 measured it. `volatile` is for hardware. `std::atomic` and mutexes are for threads. They are unrelated tools that share a name.

14.9 7. Performance — Chapter 13

`summarise()` scans 16 samples. Is it worth optimising?

Almost certainly not — and **you do not know until you measure**. Sixteen elements is nothing; if the profile says otherwise, the bottleneck is somewhere else and you would have optimised the wrong thing.

"The measurement does not justify this change" is the right answer more often than not.

14.10 How to Revise

Use the decks. Thirteen of them, and they are the course's vocabulary. Cover the answer, produce it out loud, then check. A card you get right instantly is done; a card you hesitate on goes back in the pile.

Use the old quizzes as tests, not as reading. Redo Q1–Q11 with the answers covered. You have seen them before, which makes them *easier* than the final — so treat a shaky answer as a signal, not as reassurance.

Redo an exercise from scratch, not by reading your old solution. Pick one whose solution you remember and write it again from a blank file. The gap between "I remember this" and "I can produce this" is the whole difference between the two feelings, and it is cheaper to find today than in the exam.

Emphasise Chapters 8–13. The final is weighted toward the post-midterm material.

Do not re-read your notes. It feels productive because the words look familiar, and familiarity is recognition rather than memory. This whole course has been an argument against that instinct; this is the last week it can cost you.

14.11 The Argument, Closed

Fourteen chapters ago the claim was that C++ takes away the runtime and hands you the machine. Everything since has been a consequence of that trade:

- **You free what you allocate** — and then RAII means you do not have to remember.
- **You decide who owns a resource** — and then `unique_ptr` makes the wrong answer uncompilable.
- **You choose compile time or run time** — templates or virtual dispatch, paying in flash or in RAM.
- **You measure instead of guessing**, because there is no runtime to blame.

That is what to take to the embedded courses. The syntax will be there when you need it. The judgement about where things live and who owns them is the part that transfers.

14.12 Exercises

1. **(Read the whole thing)** Take the `SensorNode` declaration above and write, for **every** line, which chapter it comes from and what would go wrong if it were written the obvious naive way instead. Eleven lines, eleven answers.

2. **(Predict every break)** For each change below, say what breaks, when it breaks (compile, link, run time, or silently), and which chapter covers it.

   - `~SensorNode()` is removed entirely.
   - `Controller`'s destructor is not `virtual`.
   - `buffer_` becomes `std::vector<SensorSample>`.
   - `port_` loses its `volatile`.
   - `poll()` uses `*port_ = mask;` instead of `*port_ |= mask;`.
   - Two threads call `poll()` concurrently.

3. **(One question per chapter)** Without looking anything up, write one sentence answering each: what does the linker do; where does a local live; what does `&` mean in a parameter; what does a destructor guarantee; what does `std::move` do; what does a vtable cost; what does a template generate; what does `end()` point at; what does unwinding do to locals; why encode field by field; what does a mutex protect; and what must a performance claim include.

4. **(Which tool, and why)** For each, name the tool and justify it in one sentence: a program crashes on the third record of a file; a total is wrong by a different amount each run; a function is slower than expected; a class leaks memory; a derived destructor never runs; a binary file reads correctly on Mac and wrongly on Windows.

5. **(Write from scratch — the capstone)** Implement `SensorNode` for real. A bit-masked register interface, a fixed-capacity `RingBuffer` member, a `unique_ptr`-owned `Controller` with a virtual destructor, a `summarise()` using `<algorithm>`, a Catch2 suite covering the

buffer's wrap and the exception path, and a `std::chrono` measurement of `summarise()` at `-O0` and `-O2`. Then write half a page: which chapter's rule each design decision comes from, and what you would have to change to run it on a microcontroller with 64 KB of RAM and no exceptions.

14.13 Self-Check

1. `RingBuffer<SensorSample, 16>` is a member of `SensorNode`. Where does its storage live, and who frees it?
2. Why will `SensorNode` not compile if someone tries to copy it, and which rule is responsible?
3. Name the three exit paths a destructor covers that a cleanup statement at the end of a function does not.
4. `Controller` is a polymorphic base held by `unique_ptr`. What must it declare, and what is the symptom if it does not?
5. Why is `*port_ = (1 << 3);` wrong, and why does it often appear to work?
6. In `volatile uint8_t* const port_`, which qualifier stops repointing and which accounts for the hardware?
7. Two threads call `poll()`. What is the defect called, and would `volatile` fix it?
8. `summarise()` scans 16 elements. Should you optimise it?
9. Give the two costs — one in RAM, one in flash — of virtual dispatch and of templates respectively.
10. Name three revision activities that are retrieval, and one that is not.

14.14 Summary

Close the book and answer from memory. If you can do all of these, you are ready.

- Where does each member of `SensorNode` live, and who frees it?
- Give three ways the compiler can be made to reject a bug instead of you having to remember it.
- Name the four rules that fire, in order, when a `unique_ptr<Base>` owning a derived object goes out of scope.
- State one thing from this course that is true of `volatile` and one that is not.
- What is the difference between recognition and retrieval, and which one is studying?

Appendices

Appendix A

Self-Check Answers

Answers to the self-check questions at the end of every chapter, with a short reason for each. The reason matters more than the answer: a question you got right for the wrong reason is one you will get wrong next time.

Answer from memory first, then look. Checking an answer you never committed to teaches you almost nothing — the moment of finding out you were wrong is the moment the correction sticks, and it only happens if you were willing to be wrong first.

Worked solutions to the exercises are not here. Your instructor releases those a couple of days after an exercise set is assigned, so you get a real run at the problems before the answers are available. Use that gap — a problem you struggled with and then saw solved is worth several you read the solution to first.

A.1 Chapter 1 — The Toolchain and the Build

1. Preprocess, compile, assemble, link. Each consumes the previous stage's output: source text, then a translation unit, then assembly, then object files.

2. Assemble. It produces machine code in a `.o` file — machine code that is real but not yet runnable, because nothing has resolved its references to other files.

3. Compiles and assembles, but does not link. `-c` stops the build one stage early and leaves you holding `main.o`.

4. The **link** stage. Look for a missing `.cpp` in your build — a source file you forgot to pass on the link line or to list in `add_executable`. Do not look for a typo on the calling line: the compiler already accepted that line, which is precisely what the declaration promised it.

5. One source file after the preprocessor has run, with every `#include` and macro fully expanded. It is what the compiler actually translates — usually tens of thousands of lines even for a program you wrote ten lines of.

6. False. `std::cout` never adds a newline. It is the equivalent of `Console.Write`; the line breaks only where you write `\n` yourself.

7. False. It runs perfectly well. The source is a compile-time artifact and the executable is the run-time one — native machine code with no memory of the file it came from.

8. `-E`. It runs the preprocessor and stops, so you can see exactly what `#include` pasted in.

9. C# has a **runtime** — the CLR, which JIT-compiles the program and runs a garbage collector that frees memory for you. C++ has none of that: it compiles straight to native machine code, so every allocation you make is one you must release yourself.

10. Theirs came from the **compiler**, because it names a file, line, and column in their source. Yours came from the **linker**, because it is prefixed with the linker's own name and names no line of your source at all. Theirs is a problem with something they wrote; yours is a problem with what was, or was not, included in the build.

A.2 Chapter 2 — Types, Bits, and the Preprocessor

1. It runs first, before the compiler sees anything, and it understands no C++ at all. It performs text substitution driven by lines beginning with `#` — it does not know what a type, a function, or a scope is.

2. `-E`. It stops the build after preprocessing so you can read exactly what `#include` pasted in and what your macros became.

3. Any two of: it has a type; it obeys scope; it survives into the debugger; errors point at the line you wrote rather than at an expansion. `#define` has none of these, because by compile time it no longer exists.

4. It expands to `3 + 1*2`, which evaluates to **5**, not 8. Text substitution puts the `*2` directly against the `1`, and multiplication binds tighter than addition.

5. False. That is C#. C++ guarantees only that `int` is at least 16 bits; the actual size is implementation-defined. In practice it is 4 bytes nearly everywhere you will meet, but `long` is 4 bytes on Windows and 8 on Linux — that is where the lack of a guarantee bites first.

6. `uint8_t` — `u` for unsigned, 8 for the bit count, `_t` for type.

7. `a + b` is **300**, and `sizeof(a + b)` is **4**. Both operands are promoted to `int` before the addition, so the arithmetic happens in 32 bits and 300 fits easily. The wrap to 44 happens only if you store the result back into a `uint8_t`.

8. `reg &= ~(1 << 4);` — the `~` is the whole answer.

9. Because it keeps **only** bit 4 and clears all the others — the exact opposite of the intent. `~(1 << 4)` is a mask of all ones except bit 4, so ANDing with it preserves everything you did not name.

10. `1 << 6` produces a value with a single 1 in bit position 6 (decimal 64). Write it that way because it says *bit six*, which is what you actually mean, and because it keeps working when the position is a variable or a named constant.

11. Reading the current value, changing part of it, and writing the whole thing back. `|=`, `&=`, and `^=` each perform one. It is required because you cannot address a single bit — the smallest thing you can write is the whole register.

12. `&` binds looser than `==`, so it means `reg & (0x08 == 0)`, which is `reg & 0`, which is always `0`. The compiler tells you, if you let it: `-Wall` emits `warning: & has lower precedence than ==`. Parenthesise: `(reg & 0x08) == 0`.

13. `std::cout` treats `uint8_t` as a character type and printed the glyph for character code 65. Fix it with `static_cast<int>(value)`.

A.3 Chapter 3 — The Stack and the Heap

1. The compiler frees it, automatically, at the closing brace of the scope it was declared in. Not eventually and not at a convenient moment — at the brace.

2. `c`, then `b`, then `a` — the reverse of construction order. That order is what makes it safe for a later object to depend on an earlier one: whatever you were built on is still alive when you are torn down.

3. False. A bare `{ }` block is a scope like any other, and an object declared inside it is destroyed at its closing brace. This is the part of the Tracer example most people get wrong.

4. The stack, like any other local. A pointer is just a variable holding an address; it lives wherever you declared it. What it *points at* is on the heap — those are two different things, and confusing them is the central error of this chapter.

5. `delete[] p;`. `new[]` pairs with `delete[]`; mixing the forms is undefined behaviour, and it will often appear to work, which is worse than crashing.

6. Unchanged — it still holds the same address it did before. `delete` returns the memory but does not touch your pointer. That is what makes use-after-free so hard to spot: the pointer looks perfectly fine.

7. Any two of: there is very little RAM; fragmentation eventually fails a large request even when total free space is sufficient; `new` has unpredictable timing, which real-time code cannot afford; there is nowhere to report an allocation failure.

8. It includes debug information — line numbers and variable names — in the executable, so the debugger can show your source rather than machine code. LLDB needs it.

9. `frame #0` is the function currently executing, and the numbers count *outward* toward the caller. The bottom of the list is `main`.

10. Three of the four exits leak. The defect is not a missing `delete[]` — there is one; it guards only the last path. The fix that cannot be forgotten is not to own the memory by hand: return a `std::vector`, or use the `unique_ptr` of Chapter 6, so that release happens in a destructor rather than in a statement someone has to remember to write.

11. Returning an `int` by value copies the value out before the frame is popped, so the caller gets a copy of something. Returning a pointer to a local array copies out the *address* — and the frame it points into is destroyed the instant the function returns, so the caller holds the address of memory that now belongs to the next call.

A.4 Chapter 4 — Pointers, References, and `const`

1. A variable that holds an address. That is the entire definition, and nearly every pointer bug is a failure to keep it in mind.

2. `p` holds the address of `a`. `*p` gives you the object living at that address — `a` itself, not a copy of it.

3. `sizeof(arr)` is 16, four ints. `sizeof(p)` is 8, one address. They differ because the array and the pointer are different things: one is the storage, the other is a way to reach it.

4. The element two positions along — not the address plus two bytes. For an `int*` it reads 8 bytes past `p`, because pointer arithmetic is scaled by `sizeof(int)`. It is the same thing as `p[2]`.

5. `*(p + 1)` is the *next element*. `*p + 1` is *this element, plus one*. Dereference binds tighter than `+`, so the parentheses are the whole difference.

6. A pointer. A reference must be bound to an object when it is created and can never be null — which is exactly why you choose a pointer when an argument is genuinely optional.

7. False, and this is the trap. `f(v)` looks identical whether the parameter is `int` or `int&`. You have to read the signature. C# tells you at the call site with `ref`/`out`; C++ does not.

8. `const int* p` — "p is a pointer to const int": the value is protected, the pointer can move. `int* const q` — "q is a const pointer to int": the pointer is protected, the value can change. Read right to left from the variable name.

9. Neither line fails. `p` points to a `const int`, so the *value* is protected, but `p` itself is an ordinary pointer and may be moved wherever you like. The line that would fail is `*p = 5;`.

10. `const T&`. By value would copy the whole struct for no reason; the reference avoids the copy, and the `const` promises the caller you will not modify their object — a promise the compiler enforces.

11. It tells the compiler that the location can change outside the program's control, so reads and writes to it must not be optimised away or cached. It does **not** make anything atomic, and it does not order operations between threads — it is not a synchronization tool.

12. The **address** is fixed and the **value** can change. That is the right way round because a hardware register sits at a location the chip designer chose — moving the pointer is always a bug, so `const` makes it uncompilable — while the hardware itself writes to the value whenever it likes, which is what `volatile` accounts for.

A.5 Chapter 5 — Classes, Structs, and RAII

1. Default access. A `struct`'s members are public unless you say otherwise; a `class`'s are private. Everything else — constructors, destructors, inheritance, methods — is identical.

2. False, and this is the C# habit most worth unlearning. That is the C# distinction. In C++, where an object lives is decided by how you declare it, not by which keyword defined its type: a `class` can be a stack local, a `struct` can be `new`ed.

3. Member construction (in declaration order), then the constructor body; later, the destructor body, then member destruction (in reverse declaration order). Members exist before your constructor body runs and are still alive during your destructor body — which is what makes both bodies safe to write.

4. Because the body runs *after* every member has already been initialised, and assignment to a `const` member is not allowed. The member init list is the only place a `const` member can be given

its value. The same applies to reference members.

5. Declaration order — regardless of the order you write them in the init list. Writing them out of order earns a `-Wreorder-ctor` warning, and clang prints the correct order underneath it.

6. `this` is a pointer to the object the member function was called on. `*this` is that object, which is what you return when you want to chain calls.

7. The out-of-class *definition* — something like `int Counter::instances_ = 0;` at file scope. The **linker** reported it, which you can tell because the message is prefixed with the linker's name and names no line of your source. The declaration inside the class was a promise; nothing kept it.

8. Tie a resource's lifetime to an object's: take it in the constructor, give it back in the destructor, and scope guarantees the rest.

9. Because it is correct only when every caller remembers it, on every path. A function with an early `return` never reaches the `release()` you wrote at the bottom — and neither does one that throws. A `release()` method documents an obligation; a destructor removes it.

10. A `uint32_t` must start at an address divisible by 4, so the one-byte `a` forces three bytes of padding before `b`, and the struct is rounded up so the next array element also aligns. Put the largest field first — `uint32_t b; uint8_t a; uint8_t c;` — and it is 8.

11. When the struct must match a layout defined outside your program: a hardware register block, a binary file format, or bytes on a wire. It costs performance, because unaligned access is slower on x86 and faults outright on some architectures. Pack where an external format demands it; reorder fields everywhere else.

12. Ownership. RAII settles *when* a resource is released, but not *how many objects think they own it*. Copy such an object and two destructors will release the same thing.

A.6 Chapter 6 — Smart Pointers, Copy Semantics, and Custom Allocation

1. The same size: 8 bytes on a 64-bit machine, exactly like `int*`. The safety costs no memory, because it is all enforced at compile time. (The reference counting people are thinking of belongs to `shared_ptr`, which is 16.)

2. Nothing at run time. It is a cast, and the type on the receiving end decides what the cast means; for `unique_ptr` it means ownership transfers and the source is left empty.

3. Because a copy would produce two owners of one address, and both destructors would `delete` it. The type deletes its copy constructor precisely so that the double-free is impossible to write — the error is the library refusing to let you make the mistake.

4. A copy constructor, a copy assignment operator, and a move constructor. Declaring a destructor suppresses the automatic move operations in most cases, but the copies are still generated, which is exactly how the double-free gets written on your behalf. There is no such thing as a "default constructor taking your destructor's arguments".

5. Design classes so you write **none** of the five special functions, by holding resources in types that already manage themselves — `unique_ptr`, `vector`, `string`. It is preferred because the compiler-

generated operations are then correct for free: implementing copying by hand is roughly twenty lines that have to stay correct under every future edit.

6. Copied **no**, moved **yes**. Non-copyable propagates from the member automatically, so the double-free becomes uncompilable rather than merely unlikely. Movable also propagates, which is what lets you still return such an object from a function.

7. True: the object is destroyed when the last `shared_ptr` goes away; it is twice the size of a `unique_ptr`; and every copy and destruction adjusts a reference count. **False:** a cycle is the exception — neither count ever reaches zero, so neither destructor runs and the memory leaks, through the mechanism meant to prevent leaks.

8. No. Take `const T&`. Passing a `shared_ptr` by value copies it — an atomic increment and decrement per call — to participate in ownership the function does not need. Beyond the per-call cost, it misrepresents the design: ownership belongs in a signature only when the function actually shares it.

9. Nothing. Placement `new` constructs an object in memory you already have. That is why it must be paired with an explicit destructor call (`p->~T()`) rather than `delete` — there was no allocation to undo.

10. From Chapter 3: allocation time is unpredictable; the heap fragments; the memory cost is not knowable in advance; and exhaustion fails in a place you cannot handle. A fixed pool answers all four — allocation is a constant-time scan of a small flag array with no free-list search; every slot is identical so nothing fragments; the total is `sizeof(T) * N` and visible in `sizeof` at compile time; and exhaustion returns `nullptr` at the call site, where you can decide what to do about it.

A.7 Chapter 7 — Polymorphism and Abstract Base Classes

1. From 4 bytes to 16 — an 8-byte vtable pointer plus alignment padding. The class gained no member; it gained the *ability to dispatch*, and that ability is stored in every object.

2. `~Base()` only. `~Derived()` never runs, and anything it owned leaks. Destruction is polymorphic only if you write `virtual`. What makes this surprising is that the ordinary function call above it *would* dispatch correctly — dispatch and destruction are separate decisions. Clang warns: `-Wdelete-non-abstract-non-virtual-dtor`.

3. False — this is **slicing**. Only the `Sensor` part is copied, including throwing away the vtable pointer, so the base version runs. Polymorphism requires indirection: reference, pointer, or smart pointer, never by value. The call site looks identical either way, so you have to read the signature.

4. At least one pure virtual function — a `virtual` member declared `= 0` with no body. Trying to instantiate one gives `error: variable type 'S' is an abstract class`, followed by a `note:` line naming the specific unimplemented function. Read the note; it is the more useful half.

5. Because without it, a mismatch is silent. Misspell the name, or get the `const` wrong, and you have quietly declared a *new* function rather than overriding anything — the base's version still runs, or the pure virtual stays unimplemented and the error surfaces somewhere else entirely. `override` turns a silent behavioural bug into a compile error at the line you got wrong.

6. Do you know the set of types at compile time? If the types are decided at run time — a container of mixed sensors chosen by a config file — you need runtime dispatch. If they are known

when the program is built — a buffer holding `int` here and `double` there — templates do the job with no vtable and no indirection.

7. Interface Segregation: `Sensor` declares `readMilliVolts` and `name` and nothing else, so no derived class is forced to implement anything it has no use for. **Dependency Inversion:** the line `for (const auto& s : sensors)` uses `Sensor`, not `Thermocouple` — the loop depends on the abstraction, so adding a sensor type changes no existing code.

8. Because the vtable pointer is stored in **every instance**, while the table itself exists once per class. Ten thousand objects cost ten thousand pointers — 80 KB on a 64-bit machine. On a desktop that is nothing; on a microcontroller with 64 KB of RAM it is the entire budget, which is why C++ makes you ask for dispatch rather than giving it to you by default.

A.8 Chapter 8 — Templates and Generic Programming

1. When they are resolved. C# generics are resolved at run time by the CLR, and one generic method exists in the assembly. C++ templates are resolved at compile time, and a separate concrete function is generated for every type you use. Everything else — the error timing, the code size, the header rule — follows from that one difference.

2. The generation of a concrete function or class from a template, for a particular set of template arguments. The compiler does not compile the template itself; it compiles what the template generates.

3. Read the symbol table: `nm -C` on the object file or binary lists each generated function with its template arguments demangled. Counting them is the honest way to check a code-size worry, rather than guessing.

4. True: a separate copy of the code is generated per type used; there is no run-time indirection, unlike a virtual call; and the binary can grow substantially with many instantiated types. **False:** nothing is added to the objects. That is the *virtual* cost — a vtable pointer per object. Templates cost flash, not RAM, which is exactly the trade against Chapter 7.

5. Do you know the set of types at compile time? If yes, templates — no indirection, cost paid in binary size. If the types are decided while the program runs, you need virtual dispatch — cost paid in RAM per object and an indirect call.

6. Because there is one `T` and two arguments of different types, so deduction cannot choose: `deduced conflicting types for parameter 'T' ('int' vs. 'double')`. It is not that `int` and `double` cannot be compared — they compare fine. The template as written requires both arguments to be the *same* type. Fix it with `largest<double>(3, 9.5)`, or by giving the template two type parameters.

7. Almost nothing about how badly you failed. It tells you how **deep the library call stack was** when the mistake surfaced — `std::sort` instantiates a great deal of machinery, and each layer reports. Of those 308 lines, 44 were `in instantiation of` notes and exactly **one** named a file the student wrote. Volume is a property of the library, not a measure of the error.

8. The last `in instantiation of ... requested here` note that names **your** file. The first error line says *what* is wrong, often deep inside a header you cannot change; that note says which line of your code asked for it, and that is the line you can actually edit.

9. Because instantiation needs the **body**. When the compiler builds `main.cpp` and finds a call to `largest<int>`, it must generate that function then and there — and it can only see the current translation unit. If the definition is in another `.cpp`, nothing ever generates it. The **linker** reports the failure, `undefined symbol: int largest<int>(int, int)`, and names no line of your source, because there is no line to name.

10. True: it knows its own length inside a function; and its size is part of its type. **False:** it is not larger than `int[4]` — `sizeof(std::array<int,4>)` is 16, exactly four ints, with no stored size (`N` is a template argument, known at compile time, so it costs no storage). And it does **not** decay to a pointer, which is the main reason to prefer it.

A.9 Chapter 9 — The STL: Containers, Iterators, and Algorithms

1. Because the `vector` object itself holds only three pointers — begin, end, and capacity-end — and the elements live in a separate heap block it points at. Pushing more elements changes what those pointers address, not how many of them there are. A `vector` of a million ints still has `sizeof` 24.

2. That `std::array` **is** the storage rather than pointing at it: 16 bytes is exactly four `int`s, with nothing left over. The size `N` is a template argument, known at compile time, so it costs no storage at all. That is the contrast with `vector` in one line.

3. *On the workstation, reach for* `vector` *first; on the device, reach for* `array` *first.* If the size is known at compile time, `array` is strictly better — no allocation, no indirection, and the size is part of the type. `vector` is not banned, it is charged for: it allocates, and it can reallocate at a moment you did not choose.

4. An iterator **one past** the last element. It is not an element and must never be dereferenced — it is a marker for "stop here", which is why loops compare `!= end()`.

5. True: `push_back` may reallocate and invalidate every iterator into the vector; a pointer to an element is invalidated by the same reallocation; and after invalidation, dereferencing usually still appears to work. **False:** `std::array` has a fixed size — you cannot add an element to one, so there is no reallocation to invalidate anything.

That third true statement is the dangerous one. A dangling read into a freed vector buffer typically returns a plausible number rather than crashing.

6. False. `v[i]` does not check and reading out of range is undefined behaviour; `v.at(i)` throws `std::out_of_range`. Use `at()` when the index came from outside your program.

7. `std::accumulate`, and it is in `<numeric>` — not `<algorithm>`, which is where most people look first.

8. Readability, not speed. They frequently compile to the same machine code as your loop. The gain is that the name states the intent: a reader of `std::accumulate` knows the result immediately, whereas a hand-written loop has to be checked for off-by-one bounds before it can be trusted.

9. Because `count` was captured **by value**, and a by-value capture is `const` inside the lambda by default — so `++count` is modifying a `const` copy. Capture by reference, `[&count]`, if the referent will outlive the lambda; otherwise you have Chapter 4's dangling reference again.

10. True: it prints `0`; it inserts an entry for `"missing"`; and `m.size()` is afterwards 1. **False:** it does not throw — `operator[]` on a `map` default-constructs a value for an absent key and returns a reference to it. That is why a line that reads like a lookup can silently grow the map, and why `find` or `contains` is the right tool when you only want to look.

A.10 Chapter 10 — Error Handling and Unit Testing

1. It abandons the function immediately — nothing after the `throw` in that function runs — and begins unwinding the stack, destroying every fully-constructed local in reverse order, until a matching `catch` is found.

2. `std::out_of_range` is a `std::logic_error`. That classification is a claim about whose fault it is: a logic error means the caller broke a precondition, so it is a **bug to fix in code** rather than a condition to handle at run time. Compare `std::runtime_error`, which is the world misbehaving — a missing file, an unresponsive device — and is exactly what you should be catching and recovering from.

3. Because the destructor already is one. Stack unwinding destroys every local on the way out, so an RAII object releases its resource on the exceptional path without anyone writing release code. A `finally` block exists to run cleanup you wrote as a *statement*; C++ has no such statement to run, because the cleanup lives in a destructor.

4. Because catching by value **slices**: it copies the base part of the exception and discards everything the derived type added — which is usually the information you threw the exception to convey. Catch a `SensorFault` by value and `id()` goes with it. Chapter 7's slicing, in a place where it costs you the diagnosis.

5. `SensorFault` must come first. Handlers are tried in order, and `SensorFault` derives from `std::exception`, so a `std::exception` handler listed first would match everything and make the specific handler unreachable. Clang warns about this.

6. True: a `noexcept` function that throws calls `std::terminate`; destructors are `noexcept` by default; and a destructor that throws during unwinding terminates the program — there is no sensible answer to which exception should win. **False:** `noexcept` is not a general speed switch. It can enable some optimisations, notably letting containers move rather than copy, but "faster in all cases" is not a claim to make.

7. Because exceptions cost **binary size** — the unwinding tables — and their timing is **not bounded**, which hard-real-time code cannot accept. What replaces them is a **returned error value**: an enum or status code the caller must check. That is the right technique in that context, not a lesser one; the competency is being able to say which mechanism applies where.

8. A **unit** test exercises one class or function in isolation — `RingBuffer::push` refuses when full. An **integration** test exercises several pieces together — the parser feeds the queue and the consumer sees every record. Start with unit tests, because when one fails you already know where the bug is.

9. `REQUIRE_THROWS_AS(expression, ExceptionType)`. Note that the type must match the actual exception's type or one of its bases — `std::out_of_range` is a `logic_error`, not a `runtime_error`, and asserting the wrong branch fails.

10. It shows the **actual values** of the expression that failed. Catch2 re-runs the expression and prints, for example, `10 == 99`, so you learn what the value really was without adding a print statement and rebuilding. Together with the test-case name and the two line numbers, that is usually enough to diagnose the failure without touching the code.

A.11 Chapter 11 — File I/O and Streams

1. False, and this is the most consequential false belief in the chapter. The stream enters a failed state silently: `is_open()` and `good()` are both 0, no exception is thrown, and every subsequent read does nothing. A read loop simply runs zero times, so the program reports a successful empty result. Always `if (!in) { ... }` immediately after opening.

2. Because without it the stream performs newline translation: a byte that happens to equal `0x0A` is rewritten on the way out. In text that is the intended behaviour; in binary data it silently corrupts any record containing that byte value. The file will look fine on a Mac and be wrong on Windows, which is the worst way to find out.

3. Because 3 is not a multiple of 4, so **no C++ struct can place a `float` there** — the compiler will insert padding to align it. That is why the wire record is 11 bytes and the struct is 12, and why the record must be encoded field by field rather than by writing the struct's memory. On strict-alignment processors, a 4-byte load from an unaligned address faults outright.

4. Shift and OR, most significant byte first:

```
uint32_t v = (uint32_t(b[0]) << 24) | (uint32_t(b[1]) << 16)
           | (uint32_t(b[2]) <<  8) |  uint32_t(b[3]);
```

5. Because integral promotion turns the `uint8_t` into a **signed `int`** before the shift, so a byte with its high bit set shifts into the sign bit and the result is negative. The bug hides when you print it in hex — the bit pattern looks right — and appears the moment the value is widened or used arithmetically: assigned to a `uint64_t` it sign-extends to `FFFFFFFFDEADBEEF` instead of `00000000DEADBEEF`, and divided as a `double` it yields `-570425.3` instead of `3724542.0`.

6. Assemble the four bytes into a `uint32_t` in host order first, then convert the *bits* with `std::bit_cast<float>`. Never byte-swap a `float` directly — shifting a floating-point value is not defined, and reinterpreting through a pointer breaks strict aliasing.

7. True: `#pragma pack` is a compiler extension, not standard C++; it writes host byte order; and reading the bytes back through a different pointer type is undefined behaviour. **False:** it does **not** always fault on x86-64 — Clang and GCC access packed members byte-wise, so it works on your machine. That is exactly what makes it dangerous, and overstating the charge loses the argument with anyone who has tried it.

8. Because `eof()` is only set **after** a read has already failed. The loop tests the flag before the failing read has happened, so the body runs one extra time on stale or partial data — typically processing the last record twice. Loop on the operation instead: `while (in.read(buf, 11)) { ... }`. Same rule for `>>` and `getline`.

9. It means the file ended **mid-record**: the last read pulled 3 bytes when 11 were required, so the file is truncated or corrupt. The program must **report it**, not ignore it and not treat the partial data as a record. A clean file leaves `gcount()` at 0 after the loop.

10. Because a decode that writes fields into the caller's object as it goes will leave that object **half-updated** if it fails partway — some fields new, some old, and no way for the caller to tell. Filling a local temporary and assigning only on success means a failed decode leaves the caller's object exactly as it was. That is the same all-or-nothing guarantee Chapter 10 called exception safety, achieved without exceptions.

A.12 Chapter 12 — Concurrency and Multithreading

1. You must `join()` it (wait for it) or `detach()` it (abandon it) before its destructor runs. Do neither and the destructor calls `std::terminate` — the program aborts. That is deliberate: the alternative, silently detaching, would leave a thread running over dead locals.

2. A number **less than 200000, and a different number each run** — around 100000 in practice, because roughly half the updates are lost. `++counter` is read, add, write; two threads interleave those steps and overwrite each other's results.

3. No. A race is undefined behaviour whose appearance depends on timing, and timing that happens to work on your machine, at your load, with your loop count, proves nothing about another machine or another day. Worse, a smaller loop count often prints the right answer every time. Correctness here comes from reasoning about the code, not from repeated runs.

4. `std::lock_guard` is **RAII for a mutex**: it locks in its constructor and unlocks in its destructor. Prefer it because the unlock then happens on **every** exit path — the early return added six months from now, and the exception you did not expect. It is Chapter 5's argument with a different resource.

5. Because of what each failure costs. A missed `delete` leaks memory: the program keeps running, and on a workstation you may never notice. A missed `unlock` **stops the program forever** — every other thread that wants the mutex waits on a holder that will never release it, and there is no error message, no crash, and no output to diagnose from.

6. Because of **spurious wakeups**: a `wait` may return without anyone having notified it. With a predicate, `wait` re-checks the condition each time it wakes and goes back to sleep if it is still false. Without one, you must write that loop yourself and most people forget, producing a consumer that proceeds when there is nothing to consume.

7. True: the `done` flag must be protected by the same mutex as the queue; `wait` releases the mutex while it sleeps; and a `lock_guard` cannot be used with `wait` — `wait` must be able to unlock, so it requires a `unique_lock`. **False:** the consumer should **not** exit as soon as `done` is true. Production finishing does not mean the queue is empty; the exit condition is `q.empty() && done`, and getting this wrong silently loses the last few records.

8. `notify_all`. `done` is a terminal condition that *every* waiting consumer needs to learn. `notify_one` wakes a single waiter and leaves the rest asleep forever, waiting for a notification that will never come.

9. Deadlock is two or more threads each holding a lock that another needs, so none can proceed. Nothing crashes and nothing is reported — the program simply stops, which is why you diagnose it by attaching a debugger and reading the stacks. The discipline that prevents it is to **acquire multiple locks in the same global order everywhere**; if every thread takes A before B, the cycle cannot form.

10. True: it prevents the compiler optimising away accesses, and it is the right tool for memory-mapped hardware. **False:** it does not make `++v` atomic — that is still read-add-write and still races — and it does not order operations between threads. The belief that it does is inherited from Java and C#, where `volatile` really does carry threading semantics. In C++ it does not.

A.13 Chapter 13 — Architecture and Performance

1. Interface Segregation and **Dependency Inversion** came from Chapter 7 — you applied both when you wrote an abstract `Sensor` with two small pure virtual functions and a loop that depended on the abstraction. Single Responsibility, Open/Closed, and Liskov Substitution are this chapter's.

2. What would make this change? List the reasons the class would need editing; if they come from different directions — a new sensor, a new storage format, a UI redesign — split it. The test is deliberately *not* "does it do one thing", because anything can be described as one thing, which makes that version unfalsifiable.

3. Interface Segregation. The interface is too fat: it forces implementers to supply functions they have no use for, and the only way to comply is to fail at run time. Split it into smaller interfaces so a class can implement exactly what it supports. (This is also a **Liskov** violation once such an object is used through the base — accept that as a second answer if it is argued.)

4. True: the model knows nothing about how it is displayed; the view makes no decisions; the controller coordinates input, model, and view. **False:** the model does not format its own output — the moment it does, it cannot be tested without a screen, which is the main thing the separation buys.

5. Sort-then-`binary_search` at `-O0`, by roughly four times. The unoptimised good algorithm beats the optimised bad one. You cannot compile your way out of a bad algorithm.

6. Because complexity describes how cost **grows**, not what it costs. `std::set` is a node-based tree: every element is a separate heap allocation somewhere else in memory, so each lookup chases pointers around the heap and misses the cache. A sorted `std::vector` is one contiguous block, which the CPU's prefetcher and cache handle far better. Same big-O, very different constant.

7. The linear scan. Sorting has an up-front cost that is repaid over many lookups; with only three lookups you never recover it. The right data structure is a property of the workload, not of the data.

8. The optimiser deleted the work. If the results of your loop are never used, `-O2` is entitled to remove the whole computation. Accumulate the results into a `volatile` sink so the work cannot be discarded.

9. Reject it, unless a measurement shows that this code is on a path where 8% matters. An optimisation that costs readability and buys a few percent is a bad trade, and being able to say *"the measurement does not justify this change"* is as much a competency as making the change.

10. Numbers. Specifically: what was measured, on what data, at which optimisation level, before and after **one** change — plus a statement of what was given up (readability, flexibility, memory). A recommendation without figures is a preference. "Keep neither" is a perfectly acceptable recommendation and needs the same evidence.

A.14 Chapter 14 — Integration

1. Wherever the `SensorNode` itself lives — on the stack, or in static storage — because `RingBuffer` holds a `std::array<T, N>` **member** rather than a pointer to heap storage. Nobody frees it; it dies with the node. That is Chapter 3's avoid-the-heap rule satisfied by construction rather than by discipline, and it is why capacity is a template parameter.

2. Because `control_` is a `std::unique_ptr`, which is not copyable, and non-copyability **propagates to the containing class**. The rule of zero: nobody wrote a copy constructor, and the double-free became uncompilable rather than merely unlikely.

3. A normal return, an early return, and an exception unwinding through. The third is the one a cleanup statement can never cover, and it is why C++ needs no `finally`.

4. A `virtual` **destructor**. Without it, `unique_ptr`'s `delete` runs only `~Controller()` and the derived part leaks. The symptom is the worst kind: **no warning at all**, because the offending `delete` sits inside a library header rather than in code anyone wrote.

5. Because assignment sets bit 3 **and clears the other seven**, where the requirement was to change one bit and leave the rest alone. It appears to work whenever the register happened to be zero already — so it passes casual testing and fails once another subsystem owns a bit. The correct form is the read-modify-write `*port_ |= (1 << 3);`.

6. Read right to left: `port_` is a `const` pointer to a `volatile uint8_t`. The `const` applies to the pointer and stops it being repointed, which is right because a register's address is fixed. The `volatile` applies to the value and tells the compiler the byte can change without the program touching it, so reads must not be cached or optimised away.

7. A **data race** on `buffer_` — shared mutable state with no synchronization, which is undefined behaviour. `volatile` **would not fix it.** `volatile` prevents the compiler optimising accesses away; it does not make operations atomic, order them between threads, or publish one thread's writes to another. Use `std::atomic` or a mutex.

8. No — and more precisely, you do not know until you measure. Sixteen elements is nothing, and if the program is genuinely slow the bottleneck is somewhere you have not looked. Optimising an unmeasured function is how you spend a day making no difference.

9. Virtual dispatch costs RAM: one vtable pointer in every object, 8 bytes on a 64-bit machine, plus an indirect call that usually cannot be inlined. **Templates cost flash**: a separate copy of the generated code for every type instantiated, with no indirection at all. Neither is free; which you can afford depends on the device.

10. Retrieval: covering a flashcard's answer and producing it before checking; redoing an old quiz with the answers hidden; rewriting an exercise from a blank file. **Not retrieval:** re-reading your notes — it feels productive because the words look familiar, and familiarity is recognition rather than memory.

Appendix B

Glossary

Every key term from the book, alphabetically. The chapter number after each entry is where the term is introduced and explained in context — the definition here is a reminder, not a substitute for that chapter.

`-g` — the compiler flag that includes debug information so the debugger can show your source. *(Ch 3)*

`<cstdint>` — the header declaring the fixed-width integer types. *(Ch 2)*

abstract base class — a class with at least one pure virtual function; cannot be instantiated. *(Ch 7)*

`std::accumulate` — totals a range; lives in `<numeric>`. *(Ch 9)*

address-of (`&`) — the operator yielding the address of an object. Unrelated to `&` in a reference declaration. *(Ch 4)*

alias — another name for an existing object. What a reference is. *(Ch 4)*

alignment — the requirement that an object of a given type start at an address divisible by some number. *(Ch 5)*

array decay — the automatic conversion of an array to a pointer to its first element, losing the length. *(Ch 4)*

`std::array<T, N>` — a fixed-size array that keeps its length, does not decay to a pointer, and costs no more than a C array. *(Ch 8)*

assemble — the build stage that turns assembly text into machine code inside an object file. *(Ch 1)*

`std::atomic` — a type whose operations cannot be interleaved; protects one variable, not an operation across several. *(Ch 12)*

automatic storage — the stack. Storage the compiler allocates and frees for you, bound to scope. *(Ch 3)*

big-endian — most significant byte first; the conventional on-disk and network order. *(Ch 11)*

`std::bit_cast` — reinterprets an object's bits as another type of the same size, without undefined behaviour. *(Ch 11)*

bottleneck — the part of a program that dominates its running time; found by measurement, not inspection. *(Ch 13)*

`bt` — the LLDB command that prints the call stack, innermost frame first. *(Ch 3)*

buffering — a stream accumulates output and writes it in blocks rather than per operation. *(Ch 11)*

cache locality — the performance benefit of data laid out contiguously in memory. *(Ch 13)*

capture by value / by reference — `[x]` copies; `[&x]` refers, and dangles if the referent dies first. *(Ch 9)*

`clang++` — the C++ compiler used in this course, from the LLVM project. *(Ch 1)*

class template — a template that generates classes. *(Ch 8)*

CMake — a build system: you describe the target in `CMakeLists.txt` and CMake generates the build commands. *(Ch 1)*

code bloat — the growth in binary size caused by generating a separate copy per instantiated type. *(Ch 8)*

compile time — everything that happens while the program is being built, before an executable exists. *(Ch 1)*

compile-time polymorphism — dispatch decided while the program is built, via templates. *(Ch 7)*

`std::condition_variable` — lets a thread sleep until another signals that a condition may have become true. *(Ch 12)*

`const T&` — a reference that cannot be written through. The default for passing anything larger than a machine word. *(Ch 4)*

`const T*` — a pointer to something that cannot be modified through it. The pointer may be moved. *(Ch 4)*

`constexpr` — a value (or function) computable at compile time. The default way to name a constant here. *(Ch 2)*

constructor — the special member function that runs when an object is created. *(Ch 5)*

container — a class template holding a collection of objects. *(Ch 9)*

`std::cout` — the standard output stream. Prints with `<<`, and never adds a newline. *(Ch 1)*

dangling pointer — a pointer to memory that has been freed or has gone out of scope. *(Ch 3)*

data race — two threads accessing the same location concurrently with at least one write, and no synchronization. Undefined behaviour. *(Ch 12)*

deadlock — two or more threads each holding a lock the other needs; the program stops without error. *(Ch 12)*

declaration — a promise to the compiler that a name exists somewhere, with a given type or signature. Distinct from a *definition*, which supplies the body. *(Ch 1)*

`#define` — a preprocessor directive that substitutes text. Reserved in this course for include guards and conditional compilation. *(Ch 2)*

`delete` / `delete[]` — return heap memory. The bracketed form pairs with `new[]`. *(Ch 3)*

Dependency Inversion — depend on abstractions rather than concrete types. *(Ch 7)*

dereference (`*`) — the operator that reaches the object a pointer points at. *(Ch 4)*

destructor — the special member function that runs when an object is destroyed. Written `~ClassName()`. *(Ch 5)*

deterministic destruction — the guarantee that an object is destroyed at a known point, the closing brace of its scope, in reverse order of construction. *(Ch 3)*

duck typing — the C++ template rule that any type works if it supports the operations used; there is no declared constraint. *(Ch 8)*

dynamic storage — the heap. Memory you request explicitly and must release explicitly. *(Ch 3)*

`end()` — an iterator one past the last element; never dereference it. *(Ch 9)*

`std::exception` — the base of the standard hierarchy; guarantees only `what()`. *(Ch 10)*

exception safety — the property that a function leaves the program in a valid state even when an exception passes through it. *(Ch 10)*

executable — the native, runnable output of the link stage. *(Ch 1)*

fixed-width type — an integer type whose size is guaranteed by name: `uint8_t`, `int32_t`, and so on. *(Ch 2)*

fragmentation — free heap memory broken into pieces too small to satisfy a large request, even though the total is sufficient. *(Ch 3)*

frame — one call's slice of the stack: its parameters, locals, and return address. *(Ch 3)*

function template — a template that generates functions. *(Ch 8)*

`gcount()` — the number of characters the last unformatted read actually extracted. *(Ch 11)*

`ifstream` / `ofstream` — RAII input and output file streams; the constructor opens, the destructor closes. *(Ch 11)*

implementation-defined — the standard permits a choice and requires the implementation to document it. The size of `int` and `long` is implementation-defined. *(Ch 2)*

include guard — `#pragma once`, or the `#ifndef`/`#define`/`#endif` idiom, preventing a header from being pasted twice into one translation unit. *(Ch 2)*

instantiation — the generation of a concrete function or class from a template for a particular set of arguments. *(Ch 8)*

integer promotion — the automatic widening of anything narrower than `int` to `int` before arithmetic. *(Ch 2)*

integration test — a test exercising several pieces working together. *(Ch 10)*

Interface Segregation — keep interfaces small so implementers are not forced to write functions they do not need. *(Ch 7)*

`std::ios::binary` — the open flag that disables newline translation; mandatory on Windows for binary files. *(Ch 11)*

iterator — a generalised pointer supporting `*`, `++`, and `!=`. *(Ch 9)*

iterator invalidation — the loss of validity of iterators, pointers, and references when a container reallocates or erases. *(Ch 9)*

`join()` — wait for a thread to finish. `detach()` — abandon it. One is mandatory before destruction. *(Ch 12)*

lambda — an unnamed function object written inline, with a capture list saying what it takes from the enclosing scope. *(Ch 9)*

leak — heap memory that can no longer be reached and will never be freed. *(Ch 3)*

link — the build stage that stitches object files and libraries into an executable. Source of `undefined symbol` errors. *(Ch 1)*

Liskov Substitution — a derived object must be usable wherever its base is expected, without the caller knowing. *(Ch 13)*

`std::lock_guard` — RAII for a mutex: locks on construction, unlocks on destruction. *(Ch 12)*

`std::logic_error` — an error representing a bug: a precondition the caller broke. *(Ch 10)*

macro — a name the preprocessor replaces with text. *(Ch 2)*

`std::make_unique<T>(...)` — allocates a `T` on the heap and returns a `unique_ptr<T>` owning it. Preferred over `unique_ptr<T>(new T(...))`. *(Ch 6)*

mask — a value whose set bits select the bits you intend to operate on, usually built with `1 << position`. *(Ch 2)*

member initialisation list — the `: a_(x), b_(y)` clause that initialises members before the constructor body runs. *(Ch 5)*

memory-mapped I/O — reaching hardware by reading and writing fixed memory addresses. *(Ch 4)*

`std::move` — a cast meaning *"I am finished with this, you may take its guts."* It performs nothing at run time. *(Ch 6)*

`std::mutex` — a lock permitting one holder at a time. *(Ch 12)*

MVC — model (data and rules), view (presentation), controller (input and coordination), kept separate. *(Ch 13)*

`new` / `new[]` — allocate on the heap and return a pointer to it. *(Ch 3)*

`noexcept` — a promise that a function will not throw; breaking it calls `std::terminate`. *(Ch 10)*

non-type parameter — a template parameter that is a compile-time *value*, such as `std::size_t N`, which becomes part of the type. *(Ch 8)*

object file (`.o`) — machine code for one translation unit, not yet runnable. *(Ch 1)*

object slicing — copying a derived object into a base-typed value, discarding the derived part including the vtable pointer. *(Ch 7)*

Open/Closed — open for extension, closed for modification: add a class rather than editing a working function. *(Ch 13)*

optimisation level — a compiler flag such as `-O0` or `-O2` controlling how aggressively code is transformed. *(Ch 13)*

`override` — an optional keyword asserting that a function overrides a base virtual. Always write it; it turns a silent mistake into a compile error. *(Ch 7)*

padding — unused bytes the compiler inserts between members to satisfy alignment. *(Ch 5)*

pass by reference — passing an alias, so the function operates on the caller's object. *(Ch 4)*

pass by value — passing a copy, so the caller's object cannot change. *(Ch 4)*

placement `new` — constructs an object in memory you already have, allocating nothing. *(Ch 6)*

pointer arithmetic — arithmetic on pointers, scaled by the size of the pointed-to type. *(Ch 4)*

pool allocator — a fixed block of storage divided into equal slots, handed out in constant time without touching the heap. *(Ch 6)*

`#pragma once` — the one-line include guard used in this course. *(Ch 2)*

`#pragma pack` — a directive removing padding, so a struct matches an external byte layout exactly. *(Ch 5)*

preprocess — the build stage that expands `#include`s and macros by pure text substitution. *(Ch 1)*

pure virtual function — a virtual function declared `= 0`, with no body, that derived classes must implement. *(Ch 7)*

RAII — Resource Acquisition Is Initialisation: acquire in the constructor, release in the destructor, and let scope guarantee the release. *(Ch 5)*

random-access iterator — an iterator that can jump by n; required by `std::sort`. *(Ch 9)*

read-modify-write — reading a register, changing part of it, and writing it back. What `|=`, `&=`, and `^=` each perform. *(Ch 2)*

reference count — the number of `shared_ptr`s currently owning an object. Copying increments it; destruction decrements it. *(Ch 6)*

`REQUIRE_THROWS_AS` — the Catch2 assertion that a given expression throws a given exception type. *(Ch 10)*

rule of five — the rule of three plus the move constructor and move assignment operator. *(Ch 6)*

rule of three — if you write any of destructor, copy constructor, or copy assignment, you almost certainly need all three. *(Ch 6)*

rule of zero — design so you write none of the five, by holding resources in types that manage themselves. *(Ch 6)*

run time — the period during which the finished executable executes. *(Ch 1)*

runtime — a piece of software that manages a program as it executes, such as C#'s CLR. C++ has none. *(Ch 1)*

runtime polymorphism — dispatch decided while the program runs, via virtual functions. *(Ch 7)*

`std::runtime_error` — an error representing the world misbehaving. *(Ch 10)*

`std::shared_ptr` — a smart pointer allowing multiple owners; the object is destroyed when the last one goes away. *(Ch 6)*

Single Responsibility — a class should have one reason to change; the test is *what would make this change? (Ch 13)*

spurious wakeup — a `wait` returning without a notification, which is why `wait` takes a predicate. *(Ch 12)*

stack unwinding — the destruction of every fully-constructed local, in reverse order, as an exception propagates. *(Ch 10)*

static storage duration — storage that lives for the whole run of the program, as given by `static`. *(Ch 3)*

`static` member — a member belonging to the class rather than to any instance, existing once and shared by all. *(Ch 5)*

`steady_clock` — a monotonic clock suitable for measuring elapsed time. *(Ch 13)*

`std::streambuf` — the byte-moving half of a stream, separate from the formatting half. *(Ch 11)*

strict aliasing — the rule that reading an object through a pointer of an unrelated type is undefined behaviour. *(Ch 11)*

structured binding — `auto [a, b] = pair;`, unpacking into named variables. *(Ch 9)*

`T* const` — a pointer that cannot be moved. What it points at may still be modified. *(Ch 4)*

template — a pattern from which the compiler generates concrete functions or classes. *(Ch 8)*

`this` — inside a member function, a pointer to the object the function was called on. *(Ch 5)*

thread — an independent flow of execution within a process. *(Ch 12)*

`throw` — abandons the current function and begins unwinding the stack. *(Ch 10)*

translation unit — one source file after preprocessing, with every `#include` and macro expanded. The actual input to the compiler. *(Ch 1)*

type parameter — a template parameter standing for a type, written `typename T`. *(Ch 8)*

`uint8_t` — an exactly-8-bit unsigned integer. The register type for this course. *(Ch 2)*

`undefined symbol` — the linker found a call to something no object file defines. GCC words this as `undefined reference`. *(Ch 1)*

`std::unique_lock` — a lock guard that can be released and re-acquired; required by `condition_variable::wait`. *(Ch 12)*

`std::unique_ptr` — a smart pointer expressing exclusive ownership. Non-copyable, movable, and the same size as a raw pointer. *(Ch 6)*

unit test — a test exercising one class or function in isolation. *(Ch 10)*

use-after-free — reading or writing memory through a pointer after it has been deleted. *(Ch 3)*

virtual destructor — a destructor declared `virtual`, so deleting through a base pointer runs the derived destructor too. *(Ch 7)*

`volatile` — tells the compiler a location may change outside the program's control, so accesses must not be optimised away. Not a synchronization mechanism. *(Ch 4)*

vtable — a per-class table of function addresses used to dispatch virtual calls. *(Ch 7)*

vtable pointer — the hidden pointer added to every instance of a polymorphic class, costing 8 bytes on a 64-bit machine. *(Ch 7)*

`std::weak_ptr` — a non-owning observer of a `shared_ptr`'s object; must be `lock()`ed to be used. *(Ch 6)*

wire format — the byte layout of a record on disk or on a link, specified independently of any struct. *(Ch 11)*